Routing the Silent Half: Per-Unit Choice of Estimator and Evolution Target in Self-Evolving LLM Agents

Abstract

Group-relative policy optimisation assigns zero advantage to any group whose samples score alike, so a unanimous group contributes exactly nothing to the update. We measure this directly: 57.4% of groups in a live run are RL-silent. We show the silent half splits into two regimes that require opposite responses, and that the split is not symmetric — on the solved half a supervised target is *free*, because the group already contains a correct sample; on the unsolved half no target exists unless one is supplied, which makes harness evolution the only consumer that can use those units at all. We therefore make the choice of estimator and the choice of evolution target a per-unit decision rather than a per-run schedule, with the two as orthogonal axes so a single trajectory can feed both. The closest prior work, DAPO's dynamic sampling, acts on exactly this set and responds in the opposite way — it discards the silent groups and oversamples to refill — which on our own measurements costs 1.5–2.4× the rollouts, a multiplier that *grows* with training because the silent fraction itself grows from 0.33 to 0.58 on an arm with no routing at all. We report the negative result that located this leverage: a token-level version of the same idea, reaching 1.7% of tokens, moved held-out accuracy by less than the noise floor. Every capability is flag-gated and reduces to unmodified GRPO when disabled.

1 Method

1.1 Where the gradient is zero

Let a *unit* be a prompt with G sampled answers and binary rewards $r_1, \dots, r_G \in \{0, 1\}$. GRPO centres rewards within the group,

$$A_i \;=\; r_i - \bar{r}, \qquad \bar{r} \;=\; \frac{1}{G}\sum_{j=1}^{G} r_j, \tag{1}$$

so if every sample in the group scores alike then $A_i = 0$ for all i and the unit contributes exactly nothing to the update — all G sequences, every token. Writing p for the per-sample success probability, the probability that a unit carries any signal at all is

$$I_{\mathrm{RL}}(p, G) \;=\; 1 - p^G - (1-p)^G, \tag{2}$$

which is zero exactly at $p = 0$ and $p = 1$ and is maximised at $p = \frac{1}{2}$. Equation (2) is not an approximation: it is the probability that the group is not unanimous, and unanimity is precisely the condition under which (1) vanishes.

Two consequences drive the method.

The silent fraction is large and measurable. Instrumenting a live run (Sec. 3) puts $\hat{P}(\text{silent})$ between 0.29 and 0.57 depending on the training step, at 64 groups per batch: a third to a half of every batch is RL-dead. Even the low end is an order of magnitude above the reach of the token-level shared-prefix channel we measured first (1.7%), and that gap explains, without appealing to noise, why an intervention confined to that channel moved nothing.

The two silent regimes need opposite responses, and they are not equally common. A unit silent because $\hat{p} = 1$ and a unit silent because $\hat{p} = 0$ are both invisible to (1), but they are not the same object, and routing them identically wastes one of them. Measured on the control arm, the silent channel is 87.5% solved (Table 4), which decides which of the two branches carries the method.

The composition is a property of the task *and* the model, and it moves in both directions. The 87.5% is GSM8K with Qwen2.5-1.5B-Instruct; on MATH, holding the model fixed, it inverts to 39.1% solved and 60.9% unsolved (Table 6). Re-measured on the arm this paper trains — decontaminated DeepMath with Qwen2.5-32B-Instruct, over 245 logged steps of the baseline — the channel is 0.505 of all groups and splits 0.338 solved, 0.167 unsolved and 0.000 unclassified, i.e. 66.9% solved and 33.1% unsolved. The unsolved share is therefore roughly *half* what the MATH column reports, and we state the confound plainly: that pair varies the model as well as the task, so it cannot separate a task effect from a scale effect, and both readings shrink the branch. Two consequences are load bearing. First, any statement sized on the 60.9% figure must not be carried onto this arm: the unsolved branch is about a sixth of every batch here, not a third. Second, the 0.000 unclassified mass means no group on this arm is silent for a reason the correctness split cannot express, so unlike on GSM8K the composition ratio has a clean denominator and unsolved/silent and unsolved/(solved + unsolved) coincide.

1.2 A free target exists on exactly the solved half

The asymmetry is sharper than "different regimes". For a unit with $\hat{p} > 0$, at least one sampled answer was correct, and that answer *is* a supervised target drawn from the model's own rollout. No external teacher is required; this is rejection-sampling fine-tuning. Define

$$\text{selfTarget}(u) \;=\; \mathbb{1}[\hat{p}_u > 0]\,, \qquad \text{target}(u) \;=\; \text{selfTarget}(u) \;\vee\; \text{teacher}(u). \tag{3}$$

The units on which $I_{\text{RL}} = 0$ because $\hat{p} = 1$ are therefore exactly the units on which a target is *guaranteed* to exist at zero marginal cost. The units on which $I_{\text{RL}} = 0$ because $\hat{p} = 0$ are exactly the units on which no target exists unless one is supplied. The gradient-free half of the batch splits cleanly into a half that is free to reuse and a half that is not reusable at all.

1.3 Supplying a target for the half that has none

"Not reusable at all" is a statement about (1), not about the batch. An advantage is a coefficient on tokens the model *emitted*, so on a unit with $\hat{p} = 0$ every value that could be written into the advantage tensor multiplies a wrong derivation, and a positive constant there is a step toward the error — which is why $c_- \leq 0$ is required and why no router in our registry reaches this branch. The only altitude at which a correct row can receive gradient is *batch construction*: put the correct response in as a row, with its own prompt and its own loss mask, and the estimator that already runs on every other row runs on it too. This is the construction of LUFFY and of DyME, and it is what makes the unsolved branch addressable at all.

The supplier is an axis, not a constant. Once the row enters by construction, where it came from is a separate question from where it goes: the splice, the masks, the counters and the off-policy treatment are properties of inserting a correct row and do not depend on its origin. We therefore define a supplier interface with four implementations — the dataset's gold solution; a *self-generated* correct rollout of the same prompt from an earlier step or a higher-temperature resample; a row from an offline *corpus* of solved examples; and a verified completion from a stronger *teacher* — and route a unit to one of them. Prompt identity is derived from the prompt tokens rather than from a batch-local index, because a self-generated row is by definition one the same prompt received at a different step.

A supplier that has nothing must refuse, and the refusal is the measurement. Each supplier either returns token ids or raises a typed refusal, and every reason has its own counter that is reported whether or not it is zero: a missing dataset solution, a prompt never solved before, a derivation too long for the row, and a teacher completion that fails verification have four different remedies, and a single "skipped" total sends the reader to the wrong one. No supplier substitutes a row belonging to a different prompt, and no supplier truncates: a cut-off derivation is a wrong target that still looks like a target. The teacher requires an explicit verifier for the same reason — a stronger model is not a correct one, and this is precisely the branch on which nothing else would catch the error.

The source decision is fixed, and its control is not optional. The choice of supplier is expressible per unit, but we do not learn it here. We use an explicit ordered policy, cheapest-and-most-trusted first, and compare it against a control that replays the policy's own realised source assignment permuted across units, so the mixture of sources is matched exactly by construction and only the *targeting* differs. This is the same falsification the routing control performs one axis over, and we run it for the same reason: in this project a learned mechanism has repeatedly matched a feature-blind control at matched proportions once the control was finally run, and a claim about choosing sources that has not been separated from a claim about mixing them is not a claim.

1.4 Routing: the harness is a destination, not a schedule

Prior work treats the choice of what to evolve as a property of the *run*: a schedule or a global flag. We make it a property of the *unit*, and we make it two-dimensional. A routing decision is a pair

$$\pi(u) \;=\; \big(m(u),\, h(u)\big), \qquad m \in \{\textsc{rl}, \textsc{sft}, \textsc{skip}\}, \qquad h \in \{\textsc{none}, \textsc{propose}, \textsc{validate}\}, \tag{4}$$

where m names the estimator applied to the unit's tokens and h names what the unit contributes to harness evolution. The two axes are orthogonal by construction, so a single trajectory can feed the gradient and the harness at once. This is deliberate: the two consumers read different things from a trajectory — the estimator reads the tokens, the harness evolver reads the failure mode — and a partition discards one of the readings. Co-Harness's success→model / failure→harness rule is the special case in which π is constrained to be a partition, and we retain it as an ablation (`partition=True`) rather than a rewrite.

The rule implied by (2) and (3) is then forced rather than chosen:

$$\pi(u) \;=\; \begin{cases} (\textsc{sft}_{\text{self}},\ \textsc{validate}) & \hat{p}_u = 1 \quad \text{(RL dead; target free)} \\ (\textsc{sft}_{\text{teacher}},\ \textsc{propose}) & \hat{p}_u = 0 \ \wedge\ \text{teacher}(u) \\ (\textsc{skip},\ \textsc{propose}) & \hat{p}_u = 0 \ \wedge\ \neg\,\text{teacher}(u) \quad \text{(harness is the only consumer)} \\ (\textsc{rl},\ \textsc{none}) & 0 < \hat{p}_u < 1 \quad \text{(RL has signal; leave it alone)} \end{cases} \tag{5}$$

Note the third case. When a unit is unsolved and no teacher is available, the model arm has nothing to learn from it, and any method that only evolves weights must discard the unit. Routing it to the harness is what makes it recoverable at all, which is the concrete sense in which the harness axis is not a convenience.

One trajectory, two consumers. (5) as written still couples the two axes: a unit that RL can learn from contributes nothing to the harness. That is a residual partition, and it discards information — a *mixed* unit contains failed samples, and their failure mode is exactly what a harness evolver reads, while the RL update consumes only the reward. We therefore decouple the harness action with its own threshold τ_h:

$$h(u) \;=\; \begin{cases} \textsc{validate} & \hat{p}_u = 1 \\ \textsc{propose} & \hat{p}_u \le \tau_h \qquad \tau_h \ge \tau_{\text{fail}}, \\ \textsc{none} & \text{otherwise}, \end{cases} \tag{6}$$

so at $\tau_h = \tau_{\text{fail}}$ the rule is unchanged, and raising it lets a unit take $(\textsc{rl}, \textsc{propose})$ — feeding the gradient and the harness from the same trajectory. Under `partition` this is disabled, so the Co-Harness special case is preserved exactly.

Rollback. Every capability above is gated. With the harness arm disabled, $h(u) \equiv \textsc{none}$; with routing disabled, $\pi(u) \equiv (\textsc{rl}, \textsc{none})$ for every unit and the update is bit-identical to unmodified GRPO; with the supplier axis of Sec. 1.3 disabled, the batch construction reduces to its single original source and its output is byte-identical, key set included, which we verify against a digest recorded before the generalisation existed rather than by inspection. Each ablation in Sec. 3 is one flag away from the vanilla baseline, so a difference cannot be attributed to an incidental change of configuration.

1.5 Who decides: from a threshold to a controller

Everything above specifies an action space. What chooses within it is a separate question, and it is where the contribution lies — a rule keyed on $\hat{p}$ is a heuristic, and a learned policy that sees only $\hat{p}$ cannot beat the best threshold on $\hat{p}$ by more than noise. We therefore give the controller features, and instantiate it three ways so the comparison is an ablation rather than a demonstration.

Features, at zero marginal cost. Each unit is described by seven statistics computed from tensors the batch *already carries* — rewards, the loss mask, and the sampled log-probabilities. No extra forward pass, no extra generation. The constraint is deliberate: a feature set needing its own inference budget would have to beat spending that budget on more rollouts, which is a far higher bar than beating a threshold. Two of the seven carry information $\hat{p}$ cannot: the *truncated fraction* separates a unit that failed from one that merely ran out of budget, and the *log-probability dispersion* separates a group unanimous in its answer from one also unanimous in its reasoning. A solve-rate threshold collapses both distinctions by construction.

Three controllers, one action space.

- **Rule** — thresholds on $\hat{p}$, as in (5). The baseline the others must beat, not a strawman: it encodes the exact zero-gradient condition.

Algorithm 1 Per-batch signal routing. $\mathcal{B}$ is a batch of prompts, G samples each.

Require: thresholds τ_{solve}=1, τ_{fail}=0, $\tau_h \geq \tau_{\text{fail}}$; weights $c_+ \geq 0$, $c_- \leq 0$; flags `routing`, `harness`

1: roll out G samples per prompt; score to binary rewards $r_{u,i}$
2: $\hat{p}_u \leftarrow \frac{1}{G}\sum_i r_{u,i}$ ▷ group solve rate
3: $A_{u,i} \leftarrow r_{u,i} - \hat{p}_u$ ▷ group-centred advantages, (1)
4: $\mathcal{S} \leftarrow \{u : \max_i |A_{u,i}| = 0\}$ ▷ RL-silent: exactly the unanimous units
5: **if** `routing` **then**
6: **for all** $u \in \mathcal{S}$ **do**
7: **if** $\hat{p}_u \geq \tau_{\text{solve}}$ **then** ▷ solved: a correct sample is its own target
8: $A_{u,i} \leftarrow A_{u,i} + c_+$ for all response tokens
9: **else if** $\hat{p}_u \leq \tau_{\text{fail}}$ **then** ▷ failed: no self-target exists
10: $A_{u,i} \leftarrow A_{u,i} + c_-$ **if** $c_- \neq 0$, else leave at 0
11: **end if**
12: **end for**
13: **end if**
14: **if** `harness` **then**
15: $\mathcal{P} \leftarrow \{u : \hat{p}_u \leq \tau_h\}$; $\mathcal{V} \leftarrow \{u : \hat{p}_u = 1\}$
16: emit $(\textsc{propose}, u)$ for $u \in \mathcal{P}$ and $(\textsc{validate}, u)$ for $u \in \mathcal{V}$
17: **end if**
18: PPO update on $\{A_{u,i}\}$, clipping unchanged

- **Learned weights** — LinUCB, one ridge model per mode over the features, updated from observed outcomes. Ridge rather than a network because the outcome channel is confounded and low-signal — one scalar per batch, produced by every decision in it — and capacity would mostly fit that noise.
- **Learned code** — the policy is generated *source*, validated against an allowlist and cost-vetted in a subprocess before adoption. Strictly larger hypothesis class than weights: it can compose features into ratios and nested cases rather than only reweighting them.

The control that decides the claim. A fourth arm shuffles which units receive which mode while holding the *proportion* of units in each mode fixed. Without it, a controller that helps cannot be distinguished from a change in how much SFT the run did — and the latter is not a controller at all. Any improvement reported for the learned arms is the improvement over this control, not over the fixed rule alone.

What we verify about the learned arm, and what we do not. On a synthetic environment whose two regions differ *only* in the truncated fraction, with $\hat{p}$ held identical, the contextual controller reaches the optimal mode in both regions across seeds, while a context-free bandit on the same stream provably cannot exceed 0.5 in the worse region. That establishes the mechanism can use a feature a threshold cannot see. It does *not* establish that it learns under the confounded batch-level channel of a real run, which is a separate claim and is not made here.

1.6 The procedure, end to end

Algorithm 1 is the whole method. Every line is either an existing GRPO step or a decision from (5)–(6); nothing else is added, and with routing disabled lines 6–13 do not execute.

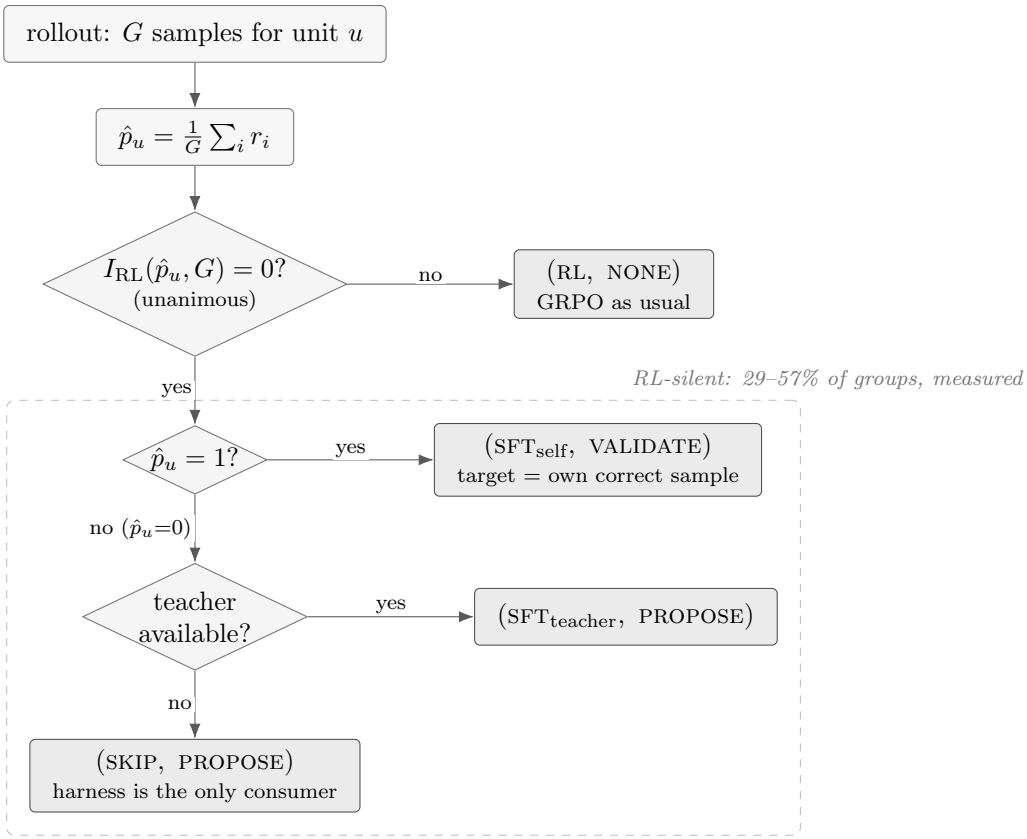


Figure 1: **Routing decides, per unit, both an estimator and a harness contribution.** The branch on I_{RL} is exact, not heuristic: a unanimous group has every advantage identically zero by (1). The dashed region is the part of the batch ordinary GRPO cannot learn from, measured at 29–57% of groups depending on training step. It does not split evenly: 87.5% of it is solved, where a supervised target is free (the unit's own correct sample), and the remainder has no target unless one is supplied — and on that remainder the harness is the only consumer that can use the unit at all.

Why the update needs no new loss term. Line 8 is the whole of the supervised branch. For a solved unit every $A_{u,i}$ is zero, so adding c_+ makes the objective $c_+ \sum_i \log \pi(y_{u,i})$ over samples already known to be correct — which is supervised fine-tuning on them, with the importance ratio at 1 because the data is on-policy. PPO's clip still bounds the step, which matters because sharpening an already-correct policy is the mechanism's predicted failure. Line 10 is its mirror: $c_- \leq 0$ pushes down samples known to be wrong, and $c_- > 0$ is rejected rather than trusted, since it would train the model to reproduce them.

Cost. Lines 5–13 add no generation. The batch is the batch that GRPO already rolled out; the silent units are re-read, not re-sampled. This is the whole of the difference from DAPO, which reaches the same set by discarding it and paying to generate a replacement.

2 Related work

We organise prior work by the claim it already owns, because our contribution is only meaningful as the complement of those claims.

Discarding the silent groups. DAPO's dynamic sampling is the closest prior work, and it is close in the strongest sense: it acts on exactly the set we act on. Its rule keeps a prompt only when the group's reward standard deviation is positive, and a zero standard deviation is unanimity, which by (1) is precisely the zero-advantage condition. DAPO therefore *detects* the silent channel and *discards* it, oversampling until the batch refills. We reuse it. Same phenomenon, opposite response — so DAPO is the baseline this claim must be measured against rather than a method to cite in passing, and Sec. 3.6 reports what its response costs.

Choosing what to update. SIA is the nearest work on the *decision* axis, and our earlier reading of it was wrong in a way worth stating rather than silently fixing: it is not task-level algorithm selection. It is a feedback agent that updates *both* the harness and the weights, it is built on a 120B base, and it reports no mathematics or code benchmark at all. Two consequences follow. Its base is outside what our stack can train, and its evaluation domains do not intersect ours, so a head-to-head on its own benchmark would confound the method with the base model and with the domain. The defensible comparison is its rule reimplemented on our base at matched budget, and that is how we report it. The granularity argument itself is unaffected: what a task-level assignment cannot express is that units *within* one task differ in whether a gradient exists at all, since by (2) the silent and informative units of a single task require different estimators.

Deciding what to evolve. Co-Harness co-evolves harness and weights and establishes the success→model / failure→harness assignment. EvoTrainer co-evolves a policy with its training harness. Both fix the assignment as a rule over outcomes. We keep their rule as a special case ((4) constrained to a partition) and relax it, because a trajectory read by two different consumers need not be spent on only one of them.

Evolving the reward itself, and the anchor that is missing. Two recent systems bracket this question and, between them, leave the interesting case open. BigBang-v1 closes a generator–critic loop in which a code agent rewrites the *data-synthesis program* while a critic filters what it produces, and a meta-critic corrects the critic against the training outcome it failed to predict; the loop is explicitly anchored on held-out real-task utility. It has no reward function anywhere, and no reinforcement learning. Ornith-1.5 is its mirror image: the harness it generates per task *is* the reward function, and that harness is itself optimised by policy gradient against a score that includes a hack-resistance term. An evolving reward is therefore already shipped, and we do not claim it. What neither provides is the combination. BigBang has the anchor and nothing that moves; Ornith moves the reward and states no external anchor inside its loop. Its own predecessor held the verifier immutable and outside the model's reach, and the successor — which now *generates* the object that grades it — does not say what replaced that boundary, while its hack-resistance term is named without being given a form or a scorer. An evolving reward anchored on a fixed external utility is thus unoccupied, and it is the claim we make.

What neither system decides. Neither has a router. BigBang's critic has a binary codomain — a candidate is admitted, revised, or dropped — and nothing about *how* to learn from an admitted sample varies: one loss and one source throughout. Ornith operates at whole-task granularity, which is the finest thing it does, and what varies per task is artifact content rather than the learning mechanism. It states that per-category strategy emerges rather than being chosen, and it does not test that claim against a frozen or randomised scaffold, which is precisely the absorption control we impose on ourselves and therefore run against our own arms. Neither system conditions on

behavioural clustering features or on trajectory statistics, which is where our decision variable is read.

Deciding *when* to evolve. *Learning, Fast and Slow* separates fast and slow adaptation and reports the stability benefit of a two-timescale schedule. That bounds what a cadence claim can be worth, and we treat cadence as a control variable rather than a contribution.

Clustering trajectories. MEDS reuses layer-wise logits from the forward pass as lightweight representations of reasoning behaviour and clusters error patterns with HDBSCAN. We adopt this directly as the learned instantiation of our cluster key, against a derived key computed from the silence side of (2); the two are a controlled ablation of *how* to cluster, holding what is done with the clusters fixed. *Scope, narrowed by measurement.* A four-partition probe on exact per-cluster gradients finds MEDS clusters indistinguishable from a size-matched random partition, with the sign of the separation flipping across settings. We therefore do not claim that MEDS separates conflicting updates, and we do not build per-cluster experts on it. The use that survives is as a targeting key for *where* a supplier should act, which requires only that clusters identify consistent error patterns rather than separable gradients, and which is a weaker and separately testable condition. A confirmatory run at a later checkpoint is in flight.

Policies as weights and as code. Contextual bandits are the standard tool for per-instance algorithm selection under a scalar reward, and we use LinUCB unmodified; the contribution is not the estimator but what it is given to see. Generating a policy as executable source is the other established line, and we adopt it as a distinct arm rather than an alternative implementation, because the two differ in hypothesis class rather than in optimiser: weights reweight the features they are handed, code composes them.

Evaluation practice. Recent evaluation work documents how often agent results fail to survive controls that were available at the time. We adopt its checks as requirements rather than as discussion: paired tests on shared items, intervals reported on the *difference* rather than on each arm, matched-permutation controls that hold the proportion of units in each mode fixed while shuffling which units get which mode, and a noise floor established from the same checkpoint scored twice. The matched control is the one that decides our central claim, because a routing method changes both which units are routed and how many, and only the permutation control separates those.

3 Results

All numbers below are measured end-to-end on held-out benchmarks. Train reward is never reported as a result: Sec. 3.1 is the demonstration that it mis-orders checkpoints.

3.1 Train reward mis-orders checkpoints

Table 1 scores two recipes that differ only in optimiser aggressiveness, on the full MATH-500, paired by item. The aggressive ("demo") recipe's training reward rises while its held-out accuracy falls by 0.194 absolute. Any claim validated on training reward would have selected the collapsed checkpoint.

Table 1: MATH-500, $n = 500$, paired McNemar against the scaffold arm. Runs `step0d` (demo, lr 6×10^{-6}, clip 0.4) and `step0h` (scaffold, lr 1×10^{-6}, clip 0.2).

step	demo	scaffold	Δ	McNemar p
base	0.528	0.506	-0.022	same model
28	0.454	0.530	$+0.076$	7.3×10^{-4}
57	0.440	0.530	$+0.090$	1.7×10^{-4}
86	0.466	0.526	$+0.060$	4.1×10^{-3}
115	0.364	0.536	$+0.172$	2.1×10^{-13}
144	0.334	0.522	$+0.188$	9.3×10^{-15}

Table 2: The same checkpoints on OlympiadBench (675 problems), which shares no items with MATH-500.

checkpoint	MATH-500	OlympiadBench
base	0.528	0.188
gs028	0.454	0.170
gs057	0.440	0.190
gs086	0.466	0.148
gs115	0.364	0.135
gs144	0.334	0.108
gs173	0.358	0.107

Noise floor, and what it disqualifies. The base model appears in both series, so its difference is pure measurement noise: 0.0220 at $n = 500$. It *rose* from 0.0080 at $n = 250$ because the larger series was scored in two passes on different server processes, accumulating between-run variance a contiguous sweep does not have. Consequently the $+0.060$ at step 86 is only 2.7× the floor and we do not report it standalone; the $+0.172$ and $+0.188$ are $\approx 8\times$ it. The claim is *preservation*, not improvement: the scaffold arm stays within $[0.506, 0.536]$, which brackets its own base.

3.2 The collapse replicates on an independent benchmark

$0.188 \to 0.107$ is a 43% relative drop in the same direction as MATH-500's $0.528 \to 0.358$, on an independent problem set.

Benchmark selection, stated as a measurement rather than a preference. MATH-500 is saturated at the frontier tier (0.966 at 27B) and AIME is unusable at both ends (0.000 at 1.5B; 0.867 at 27B with a 24-point interval on 30 problems). OlympiadBench resolves a few points on 675 items and leaves both ends off the rails: 0.188 at 1.5B and 0.716 for Ornith-1.5-35B-A3B. The 0.716 is a *lower* bound — 174/675 generations (26%) hit the token cap — so the headroom is smaller than the 28 points it suggests, and we quote it as a bound.

3.3 Negative result: token-level routing at 1.7% reach

Token-level routing changes nothing detectable. We report this rather than dropping it, because it is the measurement that located the method's leverage. The rule reaches at most 1.7% of loss-carrying tokens, and an intervention on 1.7% of the gradient cannot move a benchmark by more than noise; the between-arm noise floor here (0.030 on the shared base checkpoint) is itself larger than every routing difference in the table.

Table 3: MATH-500, $n = 500$. `step0j` adds token-level routing to the scaffold recipe. p is paired McNemar, scaffold vs. scaffold+routing.

step	demo	scaffold	+routing	p
base	0.528	0.506	0.536	0.092
28	0.454	0.530	0.516	0.534
57	0.440	0.530	0.526	0.920
86	0.466	0.526	0.522	0.923
115	0.364	0.536	0.496	0.065
144	0.334	0.522	0.526	0.922

Table 4: Decomposition of the RL-silent channel. `step0l`, GSM8K, Qwen2.5-1.5B-Instruct, first 18 logged batches.

quantity	mean	range
silent groups	0.3592	[0.2891, 0.4414]
silent because *solved*	0.3145	[0.2461, 0.4219]
silent because *unsolved*	0.0447	[0.0117, 0.0703]
solved share of the silent channel	**0.875**	[0.827, 0.959]

Confound, stated rather than buried. `step0j` differs from `step0h` in three ways — routing, `kl_ctl` ($0.01 \to 0$) and `adv_norm` (batch→off) — because the rule's precondition does not hold otherwise. A significant difference here could not have been attributed to routing alone. The routing-off arm at `step0j`'s exact configuration is therefore a required control, and it is running (`step0l`).

3.4 Where the leverage actually is

Measured live on `step0l` at 64 groups per batch:

$$\widehat{P}(\text{silent}) = 0.574, \qquad n_{\text{groups}} = 64.$$

More than half of every batch contributes exactly zero gradient — 34× the token-level channel, and the reason Sec. 3.3 was null is that the intervention was on the wrong tier, not that the idea was wrong.

A metric bug found and fixed, reported because the affected runs are still cited. The accompanying split first reported 0.000 solved at a measured 82% solve rate, which is impossible. It thresholded `reward_score`, which by that point has had bias, scaling, clipping and normalisation applied. It now reads the raw rewards captured before any transformation. Runs launched before the fix carry the old code and their solved/unsolved split must be ignored; $\widehat{P}(\text{silent})$ is unaffected because it is computed from the advantages being zero.

3.5 The silent channel is 87.5% solved, so most of it needs no teacher

Measured on the routing-off control (`step0l`) under the corrected metric, over 18 logged batches at 64 groups each (1152 group observations):

Table 5: Rollout cost of discarding versus reusing the silent channel. s is measured on the routing-off arm; the multiplier is $1/(1-s)$.

point in run	silent fraction s	DAPO	ours
first 10 batches	0.327	1.49×	1.00×
whole run	0.525	2.10×	1.00×
last 10 batches	0.583	2.40×	1.00×
max observed	0.633	2.72×	1.00×

The identity solved + unsolved = silent holds to a maximum residual of 1.0×10^{-5} across all 18 batches, which is float32 rounding. We check it because an earlier version of this metric failed it, reporting 0.000 solved at a measured 82% solve rate.

This inverts the cost of the method. By (3), the solved groups already contain a supervised target, so 31.4% of *all* groups are reusable with no teacher, no extra inference and no extra GPU. Only 4.5% of groups — the unsolved ones — require an external teacher, and those are exactly the units for which the harness is the only available consumer. The reach of the free path is therefore about 7× that of the teacher-dependent one, which is the opposite of the ordering we assumed before measuring.

Scope, and what this does not claim. This is one operating point: GSM8K, Qwen2.5-1.5B-Instruct, early training, high solve rate. A harder task moves mass from solved to unsolved and shifts the balance toward needing a teacher; 0.875 is a property of the operating point, not a constant. More importantly, the table says where the reachable mass *is*, not that training on it helps. Sharpening an already-correct distribution spends entropy, and entropy collapse is the failure mode Sec. 3.1 already documents. Until the A/B exists, the solved branch of (5) defaults to SKIP.

3.6 What discarding the silent channel costs

DAPO's response to the same channel is to drop it and regenerate. If a fraction s of groups is unanimous, refilling a batch requires generating $1/(1-s)$ groups per accepted group. Substituting the silent fractions measured on the control arm (98 batches) gives Table 5.

The gap widens with training. The silent fraction is not a constant: it rises from 0.33 to 0.58 within 98 batches. Crucially this was measured on the arm with *no routing at all*, so the growth is a property of RL training rather than of our intervention — as the policy improves, more groups become unanimous and GRPO's effective batch shrinks. An overhead that grows as the model gets better is the wrong shape for the regime frontier models occupy.

Stated as a prediction, not a result. $1/(1-s)$ is the expectation under the assumption that regenerated groups are unanimous at the same rate. The DAPO arm measures the true multiplier from the accept/reject counts rather than inheriting this estimate. We fix the comparison axis in advance as *matched generation budget*, not matched steps: DAPO's kept batch is denser per step, so an equal-step comparison would flatter it.

Table 6: The silent channel does not shrink on a harder task — it inverts. Qwen2.5-1.5B-Instruct in both columns.

	GSM8K	MATH
silent groups	0.359	0.419
silent because solved	0.315	0.164
silent because unsolved	0.045	**0.255**
solved share of the channel	**87.5%**	**39.1%**
unsolved share	12.5%	**60.9%**

3.7 The solved branch is inert, and then harmful

Acting on the free self-target does not help. At $c_+ = 0.5$ the effect is null at every checkpoint and the difference-in-differences after removing the base offset is 0.000–0.002 at four of five steps. At $c_+ = 2.0$ it becomes negative: -0.054 at step 144 ($p = 0.014$, 2.5× the noise floor). We report it as suggestive rather than conclusive — one hit in five paired tests does not survive a Bonferroni threshold of 0.01 — but the *shape* argues it is real: negative or zero at four of five steps with the largest effect at the *last* checkpoint, which is what cumulative over-sharpening looks like and is not what a random hit looks like. It agrees with an independently measured entropy trace in which the routed arm descends faster mid-run.

The mechanism is unflattering and simple: a group is solved *because* the policy already places high probability on the correct answers, so supervising on them teaches what the model already knows.

3.8 The composition of the silent channel inverts with task difficulty

If that were the whole story the channel would be worthless, because on GSM8K it is 87.5% solved. It is not the whole story. Holding the model and configuration fixed and changing only the training task:

The unsolved fraction of *all* groups rises from 4.5% to 25.5%, a 5.7× increase, while the solved fraction roughly halves. So the branch we measured to be worthless shrinks to a minority exactly where the task gets hard, and the branch for which no self-target exists — and for which a teacher or the harness is the only possible consumer — becomes the majority.

This is what the routing rule of (5) is for. A fixed threshold tuned on GSM8K would spend its effort on the solved side, which is where the effort is wasted; keying on the *side* of silence rather than on a tuned constant is what survives the shift. We state the limits plainly: $n = 11$ batches for the MATH column against 98 for GSM8K, one model at one scale, and — the lesson of Sec. 3.7 — locating mass is not the same as showing that acting on it helps.

3.9 The learned router does not beat its matched control

Sec. 1.5 names the control that decides this claim: `router=random` run at the contextual arm's *measured* mode proportions, so the two arms differ only in *which* unit receives which mode and not in the mixture. Those proportions were read off the last 40 steps of the contextual run (`ctx`, 64 groups per step) — rl 0.2946, sft 0.3527, skip 0.3526 — and the control (`rnd`) was run at exactly them. Both arms are scored from `globalstep149` of their own run, GSM8K training, Qwen2.5-1.5B-Instruct, greedy decoding (temperature 0, seed 0). That step is the matched comparison point rather than a

Table 7: Learned router (`ctx`) against its matched-proportion control (`rnd`), both at `globalstep149`, greedy. Wilson 95% intervals in brackets. MATH-500, AMC23 and AIME are scored at `max_tokens`=3072 and OlympiadBench at 16384; arms are compared only against each other at the same budget.

benchmark	n	ctx	rnd	ctx − rnd
MATH-500	500	0.5240 [0.480, 0.567]	0.5260 [0.482, 0.569]	−0.0020
OlympiadBench	675	0.1837 [0.156, 0.215]	0.1837 [0.156, 0.215]	+0.0000
AMC23	40	0.1750 [0.087, 0.320]	0.1250 [0.055, 0.261]	+0.0500
AIME24	30	0.0000 [0.000, 0.114]	0.0000 [0.000, 0.114]	0.0000
AIME25	30	0.0000 [0.000, 0.114]	0.0333 [0.006, 0.167]	−0.0333

chosen one: the contextual run stalled at step 162 of 290 while the control reached 245, so 149 is the latest checkpoint the two arms share.

Stated positively rather than as an absence: **at matched mode proportions, the per-unit routing decision buys nothing over assigning the same modes at random.** What this routing arm sets is the *mixture*; which unit receives which mode carries no effect the suite can measure.

Which rows can carry that claim, and which cannot. Only the first two. MATH-500's −0.0020 is one tenth of the 0.020 systematic noise floor measured for greedy scoring on this suite (Sec. 3.1 measures 0.0220 on MATH-500 itself), and OlympiadBench returns the same score on both arms to four decimals over 675 problems. The other three rows resolve nothing: at $n = 40$ and $n = 30$ the Wilson intervals span more than 20 points, so AMC23's +0.0500 and AIME25's −0.0333 — one problem out of thirty — sit well inside them, and AIME24's 0.000 on both arms is this scale's known floor rather than a comparison. One caveat on the intervals themselves: they are per-arm Wilson intervals, not the paired intervals on differences this section reports elsewhere. The paired test was not run for this pair, so what bounds the two resolving rows is the noise floor, not an interval on the difference.

What a real effect on OlympiadBench would have to clear. A single greedy score at $n = 675$ carries about a point of jitter: the *same* `ctx` checkpoint on the *same* 675 problems scored 0.1941 at `max_tokens`=8192 and 0.1837 at 16384, with nothing changed but the budget. A future arm difference below roughly two points on this benchmark is therefore not evidence, and +0.0000 is far inside that. Both arms also leave generations that never emit a boxed final answer at any budget — 78/675 for ctx and 64/675 for rnd at 16384, against 79 and 61 at 8192 — so the truncation these rows carry is non-termination rather than a budget-bound score, and doubling the cap does not remove it.

Scope, and what this does not claim. One seed, one checkpoint, one training task, one model scale. It falsifies "this router, with this credit signal, beats its matched control". It does not show that routing cannot help, and by construction it says nothing about the mixture, on which the two arms are matched. Sec. 3.10 localises the failure, and the failure is not the bandit.

3.10 Why the router does not learn: the credit signal, not the bandit

This subsection and the next report the controller's own training-time behaviour — the mode mix it emits — and not capability. They are diagnostics of a mechanism, not results on a held-out

Table 8: Mode mix under per-batch credit. `ctx`, GSM8K, Qwen2.5-1.5B-Instruct, first 129 logged steps at 64 groups per step. L_1 is the distance of the quarter's mean mix from uniform thirds; 1.33 would be total concentration on one mode.

quarter (steps)	rl	sft	skip	L_1 vs. uniform
Q1 (1–32)	0.305	0.341	0.354	0.056
Q2 (33–64)	0.299	0.338	0.363	0.069
Q3 (65–96)	0.351	0.303	0.346	0.061
Q4 (97–129)	0.344	0.320	0.336	**0.027**

benchmark, and are labelled as such throughout.

Under per-batch credit the router never developed a preference. Table 8 gives its mode mix by quarter over the first 129 logged steps of `ctx`.

The mix stays at uniform thirds and the trend is flat to *decreasing* — the last quarter is the most uniform of the four. A learner should move *away* from uniform as its arm estimates separate; this does the opposite.

Feedback was flowing, not blocked. The absence is not a plumbing failure. Over those 129 steps the router received 128 feedback records, each covering all 64 units of its batch, with exactly one batch in the whole run refused as confounded (a batch whose units all took one mode carries no comparison, and its update is skipped rather than made). The batches were not degenerate either: the most common mode's share stays between 0.352 and 0.766; all three modes are present in 95 of the 128 records, and the count never falls below 2.25; and the weak-attribution flag, which fires when one mode exceeds 90% of the units, reads 0 in 108 of the 128 and never exceeds 0.5. Both counts are averaged over four data-parallel ranks, which is why they are fractional: a value below 3 means one rank's view of the batch was short a mode, not that the batch itself was. The router got 128 updates on batches that carried a comparison and formed no preference from them.

The mechanism, as an argument. LinUCB accumulates, for the arm m that was chosen,

$$A_m \leftarrow A_m + xx^\top, \qquad b_m \leftarrow b_m + r\,x, \qquad \theta_m = A_m^{-1} b_m.$$

Per-batch credit gives every unit in the batch the same r, and under a cold start that assigns modes independently of the context x, an arm chosen n_m times accumulates $A_m \approx n_m\,\mathbb{E}[xx^\top]$ and $b_m \approx r\,n_m\,\bar{x}$, so

$$\theta_m \approx \left(n_m\,\mathbb{E}[xx^\top]\right)^{-1} r\,n_m\,\bar{x} = \mathbb{E}[xx^\top]^{-1} r\,\bar{x},$$

which is *the same vector for every arm*: the per-arm count cancels. The arms start identical and stay near-identical, selection is left to the exploration bonus, and the mix stays uniform. This rules out "the router needs more steps": an uninformative signal does not become informative with repetition.

The mechanism, as a measurement. The argument is turned into a test in `selfevo/tests/test_credit_assi` which drives the same registry-built router for 60 batches of 64 units and changes *only* the credit (seed 0, the default the test runs at). Under a shared per-batch scalar the largest pairwise distance between two arms' parameter vectors is 0.124 and no mode takes more than 0.379 of the units (the test asserts < 0.5 and < 0.55). Under credit that depends on which mode was applied, the same router separates to 1.500 and the rewarded mode takes 0.989 of the units (asserted > 1.0 and > 0.5).

Table 9: Mode mix under per-prompt credit. `ctxpc`, GSM8K, Qwen2.5-1.5B-Instruct, first 85 logged steps at 64 groups per step; the run was still training when these were read. Compare L_1 against 0.027–0.069 for per-batch credit (Table 8).

quarter (steps)	rl	sft	skip	L_1 vs. uniform
Q1 (1–21)	0.905	0.046	0.049	**1.144**
Q2 (22–42)	0.381	0.405	0.214	0.239
Q3 (43–63)	0.000	0.353	0.647	0.667
Q4 (64–85)	0.006	0.431	0.562	0.655

Table 10: RL share of the batch under raw (`ctxpc`) and batch-mean-centred (`ctxpcc`) per-prompt credit, matched over the first 89 logged steps of each arm.

steps	raw	centred
1–15	0.867	0.870
16–30	0.933	0.933
31–89	0.003	0.001

Same bandit, same contexts, same number of updates, opposite outcome — so the null of Sec. 3.9 localises to the credit signal, not to the controller or its exploration.

3.11 Per-prompt credit changes the router's behaviour; centring the credit does not

The implied fix is to credit each decision with its own prompt's change in solve rate between the batch where it was routed and that prompt's next appearance, rather than with one scalar per batch. Table 9 is the same measurement as Table 8 on the arm that does this (`ctxpc`, identical to `ctx` except for the credit).

The mix stops being uniform, and it revises: L_1 swings between 0.239 and 1.144 where per-batch credit never left 0.027–0.069. Only the credit changed — same router, same features, same exploration — so the difference is attributable to the signal.

The transition is abrupt, and it coincides with the first credit. Per-prompt credit reaches nothing until a prompt recurs. `prompt_credit/observed_units` is exactly 0 for steps 1–28 and first non-zero at step 29 (4.25 of 64 units); the first step whose `rl_groups` is 0 is step 30, the step immediately after. Before the credit arrives the arm is not routing on evidence at all: after three cold-start steps near uniform thirds (0.352, 0.328, 0.332 rl), steps 4–29 sit at exactly 1.000 rl. That is the documented tie-break artifact — with no `observe` calls the arm parameters never move, the scores tie, and `argmax` resolves alphabetically to RL — so the pre-transition plateau is not a preference either.

Centring the credit changes essentially nothing, which kills the obvious confound. The credited value is a prompt's change in solve rate across roughly 29 steps of training, over which the policy improves generally; the signal therefore conflates "the model got better" with "this mode helped this prompt". Subtracting the batch's mean delta removes the common component. Matched over the first 89 logged steps of each arm:

The two arms are the same arm to within a third of a point in every window, and the common trend that centring removes is small: `prompt_credit/mean_delta` is exactly 0 for the 28 steps before

Table 11: One 30B checkpoint scored at the cap that was requested and at the cap that was applied. `cap asked` is what the command line requested — the same value in every row — and `cap used` is what the sampler received. Nothing else changed.

benchmark	n	cap asked	cap used	accuracy	truncated
AIME24	30	32768	8192	0.2667	73.3%
AIME24	30	32768	32768	**0.8000**	20.0%
AIME25	30	32768	8192	0.2667	73.3%
AIME25	30	32768	32768	**0.7000**	20.0%

credit flows and lies in 0.005–0.090 (median 0.049) afterwards. Removing the common trend does not move the behaviour, so **the training-progress confound is not the explanation** for the collapse away from RL.

The surviving hypothesis, labelled as one. What remains is that the transition is driven by the *exploration bonus* rather than by the credit values: the heavily-used arm is the first to have its A updated, so its confidence bonus is the first to shrink, and the untouched arms become relatively more attractive whatever the credit says. This predicts exactly the value-independence observed above, and it predicts self-correction once the bonuses equilibrate. The second prediction is at least not contradicted — the raw arm's rl share returns from 0.000 across steps 43–63 to a mean of 0.336 over steps 120–132 (0.26 at step 130) and 0.367 over steps 115–152, the last step logged when this was read — but that is an observation on a run that is still training, not a test. **This has not been tested.** Two things would settle it. Logging the two terms of the LinUCB score separately at the transition: bonus exhaustion requires that the point estimates $x^\top \theta_m$ are still tied when the ranking flips, and is dead if they had already separated in favour of SFT and SKIP before step 30. And re-running the same credit with the exploration coefficient at zero: if the transition survives, it was never the bonus.

Scope, and what this does not claim. One arm per credit variant, one seed, one task, one scale, and both runs unfinished at the step counts quoted. Nothing here is a capability claim: **neither credit arm has been scored on the held-out suite**, and a mix that moves is not a mix that helps — especially this one, which drives `rl_groups` to zero, so that no group receives an ordinary policy gradient and every group is either supervised on its own correct sample or skipped. Sec. 3.7 measures that branch inert at $c_+ = 0.5$ and harmful at $c_+ = 2.0$, which is a reason to expect this regime to cost capability rather than to gain it. Until a scored arm exists, this is a demonstration that the credit signal controls the router's behaviour, and nothing more.

3.12 A benchmark score is a property of the token budget it was taken at

The scoring harness resolved generation parameters by applying a per-benchmark defaults table unconditionally, so an explicit `--max-tokens` was parsed, accepted, and then discarded. A run made with the wrong budget still succeeds and still writes a plausible number, so the defect is invisible from the score alone. Table 11 re-scores one 30B checkpoint (`Frontis-MA1-30B`, a `Qwen3MoeForCausalLM` mixture of experts) after fixing the precedence: same model, same server, same concurrency, same grader, and the only thing that differs between the two halves of each block is which cap reached the sampler.

Table 12: Length-knee onset across six GSM8K runs on Qwen2.5-1.5B-Instruct. c_+ is the constant written onto solved groups. "steps" is the number of logged training steps available for the run, so the control rows record how long a crossing had to fail to appear.

run	group routing	c_+	steps	onset
`ctx`	on	0.5	162	144
`ctxpc`	on	0.5	152	132
`sa2`	on	2.0	145	131
`step0l`	off	—	273	none
`step0j`	off	—	178	none
`g16`	off	—	116	none

AIME24 moves by a factor of three and AIME25 by a factor of 2.6, from a flag that appeared on the command line of every earlier score and never reached the sampler.

Both corrected rows are still lower bounds. 20% of generations reach even the 32768 cap and are graded wrong, so 0.8000 and 0.7000 bound this checkpoint from below rather than measuring it. OlympiadBench carries a second and unrelated lower-bound caveat at every cap: 94 of its 675 problems state more than one gold answer inside a single answer string, which the exact-match grader marks wrong however the model answers.

What made the defect visible, and it was not the score. The corrected code prints the budget it resolved (`NOTE ... BENCH_OVERRIDES default 8192 not applied`), and the results row records the generation parameters the run actually used, so the requested cap can be compared against the applied one. Recording the parameters a run used is not bookkeeping; without it a cap defect produces no symptom at all, because the run still completes and the number it writes is still in range.

The general form, which outlives this particular bug. A default that silently outranks an explicit request is indistinguishable, from outside, from the request being honoured. We therefore state the convention rather than assume it: **a benchmark number is a property of the token budget unless the budget is reported**, and a comparison taken across two budgets is not a comparison.

3.13 Length moves like a threshold, and routing *presence* is what separates

Mean response length during training does not drift upward under routing; it holds a plateau and then breaks out of it. We fix the onset criterion before reading the runs: onset is the first step of five consecutive steps whose mean response length exceeds the run's own pre-onset baseline by more than four of that baseline's standard deviations.

All three runs with group routing configured hold a flat plateau for more than 120 steps and then cross; none of the three runs without it ever crosses, over 273, 178 and 116 steps. Slope brackets the change: 0.34–0.57 tokens per step before onset across the three routed runs, and 5.6–9.2 after.

The same separation appears in free generation. Scored outside training, greedily, from the checkpoint ladders: `ctx` grows 327 → 455 tokens, while `step0l` moves 330 → 316 and is flat over 202 steps.

Presence, not magnitude. We state the surviving claim in the narrowest form the data supports. Whether group routing is *configured* predicts whether a run crosses; the *magnitude* of the constant does not predict when it crosses, which is the negative result of Sec. 3.14. Three runs on each side is enough to notice a clean split and not enough to estimate an effect, and the six runs are not a matched sweep: `g16` trains at $G = 16$ where the others use $G = 8$, `step0j` carries the token-level rule of Sec. 3.3 and is unrouted only at the group level, and the runs were allowed different numbers of steps.

How each run was classified, because the obvious way is wrong. Routing status is read from `actor.group_routing` in each run's `config.yaml`, never from the presence of `route/*` metrics in its log. `sa2` emits no `route/*` metric at all while its config carries `group_routing.enabled: true` with `solved_advantage: 2.0` and `router: null` — the actor's fixed-rule branch writes the constant without passing through the metric seam. Classified from metrics alone, `sa2` would have been counted on the wrong side of Table 12, and it is the run carrying the larger constant.

3.14 Three negative results on the routed arms

(a) The learned router shows no preference over its matched random control. Sec. 3.9 is that result and we do not restate its table: at the contextual arm's own measured mode proportions the per-unit decision is worth -0.0020 on MATH-500 and $+0.0000$ on OlympiadBench, against a 0.020 noise floor, and Sec. 3.10 localises the cause to the credit signal rather than to the controller.

(b) The dose-response between constant magnitude and knee onset is not supported. We had recorded — and described as the strongest evidence available — that the larger the constant written onto solved groups, the earlier the length knee arrives. Under the onset criterion of Sec. 3.13 it does not hold. $c_+ = 0.5$ gives onsets at steps 132 (`ctxpc`) and 144 (`ctx`); $c_+ = 2.0$ gives 131 (`sa2`), which sits inside the spread of the two 0.5 arms rather than ahead of them. Three runs at two operating points cannot resolve a dose-response, and the earlier claim is withdrawn here rather than quietly not repeated.

(c) The runs that genuinely produced non-terminating output were the *unrouted* ones. Reading training-time `no_eos_ratios` across all 18 training runs logged for this study, the routed arm `ctx` peaks at 0.0059 — six of 1024 rollouts. The two runs that produced non-terminating output at any material rate are `step0b` (0.156) and `step0` (0.117), and **both have group routing off**. Both use the same aggressive hyperparameters as the demo recipe of Sec. 3.1 (lr 6×10^{-6}, clip 0.4), which that section already measures to destroy capability.

The caveat that keeps (c) from being stronger than it is. `no_eos_ratios` is mis-instrumented: it compares sequence lengths against the *padded* batch width, so it can only count rows that reach that width and it under-counts. Every value it reports is a lower bound. That is enough to establish the positive half of (c) — `step0b` and `step0` really did produce non-terminating output — and not enough on its own to establish the negative half. What supports the negative half is Sec. 3.15, which measures the routed arm's stop probability directly, and the routed arm's free greedy output, which grows to 455 tokens against a 1024 training cap rather than running away from it.

Table 13: Integrated EOS hazard over 64 frozen greedy responses, teacher-forced through the routed ladder (`ctx`) and the group-routing-off control (`step01`). Four of the 27 probed checkpoints, at steps both ladders share.

	`ctx` (routed)			`step01` (control)	
step	H	p_{end}	H_{body}	H	H_{body}
24	0.99754	0.99754	$< 5 \times 10^{-7}$	0.99924	$< 5 \times 10^{-7}$
74	0.99958	0.99958	$< 5 \times 10^{-7}$	1.00204	0.00208
99	0.99975	0.99976	$< 5 \times 10^{-7}$	**1.20534**	**0.20538**
149	0.99958	0.99958	$< 5 \times 10^{-7}$	1.09105	0.09110

3.15 Teacher-forced EOS hazard: the routed arm's termination signal does not erode

Sec. 3.13 leaves two candidate mechanisms that the training logs cannot separate, and both are statements about the probability of stopping. Under *smooth erosion*, an objective that raises the log-probability of tokens that were produced gradually lowers the probability the model assigns to the stop token. Under a *dynamical threshold*, that probability holds and then collapses. We measured the quantity rather than argued about it.

Protocol. One set of 64 greedy responses is generated once and frozen. Each of 27 checkpoints — the routed ladder (`ctx`), the matched control with group routing off (`step01`), and the base model as step 0 — is then teacher-forced over those same token sequences, and we read the model's probability of the stop token at every position. Writing T for the position of the true final token,

$$H = \sum_{t \le T} p_{\text{eos}}(t), \qquad p_{\text{end}} = p_{\text{eos}}(T), \qquad H_{\text{body}} = H - p_{\text{end}},$$

so H is the integrated stop hazard over a response, p_{end} is the spike at its semantic end, and H_{body} is stop-mass placed anywhere else.

Two things pinned before the numbers mean anything. The stop token was determined empirically rather than assumed: all 64 frozen responses end on 151645 (`<|im_end|>`) and none on 151643. Alignment between the scored positions and the text was checked by the strongest test available here — the frozen responses are greedy, so a correctly aligned teacher-forced pass must reproduce them by argmax. Agreement at the generating checkpoint is 0.998, with $p_{\text{end}} = 0.99928$; an off-by-one gives approximately zero on both.

The routed arm's hazard does not erode. Across the ladder it rises, from 0.99754 at the earliest probed checkpoint to a maximum of 0.99979, and the minimum over the 64 prompts rises with it, $0.850 \to 0.994$. H_{body} stays below 5×10^{-7} at every probed step, so the hazard is a spike at the response's semantic end and that spike does not move. We quote the range rather than a trend because the four steps in Table 13 are not themselves ordered (0.99975 at step 99 against 0.99958 at 149, a difference of 1.7×10^{-4}); what the ladder supports is that H stays within 2.5×10^{-3} of 1 throughout and at no point declines toward zero.

This refutes both mechanisms we had entertained. Smooth erosion predicts a falling H, and it does not occur. The dynamical-threshold account is refuted at its premise rather than at its prediction: the hazard is not the failing state variable at all, because it never leaves a 2.5×10^{-3}

band while the same arm's free greedy output grows from 327 to 455 tokens. Whatever the length increase is, it is not the model losing its ability to stop.

Not a stale-text artifact. The responses are frozen, so a sufficiently drifted checkpoint could in principle be scored on text it would never itself produce. A self-generated control repeats the measurement with each checkpoint scored on its *own* greedy output ($n = 32$): `ctx`'s mean p_{end} stays in $[0.984, 0.9999]$ while its own greedy length grows $324 \rightarrow 469$.

The control is not flat, and we had claimed it was. `step01`'s H rises to 1.205 at step 99. Its end spike is pinned at 0.99996 throughout, so the extra 0.205 of stop-mass sits *mid-response*, on 16 of the 64 responses, reaching 1.81 on one. The control drifts toward stopping *earlier*, which is the opposite direction from the routed arm's length growth and therefore does not confound the comparison — but the claim that it was flat was wrong, and is corrected here rather than dropped.

What this retires on the fix side. The codebase offers exactly two explicit termination terms, `overlong_reward_penalty` and `mask_no_eos_with_zero`, and both are disabled in every arm reported in this paper. Either one acts on p_{eos}, which is already 0.9996 at the true end and does not move. There is nothing there for it to correct, so if the routed constant needs to be made safe the lever is the constant itself, not a termination penalty added beside it.

Limitations of the termination and length measurements. Four, stated together because each bounds Secs. 3.13–3.15. (i) *The post-knee regime is unmeasured.* `ctx`'s last saved checkpoint is step 149; the run continued to 162 with mean length still climbing (647 and rising) and was then killed abruptly rather than stopped on a criterion, so a hazard collapse living past the last checkpoint is not something these probes could see. (ii) *The probes are greedy and the training was not.* The frozen and self-generated sets are both greedy, while training sampled at temperature 1.0. Greedy length at step 149 (455) tracks the training-time sequence length at step 145 (478), so greedy is a fair proxy for the mean, but the sampled tail is not measured. (iii) $n = 64$ prompts for the frozen probe and 32 for the self-generated control: the hazard differences we are calling null are small, and so is the sample they are null over. (iv) *The control is itself moving*, as the paragraph above records, so Table 13 compares a flat routed arm against a drifting control rather than against a fixed reference.

3.16 Frontier comparison at an honest cap

Sec. 3.12 establishes that a benchmark number without its budget is not a measurement. Table 14 applies that convention to the two frontier candidates for the fixed strong base: `Qwen3.8-27B` and `Frontis-MA1-30B`, both scored on the same box with the same grader at an explicit 32768 cap, with the cap-precedence fix of Sec. 3.12 confirmed live on that box by the line the corrected code prints, `NOTE aime24: using explicit --max-tokens=32768` followed by `(BENCH_OVERRIDES default 8192 not applied)`. Truncation is reported beside every accuracy, because at this cap it is the column that says whether the accuracy beside it is a measurement or a lower bound.

The truncation column carries as much of the comparison as the accuracy column. `Qwen3.8-27B` is 0.8% cap-limited on MATH-500 against `Frontis-MA1-30B`'s 11.4%, so its MATH-500 figure is close to a measurement while every `Frontis-MA1-30B` figure in the table remains a lower bound. The smaller model wins on all five benchmarks and is less cap-limited on all five,

Table 14: Frontier comparison at a matched, explicit 32768 cap, greedy, one run per model. `trunc` is the number of generations that reached the cap and were therefore graded wrong. AMC23 is included for completeness and is read as saturated rather than as a score.

		`Qwen3.8-27B`		`Frontis-MA1-30B`	
benchmark	n	accuracy	trunc	accuracy	trunc
MATH-500	500	**0.9760**	4/500 (0.8%)	0.8980	57/500 (11.4%)
AMC23	40	1.0000	0/40	0.9500	2/40
AIME24	30	**0.9000**	4/30 (13.3%)	0.8000	6/30 (20.0%)
AIME25	30	**0.9000**	3/30 (10.0%)	0.7000	6/30 (20.0%)
OlympiadBench	675	**0.7733**	120/675 (17.8%)	0.7363	135/675 (20.0%)

which is why the paper's fixed strong base is `Qwen3.8-27B`: a delta measured on it is both harder to obtain and harder to dismiss as scale.

AMC23 at 1.0000 **is saturation, not a score.** With $n = 40$ and zero errors the Wilson interval is $[0.912, 1.000]$, so the benchmark cannot separate two models that both sit here and the row resolves nothing. We report it and then decline to use it: AMC23 has no resolving power left at this capability and is excluded from any comparison that has to separate strong models.

A matched cap is not automatically a fair cap. Both models truncate at 32768, so neither number measures capability; both measure *capability under a* 32768 *budget*, which is a legitimate quantity but not the one a bare ranking implies. Where a cap binds on both sides, part of the gap is verbosity rather than skill, and the ranking is trustworthy only if it is stable as the cap grows. On OlympiadBench `Frontis-MA1-30B` is visibly still on the slope — 0.6237 at 16384 (33.3% truncated), 0.7363 at 32768 (20.0%), with 65536 not run — so that row in particular is a comparison of two lower bounds. Sec. 3.17 supplies the stability check for the other four.

Scope. One run per model, greedy, one set of caps. AIME at $n = 30$ carries about 0.1 of run-to-run jitter on this pipeline, so 0.9000 against 0.8000 on AIME24 is about one jitter width and is not significant on its own; the MATH-500 gap, 0.9760 against 0.8980 at $n = 500$, is well outside it and is the row that carries the ranking. Both OlympiadBench figures inherit the grader caveat of Sec. 3.12: 94 of the 675 problems state more than one gold answer in a single answer string and are marked wrong however the model answers.

3.17 The 32768 cap is saturated on three benchmarks of four; OlympiadBench is the exception

The fairness objection to Table 14 is that a cap which binds on both sides partly measures verbosity. The check is to raise it. Table 15 re-scores `Qwen3.8-27B` at 65536: same model, same grader, same box, only the cap changed.

Correction to an earlier version of this subsection. The OlympiadBench row read *not yet measured* while its 65536 run was still going, and the subsection was titled as though saturation held on all four benchmarks. It does not. That row is now measured, it contradicts the old title for that benchmark, and the title and the framing below are the correction. The three saturated rows are unchanged.

Table 15: `Qwen3.8-27B` at 32768 against the same model at 65536, greedy, one run per cell. On MATH-500 and both AIME sets truncation more than halves and the accuracy does not follow, which is saturation. On OlympiadBench the accuracy does follow. Both OlympiadBench cells graded 675/675 with zero request failures.

benchmark	@32768	trunc	@65536	trunc	Δ
MATH-500	0.9760	0.8%	**0.9800**	0.2%	+0.004
AIME24	0.9000	13.3%	**0.9333**	6.7%	+0.033 (1 problem)
AIME25	0.9000	10.0%	**0.9333**	3.3%	+0.033 (1 problem)
OlympiadBench	0.7733	17.8%	**0.8089**	10.7%	+0.036

Three of the four are saturated at 32768**.** MATH-500 moves +0.004 at $n = 500$, where the standard error is about 0.006 — inside noise. Each AIME set moves by exactly one problem out of thirty, well inside the ≈ 0.1 run-to-run jitter recorded at that n. For those three the informative quantity is the pair rather than either number: truncation more than halves while accuracy does not follow, which is the signature of a budget that was already sufficient, and the generations being cut off at 32768 were ones that were going to be graded wrong regardless. Their 32768 figures in Table 14 are therefore measurements rather than lower bounds, and the ranking on those three rows can be stated plainly.

OlympiadBench is not saturated, and it is the largest cap effect of the four. Doubling the budget buys +0.036 there, with truncation falling 17.8% → 10.7%, on 675 problems graded with zero request failures on both rows. For this benchmark 32768 was not the plateau, and the generations it cut off were ones that would have been graded right. We attach no significance statement to +0.036: no repeat run of this benchmark at this cap exists, and the repeat spreads of Sec. 3.18 were measured on other checkpoints and another benchmark. What the row supports is the qualitative claim that here the accuracy tracks the truncation and elsewhere it does not.

The exception falls where a cap argument predicts it. Ordered by truncation at 32768 the four are MATH-500 0.8%, AIME25 10.0%, AIME24 13.3%, OlympiadBench 17.8%, and the only one that gains from a larger budget is the one whose truncation was highest to begin with. That is the saturation claim read in the other direction: a cap can only be costing accuracy where it is binding, so "the budget is sufficient" is a statement about a benchmark and not about a model, and it has to be checked per benchmark. 10.7% is still not zero, so the 65536 OlympiadBench figure is itself not fully saturated and remains a lower bound; Sec. 3.21 carries it into the comparison it was run for.

A caveat that belongs with these numbers. The 65536 MATH-500 and AIME runs used concurrency 40 against the earlier runs' 24, so the batch composition differed between the two columns on those three rows. Decoding is greedy and the requests are independent, so no answer depends on which others ran beside it; but batched matmuls do not associate identically across batch sizes, and a one-problem AIME difference is exactly the resolution at which that could contribute. This does not weaken the conclusion, which rests on the accuracy *not* moving while truncation halved, and not on the sign of +0.033. The OlympiadBench row is the exception in the useful direction: after the client-timeout failure of Sec. 3.20 it was re-run at concurrency 24, the same concurrency as its 32768 row, so that pair is matched on batch composition as well as on cap.

Table 16: An accidental repeat run. Four checkpoints from the `ctx2` (contextual) and `rnd` (random-router control) ladders, each scored twice on MATH-500 at a matched 16384 cap. *spread* is $|A - B|$.

checkpoint	run A	run B	spread	truncation
`rnd`@173	0.4600	0.4580	**0.002**	10%
`ctx2`@173	0.4580	0.4480	**0.010**	11%
`ctx2`@199	0.2660	0.2420	**0.024**	96%
`rnd`@199	0.3480	0.3220	**0.026**	99%

Table 17: MATH-500 truncation at a matched 16384 cap across both ladders. Accuracies are deliberately not tabulated past the transition: by Sec. 3.18 they are not comparable there, and truncation is the trustworthy channel.

step	149	173	174	199	202	231	260	289
`ctx2` (contextual)	6.4%	11%	13.2%	**96%**	99.4%	92.2%	77.8%	96.4%
`rnd` (random)	6.0%	10%	12.4%	**99%**	99.6%	99.6%	97.4%	98.2%

3.18 The noise floor is a function of truncation

`narrow.sh` was launched twice by mistake, so four checkpoints were each scored twice on MATH-500 at a matched 16384 cap. The duplication measures the run-to-run spread of this exact pipeline directly, rather than by assertion, and it does so at two very different truncation rates.

At $\approx 10\%$ truncated the same checkpoint reproduces to 0.002–0.010; at $\approx 97\%$ truncated it moves by 0.024–0.026, an order of magnitude worse. The noise floor is therefore not one number for this suite — it is a function of how much of the batch the cap is cutting off.

The rule this licenses. Above roughly 80% truncation an accuracy is dominated by which few generations happened to finish, and finishing is not random: the survivors are the fastest generations, hence the shortest. **We do not compare accuracies taken above $\approx 80\%$ truncation**, and where two such points differ we read the truncation channel instead. This had until now been argued rather than measured; Table 16 is the measurement.

What it retires. We had recorded that the random-router arm *retains more accuracy* than the contextual one after the transition of Sec. 3.19 — at step 289, 0.3260 against 0.1740. Both points sit at 96–98% truncation, where the repeat spread is 0.025, so what that comparison reports is which few generations finished on each arm rather than a capability difference. The sign survives; the magnitude does not, and neither does any ordering among the post-transition points. We withdraw the reading here rather than quietly not repeating it.

3.19 The collapse transition narrows to steps 174–199, and both arms cross together

Every point in Table 17 is MATH-500 truncation at a matched 16384 cap, on the checkpoint ladders of the contextual arm (`ctx2`) and the random-router control (`rnd`). The cap is 16384 and not 32768 because these are 1.5B checkpoints with `max_position_embeddings: 32768`; asking such a model for 32768 *new* tokens is rejected outright (Sec. 3.20). It is 5.3× the 3072 these arms were originally scored at.

The unmeasured window is twenty-five steps wide, and both arms cross inside it. Truncation goes 13.2 → 96 on the contextual arm and 12.4 → 99 on the random one between step 174 and step 199. The two ladders agree to within a percentage point at every measured step on either side of that window — 12.4 against 13.2 before, 99.6 against 99.4 after.

This is the strongest form of the controller-is-not-the-cause result. The random router has no learned policy, no credit signal and no preference over modes (Sec. 3.9), so the timing of this transition cannot be a property of the controller. It is a property of the routed constant, which both arms share. The earlier evidence for that conclusion was that both arms collapse; this is the stronger statement that they collapse *at the same step*.

It is also not a cap artifact, which was a live possibility. The original step-289 scores were taken at 3072 with ≈ 100% truncation, where an accuracy is a truncation rate wearing accuracy's clothing. Raising the budget 5.3× moves truncation from ≈ 100% only to 96–98%. These policies genuinely emit more than 16384 tokens; the pathology is in the model and not in the budget.

Two wrong shapes from one missing interval, recorded because the error repeats. This table has now corrected the reading of the same transition twice, in opposite directions. From training-time length curves on runs killed at step 162 we first called the change *threshold-like*; from four points at 149/174/260/289 we then corrected that to *gradual, not a cliff*; filling in 202, 231, 173 and 199 shows it is *sharp*. Both wrong shapes came from drawing a line through the same unmeasured gap. The general form is worth more than the particular correction: interpolating a shape across an interval that contains no measurement produces a confident claim whose sign is set by which endpoints happened to exist.

What the table does not support. There is no clean post-collapse plateau. After the transition the contextual arm reads 99.4 → 92.2 → 77.8 → 96.4, non-monotone across four points; its step-249 checkpoint (not shown, outside the matched step set) is the oddest, at 61.4% truncation with the ladder's lowest accuracy, so at that checkpoint the failures are not all length. That is unexplained and we do not model it.

3.20 Three ways a harness reported a number that was not a measurement

Sec. 3.12 reports one defect of this kind. It was not the only one, and the three together have a shape worth stating as a contribution rather than as an apology: in all three the run completed, exited zero, and wrote a plausible number, so nothing downstream of the score could have caught them.

(i) A defaults table silently outranking an explicit cap. `resolve_params` applied a per-benchmark `BENCH_OVERRIDES` table unconditionally, so an explicit `--max-tokens` was parsed, accepted and discarded. AIME24 read 0.2667 at an applied 8192 against 0.8000 at the requested 32768, and AIME25 0.2667 against 0.7000: a factor of three from a flag that appeared on every command line and reached no sampler. Table 11 is that measurement.

(ii) A cap larger than the model's context, which fails every request. A re-score asked 1.5B checkpoints with `max_position_embeddings: 32768` for 32768 *new* tokens. The server rejected every request, and the scorer reported `acc=nan` with `fail=500/500`. A `nan` in an accuracy

Table 18: OlympiadBench, $n = 675$, greedy, one run per cell, `trunc` the fraction of generations that reached the cap. At 65536 the two models truncate at the same rate to the precision reported and every problem is graded on both sides, so the comparison is matched on cap, on truncation and on coverage.

model	@32768	trunc	@65536	trunc	graded @65536
`Qwen3.8-27B`	**0.7733**	17.8%	**0.8089**	10.7%	675/675, fail 0
`Frontis-MA1-30B`	0.7363	20.0%	0.7615	10.7%	675/675, fail 0
gap	+0.037		+0.047		

column reads as a score to any reader who does not read the failure column beside it, which is the entire argument for printing that column.

(iii) A client timeout discarding work the server had already done. Concurrency was raised 16 → 40 on a 65536 OlympiadBench run while the launcher held a fixed `--timeout 600`. The KV pool was never the constraint; more sequences sharing the cards means each takes longer, and per-request latency crossed the fixed deadline. The server's access log records 613 generations returned with `200 OK`; the client kept 13 of 675 and counted 662 failures. The scorer then reported `acc=0.6154, n=13/675, fail=662` and exited zero. Roughly a GPU-day of generation produced thirteen usable answers, and the 0.6154 is void: it is an average over self-selected survivors, which are the fastest generations, hence the shortest, hence not a random sample of the benchmark.

Why a warning is not a guard. The scorer did print that its accuracy was over survivors and biased upward. That is correct and it is not enough, because a warning beside a plausible number is read as a number. The abort-on-failure-rate guard that would have stopped this at 98% failures had been written, tested and committed — to a repository the scoring box cannot reach, so the box was still running a copy transferred by hand hours earlier. A fix that exists only in version control does not protect a machine that cannot reach version control. The progress monitor compounded it: it counted server-side completions, which is all a server log can show, and read 90.8% complete for a run that yielded thirteen answers.

The convention we adopt, stated as a requirement on reporting. Every benchmark number in this paper is reported with three things beside it: **the token budget it was taken at, the fraction of generations that hit that budget, and the number of requests that failed**. Any one of the three defects above is invisible without one of them, and each produced a number in the plausible range. The generalisation is that a harness which cannot fail cannot measure: if a run has no path that aborts, then a successful exit carries no information, and the score it wrote is a property of the harness's defaults rather than of the model.

3.21 The matched frontier comparison at 65536

Sec. 3.16 compares the two frontier candidates at a single cap and records why that is not enough: where a cap binds on both sides, part of the gap is verbosity rather than skill. Sec. 3.17 then shows that OlympiadBench is the one row where the cap was in fact still binding. Table 18 re-runs that row for both models at 65536. It supersedes the statement in Sec. 3.16 that 65536 had not been run for `Frontis-MA1-30B`.

Table 19: OlympiadBench at a 65536 cap, `Qwen3.8-27B`. The first three rows are averages over the responses that survived a client timeout and are *not* benchmark scores; the last is the clean rerun at concurrency 24 with a timeout that scales with the token budget. `trunc` counts *graded* generations that reached the cap, so it is a count over the survivors in the first three rows.

read	accuracy	graded	failed	trunc
survivors, first read	0.6154	13/675	662	—
survivors, further on	0.8717	265/675	410	0
survivors, furthest on	0.8891	613/675	62 (9.2%)	1
clean rerun	**0.8089**	675/675	0	72 (10.7%)

The ranking survives a matched, less-truncated budget. At 32768 the gap is +0.037 with the two models truncating at 17.8% and 20.0%; at 65536 it is +0.047 with both at 10.7%. The gap does not close when the budget doubles — it widens slightly — and it widens at the point where the truncation asymmetry that could have explained it has gone. This is what a single-cap comparison cannot establish. At one cap, "the stronger model is simply the terser one" is an alternative account of the entire gap, and no amount of care about the grader touches it, because the two models are not being asked the same question when one of them is cut off more often. Two caps, with the truncation rates converging to each other and the ordering unchanged, is the observation that rules it out.

The larger model is still on the slope. Across three caps `Frontis-MA1-30B` reads 0.6237 at 16384 (33.3% truncated), 0.7363 at 32768 (20.0%) and 0.7615 at 65536 (10.7%). It is still climbing, and so is `Qwen3.8-27B`. Neither 65536 figure is a measurement of capability; both are capability under a 65536 budget, and both remain lower bounds while 10.7% of generations reach the cap. What is now established is the *ordering*, at a budget where the verbosity confound has been reduced rather than assumed away.

Scope. One run per cell, greedy. Both rows inherit the grader caveat of Sec. 3.12: 94 of the 675 problems state more than one gold answer in a single answer string and are marked wrong however the model answers. That caveat applies equally to both models, so it moves both accuracies and not the gap.

3.22 Survivor bias is not conservative: an average over survivors is biased upward

The OlympiadBench figure at 65536 was read three times before it was measured once, and the three wrong reads are worth more than the incident that produced them, because they quantify a bias that is ordinarily assumed to run the other way. The mechanism is the one Sec. 3.20(iii) reports: concurrency was raised while the client held a fixed timeout, so generations the server had already completed were discarded client-side, and the scorer averaged over whatever survived. Table 19 places those survivor averages beside the clean rerun.

The bias is upward, and it is larger than the effect being measured. The best survivor read, 0.8891 over 613 of 675, overstates the clean 0.8089 by +0.08. The effect that run existed to measure is the cap effect of Sec. 3.17, +0.036. The bias is more than twice it. A survivor average

is therefore not a noisier version of the score with the same centre; it is a different quantity, and quoting it would have more than doubled the headline in the direction that flatters the model.

The direction is predictable, which is what makes this methodology rather than an anecdote. What the client discarded were the requests that took longest. Longest means most tokens, and on a mathematics benchmark the longest responses are the ones on the hardest problems — the same correlation that makes truncation itself a difficulty-biased filter. Removing them removes hard problems from the average, so the average rises. The truncation column shows the same selection from the other side: the 613-survivor read contains exactly *one* truncated generation against 72 in the complete run, because a generation long enough to reach the cap was also slow enough to be discarded. A read that loses the slow responses is structurally unable to see the cap effect it was launched to measure.

Against the intuition that dropping data is safe. The reflex is that a partial result is a conservative result: fewer samples, a wider interval, the same centre. That holds only when the drop mechanism is independent of the outcome, and here it is anti-correlated with the outcome by construction. The failure is also silent. All three survivor rows are plausible OlympiadBench numbers, all three came from processes that exited zero, and the 0.8891 row has a 9.2% failure rate that a reader scanning for problems would not consider disqualifying. We therefore treat the completion count as part of a score's identity rather than as diagnostics beside it, which is the reporting convention of Sec. 3.20, and we make a high failure rate abort the scorer rather than warn: a warning printed next to a plausible number is read as a number.

A correction this forces on Sec. 3.20. That subsection reports the incident as a GPU-day of generation yielding thirteen usable answers. The mechanism is right and the 0.6154 is void either way, but the magnitude is wrong. The output file accumulated several launches under one reused tag; the 13/675 line was the earliest of them and the same file's furthest-completed launch reached 613/675. The loss was overstated by reading one line of a file that other processes were still writing. A reused output tag makes a log a mixture of runs, and a single `grep` of such a file is not a measurement.

3.23 An unrouted 30B LoRA control, and what it is not

What this run is, read from its resolved configuration. `lora30b` trains a LoRA adapter on `Frontis-MA1-30B` with GRPO on eight A100s. Its resolved configuration reads `group_routing: null` and `token_routing: null`. **The run has no routing of any kind. It is the matched control for a routed arm at 30B, not an instance of the method.** We state this before the numbers because the run was described internally as the method for several days on the strength of its name. A name is not a configuration, and the only thing that settles what a run tested is the switch in the config it resolved.

The conclusion is approximately neutral. The peak rung is step 149 at both caps, and the base-to-peak gap is +0.032 at 4096 and +0.018 at 16384. At 16384, with $p \approx 0.88$ at $n = 500$, the standard error of one accuracy is 0.0145 and a difference of two runs carries about 0.021, so +0.018 sits inside one standard error of the difference. At 4096, with $p \approx 0.67$, the same figures are about 0.021 and 0.030, so +0.032 is roughly one standard error there too. Neither cap supports calling the peak an improvement. At 16384 the base is not even the lowest rung: four of the eight adapter

Table 20: `lora30b` adapter checkpoints against the adapter-free base, held-out MATH-500, $n = 500$ per cell, greedy, every cell graded 500/500 with zero request failures. The same nine rungs are scored at two caps. `trunc` is the fraction of generations that reached the cap. This is a control ladder: no rung has routing enabled.

	cap 4096		cap 16384	
step	accuracy	trunc	accuracy	trunc
base, no adapter	0.6520	41.2%	0.8820	13.8%
24	0.6600	41.0%	0.8680	15.0%
49	0.6660	39.2%	0.8720	15.0%
74	0.6720	40.2%	0.8880	12.8%
99	0.6700	39.8%	0.8800	13.2%
115	0.6680	38.6%	0.8740	14.2%
124	0.6720	40.4%	0.8880	12.6%
149	**0.6840**	39.4%	**0.9000**	11.4%
174	0.6760	40.8%	0.8820	13.2%
base to peak	+0.032		+0.018	

checkpoints score below it, the worst by 0.014. **LoRA on this base is approximately neutral on held-out MATH-500**, and that is the claim we make for it.

The gap shrinks as the cap grows, and the truncation column says why it might. +0.032 at 4096 becomes +0.018 at 16384. Truncation falls along the ladder in both columns, and the base is the more truncated end of it in both: 41.2% against the peak's 39.4% at 4096, 13.8% against 11.4% at 16384. Part of the apparent gain is therefore not accuracy but the base being cut off more often than the peak adapter, which is to say the adapter becoming more concise as training proceeds — the opposite of the length growth the routed 1.5B arms produced (Sec. 3.13). A gain that shrinks as the confound shrinks has the shape of an artefact, and extrapolated to zero truncation this one plausibly closes. Three rungs are being scored at 65536 to test that: if the gap returns at +0.018 or larger where truncation is near zero, the truncation account is wrong and the effect is real. That is a prediction and it is not a result.

What survives on the other side, recorded because it is not settled. The *shape* replicates across two budgets differing fourfold with truncation differing threefold: the same peak at step 149, the same downturn at 174, and a near-monotone ordering of the rungs in between. The rise from step 24 to step 149 is present in both columns (0.6600 → 0.6840 at 4096, 0.8680 → 0.9000 at 16384). Noise around a constant does not usually arrive sorted twice. We record that as suggestive and decline to claim it, because the significance calculation above is the binding constraint and it says no.

What the ladder does settle: the feared degradation is absent. `lora30b` inherits `eps_clip: 0.4` and `lr: 1.0e-05` from an upstream demo recipe, and the aggressive recipe of Sec. 3.1 cost `step0d` 0.194 absolute on this same benchmark, monotonically, under full fine-tuning of a 1.5B. Nothing of that shape appears here at either cap. That is why the checkpoints were scored rather than the run restarted, and it is the negative claim the ladder was launched to test. It does not transfer the 1.5B finding either, because two things changed at once — the scale, and full fine-tuning to LoRA — and adapters are conventionally trained one to two orders of magnitude above full

fine-tuning learning rates, which makes this learning rate conservative by LoRA standards rather than aggressive.

3.24 Scope of the routing claims, stated as a limitation

Every routing measurement in this paper is at 1.5B. `ctx`, `ctxpc`, `ctx2`, `rnd` and `step0j` all train Qwen2.5-1.5B-Instruct. The null against the matched control (Sec. 3.9), the credit-signal analysis (Secs. 3.10 and 3.11), the length threshold (Sec. 3.13), the termination probe (Sec. 3.15) and the collapse and its timing (Sec. 3.19) are all claims at that scale. None of them is evidence about routing at any other scale.

No routed arm at 30B has been run. The only 30B training run reported here, `lora30b` of Sec. 3.23, has routing disabled and is the control half of the comparison the paper's claim needs; the treatment half does not exist. Sec. 3.23 therefore bears on plain GRPO with a LoRA adapter at 30B and on nothing else, and no reading of it as a routing result at a fixed strong base is supported. The frontier work of Secs. 3.16, 3.17 and 3.21 selects and characterises that base; it does not measure a delta on it. A delta at a fixed strong base is the paper's stated goal and it is currently half-measured.

The two stabilisers have never been used by an experiment. The zero-mean advantage correction (`zero_mean`), which re-centres the advantage field after all routing writes so the batch mean is restored, and the exclusion of non-terminating rows from the supervised write (`exclude_truncated_from_sft`) are both implemented in the actor, both default to off, and are covered by a mutation suite that kills 39 of 39 mutants. No run reported in this paper enables either. They are code, not results: this paper makes no measurement of their effect, and the mechanism arguments that motivated them stand as arguments.

3.25 The reporting requirement, and a fourth harness failure that has now recurred

Sec. 3.20 records three ways this harness produced a number that was not a measurement, and Sec. 3.18 measures how far the same pipeline moves when it scores the same checkpoint twice. Read together they settle two things every accuracy in this paper depends on: what has to be printed beside a score, and which scores may be compared at all. This subsection states both as rules and adds a fourth failure mode — the only one of the four that has now been observed more than once, and on more than one machine.

Comparability is set by truncation, not by a single noise band. The accidental duplicate run of Sec. 3.18 is the measurement that fixes this: at $\approx 10\%$ truncation the same checkpoint reproduces to 0.002–0.010, and at $\approx 97\%$ truncation it moves by 0.024–0.026. Same checkpoints, same matched 16384 cap, same grader; the only thing that differs is how much of the batch the cap is cutting off, and the reproducibility differs with it by an order of magnitude. There is therefore no single noise floor for this suite to quote, and quoting one would be wrong in both directions at once — too permissive at 10% truncation and far too strict at 97%. The operating rule is the one Sec. 3.18 states and this paper applies throughout: **accuracies taken above $\approx 80\%$ truncation are not comparable to one another**, and where two such points differ we read the truncation channel instead. The post-transition points of Sec. 3.19 are the case that forced the rule, and Sec. 3.23 is the case where it is comfortably satisfied on both ladders.

Table 21: Three occurrences of one failure mode: a generation cap larger than the model's context window. In every row the server rejected every request, the scorer wrote a `nan` accuracy with a full failure count, and the run was recorded as complete. The collaborator's context windows are as recorded in that sweep's models rather than re-verified here.

machine	model	context	new tokens asked	reported
ours	`ctx2/rnd` 1.5B ckpts	32768	32768	`acc=nan`, `fail=500/500`
collaborator, 4×H200	`Qwen2.5-32B-Instruct`	$\leq$ 32768	32768	`acc=nan`, `fail=N/N`, DONE
collaborator, 4×H200	`Qwen2.5-Math-7B-Instruct`	4096	32768	`acc=nan`, `fail=N/N`, DONE

(iv) A cap larger than the model's context, recurring across machines. Sec. 3.20(ii) reports this mode once, on our own re-score of 1.5B checkpoints. It has since occurred twice more, inside a single sweep on an independent four-GPU H200 box belonging to a collaborator, running a revision of the harness that predates the failure-rate abort. That sweep passes a 32768 cap unconditionally; two of the three models it scored have context windows at or below 32768, one of them 4096. The server rejected every request. Both runs finished in minutes (5.5 and 3), both reported `acc=nan` with `fail=N/N` on every benchmark in the suite, and the sweep recorded both as DONE. Table 21 lists the three occurrences.

The one model in that sweep that did produce numbers is excluded rather than reported. Two independent reasons, either of which is sufficient. Its identity is inferred from the ordering and duration of the runs rather than read from a configuration, and this paper has already been wrong about what a run was on exactly that kind of evidence (Sec. 3.23). And its OlympiadBench row discarded 83 of 675 requests client-side, 12.3%, which makes it the survivor average of Sec. 3.22 rather than a score — the same construction that overstated our own 65536 figure by +0.08. We record the sweep as evidence about the failure mode and take no accuracy from it.

The durable fix is a clamp at startup, not a guard at runtime. A failure-rate abort already exists in this codebase and would have stopped all three runs of Table 21 before either wrote a number: each fails every request, which is as far past any threshold as a run can get. It stopped none of them, because in each case the machine was running a revision older than the guard. That is now the second time a guard which was written, mutation-tested, committed and pushed failed to protect a machine, and the two causes have the same shape — once the scoring box had no credentials with which to fetch the repository (Sec. 3.20), once it had simply not pulled. **A runtime guard protects only the revisions that have pulled it.** The quantity this mode turns on, however, is not a runtime observation at all: `max_position_embeddings` sits in the model's own `config.json` and is readable before a single GPU is allocated. A scorer that reads it and clamps the requested cap, or refuses to start, is correct on every revision that runs afterwards rather than on every revision that has caught up. The clamp is now implemented in the scorer and covered by a mutation suite: it reads the context from the model configuration, reserves room for the prompt, and prints what it changed and why, while an unreadable or absent configuration yields *unknown* and never clamps, since not knowing a context is not evidence that it is small. It postdates every run reported in this paper, so no number here was protected by it, and the three rows of Table 21 are what it was written against.

Table 22: The liveness probe's two controls, measured against the training run's own rollout server while it was training. A working difference metric must report the first row at or near zero and must separate the two rows. The original probe does neither. The repaired probe is the same six prompts on the same server minutes later; its signal column is exactly zero on identical weights because the comparison is now between the same tokens in the same contexts.

probe	control	compared quantity	value	verdict
original	base vs. base (identical weights)	$\max \lvert\Delta\ell\rvert$	0.978	**live**
original	adapter vs. base	$\max \lvert\Delta\ell\rvert$	1.007	live
repaired	base vs. base (identical weights)	mean $\lvert\Delta\ell\rvert$	0.00000	not live
repaired	adapter vs. base	mean $\lvert\Delta\ell\rvert$	0.0446–0.0534	live

What a score has to carry, restated with what these subsections add. Sec. 3.20 requires three things printed beside every benchmark number: the token budget, the fraction of generations that reached it, and the number of requests that failed. Two additions follow from the subsections above. First, the *denominator* is printed as graded over total rather than left to be recovered by subtracting the failure count, because Sec. 3.22 shows that an average over the survivors of a partial run is not a noisier estimate of the same quantity but a different and upward-biased one, and a number whose denominator a reader must reconstruct is a number whose denominator a reader will not check. Second, comparability is gated on truncation by the rule above, so a pair of accuracies is a comparison only when both sit below the threshold. Each of the four failure modes recorded in this paper produced a plausible number from a process that exited zero, so none of them is detectable anywhere downstream of the score; these columns are the only place they are visible.

3.26 A fifth failure mode: a guard that could not fail

The four modes above all produce a plausible *number* from a broken process. This one is worse in kind: it is a *check* that was installed to catch a specific failure, that ran on schedule, that reported a healthy verdict every time — and that would have reported the same healthy verdict if the thing it was watching for had been happening continuously. It was found by pointing it at a case whose answer is known in advance, which is the only way this class of defect is ever found.

The check, and why it was there. An arm trained with a LoRA adapter is served by the same endpoint as its base model, addressed by model id. If the id fails to resolve, the server answers HTTP 200 with the *base* model, and the arm's accuracy is then the base model's accuracy wearing the arm's label. The periodic evaluation therefore carries an adapter liveness probe: six fixed prompts, one greedy completion from the adapter id and one from the base id, per-token logprobs from each, and a verdict from $\max_i \lvert \ell_i^{\text{adapter}} - \ell_i^{\text{base}} \rvert$ over the $6 \times 32 = 192$ generated positions, against a threshold of 10^{-4}.

The negative control, which is the whole result. We pointed that probe at the base model with no adapter, so the two things being compared were the same weights and the correct answer is *not live, difference zero.* Table 22 gives what it returned, beside a real adapter measured minutes earlier on the same server. The probe reported the base model as **live**, at a value that does not separate from a genuinely different set of weights. Every liveness verdict this project had recorded was therefore uninformative, and in particular the two that had been read as confirming an arm was not silently scoring its base.

The cause is that the two sides were never compared at the same tokens. The probe generated *independently* from each side and then subtracted the two logprob streams by position. Greedy decoding on a continuously batching server is not reproducible: the same weights produced different text on one of the six prompts, and after the argmax paths diverge, position i holds a different token on each side, so the subtraction compares the logprob of one token under one model with that of another token under the other. Divergence happens only at near-ties, where the losing branch sits near $\log \frac{1}{2} = -0.693$ and the winning branch near 0; the maximum therefore lands near 1 regardless of the adapter. That is a saturation, not a measurement, and it explains the observation that first drew suspicion — two evaluation points fifty steps and fifty adapter versions apart agreeing to five decimal places, 1.04127 and 1.04126.

The run's own record already contained the refutation. At steps 50 and 100 the probe reported a text-divergence fraction of 0.167 — exactly one prompt in six, the same same-weights flip we reproduced — with $\max |\Delta\ell| = 1.041$. At step 150 no prompt diverged, and the reported difference fell to 0.117: a ninefold drop on an adapter that had only trained further. The series was tracking a coin flip in the decoder, not the weights.

A second defect the control exposed, which no threshold could have absorbed. Scoring one fixed 61-token sequence on the base route 24 times returned exactly two distinct logprob vectors, 22 and 2 times, differing by 0.605 nats at a single position. It is not adapter leakage: the deviant vector is equidistant from the adapter and from the modal base. So the same-weights noise on this server is of the same order as the adapter signal, and the shipped threshold of 10^{-4} was never usable no matter how the difference was computed.

What the repair has to do, and how it is held in place. The probe now generates nothing. It *scores* a fixed token sequence under each set of weights, so every compared position is the same token in the same context; it scores each side twice and takes the smallest cross-pair difference as signal and the largest within-side difference as an in-band negative control, so the server's own nondeterminism is measured beside the signal on every call rather than assumed; and it decides on the mean over positions rather than the maximum, which no single near-tie can carry. Pooled over six prompts and 465 positions, base against base is exactly 0 on seven independent sweeps while the adapter differs at 378 of the 465 positions with a pooled mean of 0.0525. The negative control is now a test rather than something someone has to think to run, and eight distinct mutations of the repaired probe — including comparing a distribution with itself, defaulting the verdict when nothing was compared, and reporting the configured adapter name rather than the served one — are each caught by it.

One calibration item remains open, stated as a number. The threshold is still 10^{-4} while the largest same-weights pooled mean we have measured is 0.0095 and the weakest real adapter signal is 0.0446. In 39 negative controls against the intermediate form of the repair, one came back live at 0.0095. The final form suppresses this — 0 of 27 — because the minimum over cross-pairs finds a clean pairing whenever only one scoring lands on the deviant path, but two scorings of the same side landing there together is not excluded, and at that point the threshold is the only thing standing between the base model and a verdict of live. The usable window is bounded below by 0.0095 and above by 0.0446.

min. cluster size	clusters	behavioural	size-matched random	prompt-gradient
2	23	−0.00122	−0.00138	+0.00134
3	18	−0.00000	−0.00097	+0.00357
5	8	+0.00579	−0.00329	+0.01155

Table 23: Mean pairwise cosine between per-cluster gradients at `globalstep199`, 140 informative groups. The hypothesis requires the behavioural column to sit *below* the random column. It sits above it at every setting.

The cost of the check, and what does not explain it. The evaluation's cost is rising: 104.2, 139.5 and 177.4 wall seconds at steps 50, 100 and 150, or 7.8%, 10.3% and 12.9% of training throughput. The obvious explanation, that generations lengthen as training proceeds, is false here: mean generated length is flat at 2289, 2336 and 2287 characters, the finish-reason mix is identical at 61 stopped and 3 truncated, all 64 problems graded with no failures at every point, and the log records no aborted requests. Nor is the run slowing: implied step time moves only from 26.7 to 27.5 seconds. The growth is confined to the benchmark generation phase (88.7, 123.9, 155.4 seconds) and is not the liveness probe, which after the repair is prefill-only and cannot grow with completion length at all. Three points cannot separate a linear trend from a saturating one, and step count and wall-clock are collinear here, so we do not claim a mechanism; we record that nothing measured so far bounds it, and that on the observed slope the cadence reaches the whole of training throughput near step 1900. The remedy that follows from this is a ceiling on projected cost rather than a change to what an evaluation measures: project each point's duration from the previous point and the measured step time, and skip the point with a distinct status code when the projection exceeds a configured fraction of throughput, so that the gap is visible on the curve instead of silent.

3.27 Not yet measured

Stated explicitly so the gap is not mistaken for an omission: whether training on the solved branch helps or costs entropy (Sec. 3.5 locates the mass but does not settle this); any arm in which routed units *gain* a signal rather than only losing one; and the harness-destination arm of (4). Until a target is supplied, every routed arm is a gradient-deletion ablation and is labelled as one.

3.28 The interference probe returns a null, and the pre-registered gate closes

The method's central mechanism is that behavioural subpopulations inside one task want conflicting adapter updates, so that partitioning them and giving each its own adapter removes a conflict that a single shared adapter must average away. That claim is testable before any adapter is trained, and we fixed the decision rule before computing anything: proceed only if the behavioural partition's mean pairwise gradient cosine falls *below* a size-matched random partition's by more than the bootstrap interval; treat parity with the random partition as evidence that the clusters are noise and stop.

We dump exact per-cluster LoRA gradients at `globalstep199` over 140 informative groups, and compare four partitions on the same batch, the same checkpoint and the same gradients. Table 23 reports the result at three settings of the clustering's minimum cluster size.

The sign is opposite to the hypothesis at all three settings, and the magnitudes are within noise of zero throughout. Conflict rates tell the same story more plainly: the fraction of cluster pairs with negative cosine is 0.502, 0.477 and 0.500 for the behavioural partition against 0.534, 0.484 and

0.500 for the random one, and one half is what a coin gives. Prompt-gradient clusters, which require no rollouts at all, are consistently the *least* conflicting of the three, which removes the secondary argument that behavioural features are what make the partition work.

Two readings are available and we take the conservative one. The clustering is not degenerate: it finds between eight and twenty-three clusters depending on the setting, so this is not a failure to separate anything. What it does not do is separate *gradients*. Cancellation sits between 0.24 and 0.41 for both the behavioural and the random partition, so conflict exists in the batch; it is simply isotropic with respect to any labelling we can compute. The honest summary is that there is very little structured gradient there to partition, and the The null is not an artefact of the sketching used to make the comparison affordable. Thirty-two gradients were also stored exactly, and across their 496 pairs the largest disagreement between sketched and exact cosine is 0.0314 against a resolution floor of 0.0331, so the sketch resolves what it is being asked to resolve with margin to spare. The exact mean pairwise cosine is -0.000506, agreeing with the sketched figure. Two further measurements point the same way: the per-cluster gradient norms span only about a factor of two, so no cluster dominates the average, and the prompt-side negative log-likelihood gradient is roughly 65 times larger than any of them, which is where the structure in this batch actually lives.

The honest summary stands: there is very little structured gradient there to partition, and the pre-registered rule therefore closes: we build no per-cluster adapters on this signal.

The weaker use of the same clustering as a *targeting* key requires only that clusters differ in how often the model succeeds, rather than that their gradients separate, and we test it on the same batch. It fails as well. Taking the fraction of variance in per-group solve rate explained by the labelling, the behavioural partition reaches 0.179, 0.160 and 0.071 at the three settings against 0.189, 0.190 and 0.073 for size-matched random partitions drawn over 4000 permutations, with p between 0.45 and 0.67. It explains *less* than chance at every setting.

That result is worth stating in the form that shows why the control is not optional. Cluster mean solve rates range from 0.21 to 0.88, and the labelling accounts for eighteen percent of the variance; reported alone, both numbers read as a finding. They are what feature-blind labels of the same sizes produce, because with twenty-two clusters over a hundred and twelve points small groups inflate the statistic mechanically. The control is what separates a discovery from an artefact of partition size, and here it removes the whole effect.

3.29 Validating the graders against published numbers

No published score exists for our base model on either of our evaluation sets, so neither grader had ever been checked against anyone else's run. We close that with two proxies chosen so that the target is fixed before the measurement.

Our OlympiadBench file is the 675-row open-ended English competition subset, and a released math model's evaluation parquet contains that split at exactly 675 rows, so the problem sets are identical rather than similarly named. Scoring Qwen2.5-7B-Instruct through our own grader gives 0.394 with a Wilson interval of $[0.358, 0.431]$ against a published 39.7: the grader agrees with the literature to a third of a point. On MATH-500 our base model scores 0.824, $[0.788, 0.855]$, against a published band of 84.0 to 84.6; the interval covers the target, and the point estimate sits at its low end, within the roughly 2.3-point spread that separates two published evaluations of the same released checkpoint.

The third check did not succeed, and its failure is more informative than its success would have been. A frontier 27B model reports 90.3 on our code benchmark's name, so we scored it through our grader and obtained 0.251 to 0.269 across seeds. That gap does not measure our grader: 131 to 136 of 175 generations, about 76%, exhausted the token budget and were graded as failures, and

benchmark	problems	accuracy	Wilson 95%
OlympiadBench, all	675	0.470	[0.432, 0.507]
search half	338	**0.482**	[0.429, 0.535]
report half	337	0.457	[0.405, 0.510]
LiveCodeBench (v6 slice)	175	0.309	[0.245, 0.381]

Table 24: Baseline arm at `globalstep149`. Every problem graded, none failed. The math figure is a lower bound: 94 of 675 problems carry several gold answers in one string and exact-match scores them wrong regardless of the response.

43% still truncated at a cap raised to 24,576. Raising the cap resolves it, and the resolution is the measurement. At 16,384 tokens the model scores 0.251 with 76% of generations truncated; at 61,440 it scores 0.749, [0.679, 0.807], with truncation down to 23.4%. Nothing about the grader changed between those two runs. The entire difference is the token budget.

Conditioning on completion settles it more sharply than that bound does. At the canonical cap, *all* 131 failures were truncations and the grader rejected nothing that finished. At 61,440, of the 134 generations that terminated normally, 124 passed: 92.5%, against a published 90.3. Conditioning on completion selects the easier problems, so 92.5% is not an unbiased estimate of the model's score and we do not report it as one. It does answer the binary question the check was for. Nothing our grader saw finish was wrongly rejected, and what it rejected, it rejected for running out of tokens. We therefore treat the code path as externally corroborated, alongside its internal checks: gold self-verifying on all 175 problems, a known-wrong set failing on all of them, and every mutant killed.

The general point outlives this particular check. A code benchmark score is not interpretable without the generation budget that produced it, and a difference of four times in that budget moved the same model on the same problems through the same grader from 0.251 to 0.749. Almost no published card reports its cap. Our own baseline truncated on no problems at all, so its score is a property of the policy, but that is a fact we had to measure rather than assume.

3.30 Baseline scores

Table 24 gives the vanilla arm at `globalstep149`, which is the reference every later comparison is relative to. The split is committed and content-pinned, and *every arm decision reads the search half*, so 0.482 rather than 0.470 is the operative figure; the full set and the reserved half are shown once here and the reserved half is not consulted again until the final write-up. Both sit in wide separable bands: the math score against roughly 0.83 for a frontier 27B, and the code score against the 90.3 discussed above. Neither is floored and neither is saturated, which is the property that makes an arm difference detectable at all.

The code run truncated on no problems at all and was not budget-bound, so its 0.309 is a property of the policy rather than of the cap — the contrast with the frontier model above is the clearest illustration in this paper of why a generation budget must be reported alongside any code score.

3.31 Correction to Sec. 3.29: the canonical code check ran at 2000 tokens, not 16384

Sec. 3.29 reports the third grader check — a frontier 27B model scored through our own code grader — and states that the low score was taken "at 16,384 tokens", and that "a difference of four times

Table 25: The third grader check, re-derived from the results JSON of each run. The canonical row is ten seeds at the benchmark's own default cap; the two raised caps are one seed each. `trunc` is the fraction of the 175 generations that hit the cap, read from `truncation_rate` in the same file. Runs `canonical`, `noncap` and `noncap2` under `runs/validate_v3/`. The canonical row's accuracy and interval columns are ranges across its ten seeds — the lowest and highest point estimate, and the lowest lower bound and highest upper bound — not one interval.

cap	seeds	accuracy	Wilson 95%	trunc
2000	10	0.2514–0.2800	[0.193, 0.351]	74.9–77.7%
24576	1	0.6457	[0.572, 0.713]	42.9%
61440	1	0.7486	[0.679, 0.807]	23.4%

in that budget" carried the same model from 0.251 to 0.749. Both figures are wrong. Re-derived from the run's own artifacts, the canonical cap was 2000 and the budget ratio is about 31×. The correction makes the subsection's argument stronger, not weaker, which is the reason to state it precisely.

Every one of the ten canonical seeds (`canonical_livecodebench_v6_seed0..9.json` under `runs/validate_v3/`) records `params.max_tokens: 2000`, and the launcher that produced them (`harness4/run_v3.sh`) carries `MAXTOK` defaulted to 2000, which is the benchmark's own upstream default rather than a value this project chose. No run of that model at 16,384 exists. Table 25 restates the ladder from the JSON.

Three numbers move and one does not. The across-seed range is 0.2514 to 0.2800, not the 0.251 to 0.269 recorded: the top of the range is seed 8 at 0.2800, and quoting the range short of its maximum understates the seed spread by a fifth. The budget ratio between the two ends of the ladder is 61440/2000 = 30.7×, not four. The truncation figure at the canonical cap is 131 to 136 of 175, which is what Sec. 3.29 already says and which is unaffected. So is the whole conditional-on-completion argument, which we re-derived per generation from `finish_reason` in the records files rather than from the summary counters: at the canonical cap 39 generations terminated normally and *all* 39 passed, so the grader rejected nothing that finished; at 61440, 134 terminated normally and 124 passed, 0.9254, which is the 92.5% already reported.

Why the corrected version is the stronger one. At 16,384 the reading would have been that a frontier model needs four times a generous budget to be scored fairly. At 2000 it is that the benchmark's *own published default cap* truncates three quarters of a frontier model's generations and costs it half a point of accuracy, and that a card reporting a score under that default is reporting a property of the default. That is the general claim Sec. 3.29 makes, and the corrected cap is the evidence for it. The lesson for our own reporting is the one this paper keeps relearning in a new place: the cap was read out of a summary rather than out of `params.max_tokens` in the row, and the row had carried the right number the whole time.

3.32 Corpus saturation: the baseline's first corpus left 72% of its groups silent

The A0 baseline arm (Qwen2.5-32B-Instruct, LoRA rank 32, GRPO, 8 prompts × 8 samples per step, `group_routing: null`) was first launched on a MATH corpus and then relaunched on decontaminated DeepMath with every other setting held fixed. That accident is a controlled measurement of how much of the silent channel is a property of the *corpus* rather than of the

Table 26: The silent channel under a corpus change at fixed model and configuration. Each row is one launch; `n` is its logged steps. The MATH row is `runs/a0_math_MATHCORPUS_superseded/train.log.1788330321` and `.1788333052`; the DeepMath rows are `runs/a0_math/train.log.1788336152` + `.1788344500`, `.1788371484` and `.1788377789`. The parser matches each key with the cell delimiter in front of it, because `solved_group_fraction` is a substring of `unsolved_group_fraction` and a bare match for the first also matches every row of the second — the aliasing that forced the retraction recorded on 2026-08-31.

corpus	launch	n	silent	solved	unsolved	informative
MATH	`.1788330321+.1788333052`	88	0.7188	0.6321	0.0866	28.1%
DeepMath	`.1788336152+.1788344500`	234	0.5043	0.3365	0.1677	49.6%
DeepMath	`.1788371484`	509	0.5206	0.3480	0.1726	47.9%
DeepMath	`.1788377789`	212	0.5047	0.3314	0.1733	49.5%

method, and it is the reason the arm was moved. Table 26 reads the four `*_group_fraction` series the actor logs unconditionally on every step, out of the training logs themselves.

The rollout dumps say the same thing without going through a logged metric. Counting a group as unanimous when every reward in it is equal, directly over the exported rollouts: on the MATH corpus 193 of 261 groups across two probes are unanimous, 0.7395, with the step-24 probe alone at 102 of 133, 0.767; on DeepMath 533 of 1025 groups across eight probes are unanimous, 0.5200, with no single probe exceeding 0.570. Two independent instruments — a per-step metric the trainer emits and a per-group recount of the dumped rollouts — agree to within the sampling difference between them.

What saturation does to the update, stated in the quantity that matters. On MATH the median gradient norm over the 88 steps is 1.82×10^{-4} and 11 **of the** 88 **steps have a gradient norm of exactly zero**: on one step in eight the entire batch was unanimous and the update was a no-op. On DeepMath the median gradient norm is 5.18×10^{-4}, 2.8× larger, and *no* step in 234 has a zero gradient. Mean train reward moves the opposite way, 0.7923 on MATH against 0.6034 on DeepMath, which is the point: the corpus that produced the higher training reward is the one that was teaching the model almost nothing.

Why this belongs in a results section and not in an operations note. A run on a saturated corpus does not fail. It exits zero, its loss is finite, its reward is high and rising, and its logs contain no error — and three quarters of every batch contributes exactly nothing to the update. This is the same failure shape as Sec. 3.1, one level further back: there, train reward mis-ordered checkpoints; here, train reward is high *because* the corpus is already solved, and the higher number is the worse arm. The practical rule we adopt from it is that a corpus is qualified for an arm by its unanimity rate measured on that arm's own model, before the arm is launched, and not by its reputation for difficulty.

Scope. One model, one pair of corpora, one configuration; the MATH row is 88 steps against 234–509 for DeepMath, and the two corpora were run at different points of the same day on the same box. The claim is a corpus difference at this model scale, not a property of either dataset in general.

Table 27: The unsolved share of the silent channel, at the operating point the earlier table measured and at the one now being trained. The first two rows are Table 6 restated; the last three are the three A0 launches of Table 26, whose `unclassified` series is exactly 0.0000 on every one of their steps, so unsolved/silent and unsolved/(solved + unsolved) are the same number here and the share is well posed.

task	model	n	unsolved, all groups	unsolved share of channel
GSM8K	1.5B	98	0.045	12.5%
MATH	1.5B	11	0.255	**60.9**%
DeepMath	32B	234	0.1677	**33.3**%
DeepMath	32B	509	0.1726	33.2%
DeepMath	32B	212	0.1733	34.4%

3.33 The unsolved branch is a third of the silent channel on the arm actually trained

Table 6 reports that the composition of the silent channel inverts with task difficulty, reaching 60.9% unsolved on MATH, and the plan for a data supplier is sized on that figure. Both columns of that table are Qwen2.5-1.5B-Instruct, and the MATH column is $n = 11$ batches. Re-measured on the arm this paper actually trains — Qwen2.5-32B-Instruct on decontaminated DeepMath — the unsolved share is about half of it.

Three independent launches agree, so this is not a window effect. The three DeepMath rows are three separate launches of the same configuration on three different logs, totalling 955 logged steps, and their unsolved shares are 33.3, 33.2 and 34.4 percent. Within the longest of them the share is stable rather than drifting: over the second half of the 234-step series it is 34.0% against 33.3% over the whole.

The confound is stated rather than buried, because it is not separable here. Table 6 holds the model fixed and varies the task. These rows vary both, 1.5B to 32B and MATH to decontaminated DeepMath. A larger model solves more, so a smaller share of its silent channel is unsolved by construction, and nothing in this measurement separates "DeepMath is easier for this model" from "scale shrinks the unsolved branch". It is reported as one number and not as an attribution.

What it changes for the supplier axis, which is the reason to report it. The supplier is the only consumer that can act on the unsolved branch, so the size of that branch is the size of the lever. At 64 groups per batch, 0.167 is about 10.7 groups per batch with no target of any kind, against the 16 that a 60.9% reading implies. The lever survives — a sixth of every batch still reaches the update with nothing to learn from — and it is worth about half what the quoted figure claims on the arm being trained. No statement sized on 60.9% may be carried onto a DeepMath result.

3.34 The first periodic evaluation points of the A0 baseline

The baseline arm now scores a held-out slice during training. Table 28 gives the points it has produced, read from `runs/a0_periodic/step{50,100,150}/results.json` as of 21:00 UTC on 2026-09-02, while the run continues.

Table 28: A0 periodic evaluation, OlympiadBench search half, 64 problems, greedy, one sample. `adapter` is the LoRA version the rollout server actually resolved and served for that point, recorded in the row rather than configured. Every point graded 64 of 64 with no request failures and 3 truncations. **These are not accuracies** and must not be placed beside any other number in this paper: see the budget columns.

step	adapter	trend score	Wilson 95%	budget	headline budget
50	`a0_math-v51`	0.1094	[0.054, 0.209]	2048	16384
100	`a0_math-v101`	0.1250	[0.065, 0.228]	2048	16384
150	`a0_math-v151`	0.1406	[0.076, 0.246]	2048	16384

The budget is not the headline budget, and the series is named so that it cannot be mistaken for one. These points are generated by the *training* rollout server, which publishes `context_length: 4096` in its own server info, so the evaluation runs at 2048 new tokens against the 16,384 every headline number in this paper was taken at. Each row therefore carries `budget_matches_headline: 0` and `not_comparable_with_headline: true`, and the quantity is published as `trend_score` rather than `accuracy` — there is no accuracy series in that namespace at all, so there is nothing a reader can line up against a headline accuracy by name. Sec. 3.31 is what that convention is defending against: on this project's own measurements a budget change alone moved one model on one problem set through one grader from 0.2514 to 0.7486.

What the three points do and do not support. They rise monotonically, 0.1094 to 0.1406. At $n = 64$ the three Wilson intervals overlap almost entirely, so **no trend is claimed**: the interval at step 150 contains the point estimate at step 50. What the series does establish is the thing it was built for. The adapter version advances exactly one per training step (51 at step 50, 101 at 100, 151 at 150), and it is read from the server's response rather than from the configuration, so the weights being scored are provably the weights being trained — the failure this project measured at `globalstep149`, where an evaluation could not distinguish an arm's adapter from its base model, cannot silently recur here. Truncation is constant at 3 of 64 across all three points, so the scores are not budget-bound relative to each other even though they are not comparable with the headline.

3.35 Four further ways the evaluation reported a number that was not a measurement

Secs. 3.20 and 3.25 record four ways this project's benchmark harness produced a number that was not a measurement, and Sec. 3.26 a fifth in which a check could not fail. Standing the periodic evaluation up on a live training run produced four more within one day, on a different part of the pipeline. They are recorded together because the distribution across them is the finding: three of the four *refused*, and only the one whose failure was inside the scoring path produced a number.

(vi) An aborted generation graded as a wrong answer. An eight-problem run returned a plausible 0.125. Opening the generations shows why: of the eight, **six carry** `finish_reason: "abort"`, one was cut off after 75 characters, and every one of the six was graded `correct: false` (`work_pe/dryrun_out/step50/olympiadbench_generations.jsonl`, eight rows, six `abort` and two `stop`, one correct). The cause is that the trainer sends `pause_generation` around every weight update, which drops the requests in flight; a curve built from that reads "the model gets everything wrong" when what happened is that the run interrupted its own evaluation. An abort is now a

failed request, excluded from the denominator and counted, and the same eight problems then return an empty-evaluation status and NaN instead of 0.125. This is the only one of the four that produced a number, and the number was in the plausible range for a genuinely weak arm.

(vii) A cap above the *server's* context, which the model's own config cannot reveal. Modes (ii) and (iv) are a cap larger than the model's context, which a guard reading the model's `config.json` can catch. This one is not that. A0's Qwen2.5-32B declares `max_position_embeddings: 32768`, so the guard passes, while its rollout server was launched with `--context-length 4096` and rejected every request. Ten consecutive periodic-eval points, steps 50 through 500, refused: every one is `status_code 3` (empty evaluation) with `resolved_adapter: null`, no problems graded and a cost of 0.0088–0.0094 of training throughput (ten `step*/results.json` under `runs/a0_periodic.pinnedurl_run.20260902T175124Z/`). A server flag is not visible in a model's config, so the clamp has to be against the limit the server publishes about itself.

(viii) An endpoint that could not be discovered at all. A later point refused with `status_code 6`, `resolved_adapter: null`, empty `token_budget` and `rows`, at a cost of 6.2×10^{-7} of throughput (`runs/a0_periodic/step200/results.json`, written before the launch that produced Table 28). The discovery path read an attribute that exists on the engine class the deployment does not use, so it had never worked; what had been keeping the evaluation alive was a hardcoded address, and removing the hardcoded address is what exposed it. Recorded because the diagnosis that preceded it was wrong in an instructive way: the pin was blamed, and the pin was correct.

(ix) A reused tracker run id discarding an entire run's metrics. A relaunch reused the experiment's tracker run id, so the tracker resumed the *previous* run at its last step and answered every commit from the new one with `Tried to log to step` N `that is less than the current step 509 ...this data will be ignored`. That line appears 212 times in the relaunch's own tracker output (`.../a0_math/t1/wandb/run-20260902_175715-a0_math_t1_train`), one per logged step: **every metric of that run, training statistics and evaluation points alike, was discarded at the tracker while the run itself was healthy**. The on-disk results were unaffected, which is the only reason the loss is recoverable. Run ids are now minted fresh per launch.

What these four add to the rule of Sec. 3.20. That subsection requires a token budget, a truncation fraction and a failure count beside every score. These four say the same requirement applies one level up, to the *evaluation's own* status: a point that did not run and a point that scored zero must not be the same value on a curve. Three of these four failures were survivable precisely because the module emits a status code and a NaN rather than a zero, so a broken evaluation and a genuinely inert adapter stayed distinguishable; mode (vi) was not survivable that way, because it scored. The convention we adopt is therefore that every evaluation point carries a status alongside its score, and that a plotted curve shows refusals as gaps rather than omitting them.

3.36 Correction, same day, to Secs. 3.34 and 3.35

Both corrections below were forced by the same fact, which is worth stating before them: the periodic evaluation writes each point to a directory keyed *only* by the training step, so a relaunch overwrites the previous launch's point at the same step in place. Within half an hour of the two subsections above being written, the running arm reached step 200 and overwrote a file one of them

cited. An artifact that a live run can overwrite is not a citable artifact, and the durable record is the training log, which is rotated rather than replaced.

A fourth point, and the retraction of the trend reading. At step 200 the arm scored 0.1094 on the same 64 problems at the same 2048-token budget, served adapter `a0_math-v201`, 64 of 64 graded with no failures. That is exactly the step-50 value, so the series over four points is 0.1094, 0.1250, 0.1406, 0.1094: it rises for three points and returns to where it started. Sec. 3.34 describes the first three as rising monotonically, and while that sentence is true of those three it invites a reading the fourth point removes, so we retract the framing and keep only the disclaimer that went with it — at $n = 64$ these intervals overlap almost entirely and no trend was claimed. The truncation count is also not constant as stated there: it is 3, 3, 3, 2 of 64. The adapter-version check continues to hold exactly, 51, 101, 151, 201.

The citation in mode (viii), redirected and strengthened. That paragraph cites `runs/a0_periodic/step20` for a `status_code 6` refusal. That path now holds the step-200 point described above, because the running arm overwrote it. The durable artifact for the same failure is the previous launch's rotated training log, `runs/a0_math/train.log.1788377789`, and it records the failure four times rather than once — at steps 50, 100, 150 and 200, each as a matched pair of lines:

```
periodic eval cannot read the rollout endpoint at step N: RuntimeError: RolloutController
exposes no inference server address, so there is no endpoint to evaluate against.
periodic eval step=N status=6 diagnosis=-1 model=UNRESOLVED trend=nan@max_tokens=UNKNOWN
live=na best=nan@-1 0.0s (0.0% of throughput)
```

Four consecutive refusals, none of them scoring, at no measurable cost to throughput. The substance of mode (viii) is unchanged and its evidence is four times larger than reported.

3.37 The evaluation's cost growth was a transient, and is capped anyway

The paragraph "The cost of the check, and what does not explain it" in the fifth-failure-mode subsection above reads three cost points as an unbounded rise and projects that the periodic evaluation consumes the whole of training throughput near step 1900. Five further points now exist. **They refute the projection**: the rise was a transient that peaked at step 150, and the series is flat thereafter. The reading is retracted; the remedy recommended there is implemented regardless, as a safety net rather than as a correction to a growth that is happening.

The slope collapses once the transient is out of the window. Fitted to the first three points the benchmark phase rises at +0.667 seconds per training step, which is the slope that produced the step-1900 projection. Fitted to the five points from step 200 onward it is +0.013 seconds per step, and that residual is not resolvable: across its own 200-step window it implies a rise of 2.6 seconds against a point-to-point scatter of 3.1 seconds about the mean of 122.5. In throughput terms those five points are 10.10%, 9.82%, 10.45%, 10.52% and 9.78% — mean 10.13%, standard deviation 0.31 percentage points. The series is flat, and step 150 is the maximum of all eight rather than a point on a trend. The two negative findings the earlier paragraph established hold across all eight points and are strengthened by them: mean generated length stays in $[2287, 2397]$ characters with no trend, truncations stay at 2 to 4 of 64, all 64 problems grade at every point, and the implied training step time moves only between 26.6 and 28.1 seconds. What changed is the conclusion drawn from three points, not the measurements.

Table 29: Cost of one A0 periodic evaluation point, all eight points, read from `runs/a0_periodic/step{50,...,400}/results.json`. *Benchmark* is the generation and grading phase alone; *total* adds the liveness probe and adapter resolution. *Throughput* is the total divided by the 50 training steps it is amortised over. Step time is implied from the two, as total/(50 × throughput).

step	benchmark (s)	total (s)	throughput	mean chars	truncated
50	88.7	104.2	7.85%	2289	3
100	123.9	139.5	10.30%	2336	3
150	155.4	177.4	12.95%	2287	3
200	122.9	138.6	10.10%	2323	2
250	117.4	133.1	9.82%	2397	4
300	124.0	139.8	10.45%	2287	2
350	126.7	142.5	10.52%	2382	4
400	121.5	137.2	9.78%	2312	3

We do not replace one extrapolation with another. Fitted to all eight points the slope is +0.034 seconds per step, and extrapolating *that* would cross the ceiling below near step 2200 — inside this run. The post-transient fit extrapolates to step 5400. Neither number is offered as a prediction: the all-eight fit pools a transient with a plateau and estimates neither, and the post-transient slope is smaller than the scatter it is fitted through, so its crossing point is an artefact of a slope that is not distinguishable from zero. The honest statement is the one the earlier paragraph made and this one keeps: a handful of points cannot separate these shapes, and nothing measured so far bounds the cost. What has changed is that the specific alarming projection is refuted, and that the bound is now enforced rather than predicted.

The cap, and what it is a cap on. Before each point the projected cost is computed as the *previous* point's wall seconds divided by the cadence times the current measured step time; if that fraction exceeds a ceiling the point is skipped, and the skip is emitted and persisted as a point with `status_code 11` (`cost_capped`) and NaN for every measurement. The projection is deliberately the last measured cost rather than a fitted trend: a trend fitted to a handful of noisy points is exactly what produced the step-1900 figure this subsection retracts. The ceiling is 0.15, set by `SELFEVO_PERIODIC_EVAL_MAX_THROUGHPUT_FRAC`, chosen to sit above every point in Table 29 — including the step-150 peak — so that it does not silence a curve that is behaving. Two new series, `periodic_eval/cost/projected_throughput_frac` and `periodic_eval/cost/cap`, are emitted on every point, scored and skipped alike, so a gap on the curve can be read against the number that caused it and the ceiling it was measured against. The cadence (50 steps) and the problem limit (64) are unchanged: they set what the curve measures, and the cap is about what it costs.

Three cases deliberately do not skip, and one property is worth stating. The first evaluation of a process is never capped, because there is no previous point to project from; a step time that is not yet measured yields a NaN projection, and every comparison against NaN is false, so the point runs rather than being blinded for a reason that has nothing to do with cost; and a ceiling of zero or less disables the cap outright. The property: while the cap is firing, the previous cost is not refreshed — a skipped point measures nothing — so once tripped the cap latches until the step time rises enough to bring the same projection back under the ceiling. That is intended, and it is safe to read precisely because a skipped point is a recorded point: a latched cap appears as

a run of `status_code 11` on the curve, not as an absence.

The cap does not protect the run that motivated it. The A0 arm now training imported this module at launch, before the change, so it continues under the uncapped code and the cap takes effect at the next launch. This is stated rather than glossed because the two facts are easy to combine into a false one: the running arm is not protected by the cap, and on the evidence of Table 29 it does not need to be.

The controls the cap is held in place by. Twelve mutations of the cap, applied one at a time to a copy of the tree, MUT_SUMMARY Among them: a projection hard-wired to zero so nothing is ever capped; the ceiling comparison reversed; a skipped point reporting the success status instead of its own; the decision computed and then ignored; the hook never recording what a point cost, so the cap can never fire; an unmeasurable step time skipping rather than running; the ceiling read from the wrong configuration field; the cadence dropped from the projection; the default ceiling silently changed; the projection not recorded on the skipped point; the skipped point never written to the artifact directory; and a skipped point emitting zeroes where it should emit NaN. The last of these is the mutation that matters most for how the curve reads, since a zero `trend_score` is indistinguishable from a model that got every problem wrong — the failure mode Sec. 3.35 is about.

A stale run's checkpoints were separated from the live run's, and the separation raced the saver. The saver directory for the A0 arm mixed two runs. The trial name `a0_math/t1` has been reused across seven launches, so all seven wrote checkpoints into `areal-runs/checkpoints/ubuntu/a0_math/t1/default/` keyed only by step — exactly the overwrite-in-place shape Sec. 3.36 records for the evaluation artifacts, and with the same consequence: any retrospective scoring of "A0's checkpoints by step" would have mixed two runs silently, and the highest-numbered checkpoints would have come from a run that no longer exists. The live run was near step 350; the same directory also held steps 374, 399, 424, 449, 474 and 499, whose directories all carried modification times from 16:39 to 17:38 UTC, inside the window of the launch that was killed at step 509. Those six were moved — renamed within the same filesystem, never deleted — to `areal-runs/stale_checkpoints/a0_math_t1_run_killed_at_step509_20260902T1738Z/`, and every file matched its pre-move size and MD5 at the destination.

One of the six was not stale, and the check that caught it is not the check that verified the move. Between the survey that classified the six at 22:51:20 and the checksum pass some tens of seconds later, the live run reached step 374 and its saver overwrote that path in place at 22:52:00. The move therefore carried the *live* run's fresh step-374 checkpoint out with the five genuinely stale ones. The before/after checksums agree — they were both taken after the overwrite, so they verified the move faithfully and could not possibly have caught the mislabelling. What caught it was the *file* modification time: 22:52:00 for step 374 against 16:51 to 17:38 for the other five. A directory's modification time is not evidence about the files inside it, and a survey taken a minute before a move is not evidence about the state at the moment of the move; against a live writer, provenance has to be re-established at the instant of the operation, not inherited from a listing. Step 374 has been moved out of the stale directory to `areal-runs/misplaced_live_checkpoint_step374/`, which records the account and the single command that returns it to the saver directory; until that is run the live run's saves have a hole at step 374 and nothing else is affected.

What was already lost, and what is not ambiguous. No surviving checkpoint is ambiguous: every one left in the live directory has a file modification time after the live launch began at 19:43. The losses had already happened and are not recoverable. The killed run's checkpoints for steps 24 through 349 were overwritten in place by the two launches that followed it, before any of this; its step 374 was overwritten by the live run at 22:52:00 during the separation. Steps 399, 424, 449, 474 and 499 are the only surviving weights from that run, and the margin was minutes rather than hours: the move completed at 22:52, and the live run wrote its own step 399 into the freed slot at 23:03:35 and its step 424 at 23:17:26. Both of those stale checkpoints would already have been destroyed had they been left in place, and the remaining three were inside the following hour. That is what makes the separation a rescue rather than a tidy-up — and it is also why the race in the preceding paragraph happened at all.

3.38 The A0 baseline is flat at the headline budget too, and the cheap probe was wrong about the level but right about the shape

Sec. 3.34 and Sec. 3.36 leave one question open, and it is the question that decides what to do next: the periodic series is flat around 0.11, but every point in it was taken at 2048 new tokens on 64 problems, so the flatness could be the run failing to learn or it could be an instrument too blunt to see learning. Those two readings imply opposite next moves. This subsection settles it by scoring the saved checkpoints at the *headline* budget on a problem set large enough to resolve a difference worth acting on, with the un-adapted base model included as the control. **The answer is that both halves of the worry are partly right, and they do not cancel: the cheap probe understates the accuracy by about 32 points, and its flatness is nonetheless real.**

What was run. Every arm scores all 338 problems of the committed OlympiadBench `search` half at 16,384 new tokens, greedy, one sample — the headline protocol exactly, with `max_tokens` taken from `BENCH_OVERRIDES` rather than passed on the command line, so it is the same constant the periodic evaluation reads. Thirty-two evaluations were run on eight B200s: six replicates of the base model, the 22 checkpoints from step 24 to step 549, and four repeat measurements described below. All 32 graded 338 of 338 with *zero* request failures, and truncation never exceeded 5 of 338, so no score here is budget-bound — unlike the periodic series, the cap is not what is being measured. The `report` half was deliberately not touched: it is the reserved half that makes the headline claim meaningful, and spending it on a diagnostic sweep would have destroyed the search/report discipline for a question that 338 problems can answer.

Every arm is served by an identical code path, on purpose. Each LoRA was merged into the base weights and served as a standalone model, so the base control and the checkpoints differ only in the weights and not in whether an adapter kernel ran at all. A base scored without `--lora-paths` against checkpoints scored with it would confound the one comparison this subsection exists to make.

The checkpoints are provably from the live run. That directory held a killed run's checkpoints until they were separated by hand, so identity was re-established from the weights rather than from the path. The Frobenius norm of the merged LoRA delta rises strictly monotonically across all 22 checkpoints, 0.0758 at step 24 to 3.4858 at step 549 with no break and exactly 256 merged tensors every time; file modification times are strictly increasing and all fall after the live launch at 19:43; every adapter file is 134,287,464 bytes. A checkpoint from a different run does not lie on another run's trajectory, so the monotone series is the check that the size and the path cannot

provide. The same trajectory answers a second question at no extra cost: the adapter is not inert. It grows by a factor of 46 over the run, reaching a relative perturbation of 2.46×10^{-3}, so whatever is wrong is not that the optimiser failed to move the weights.

The instrument's own noise band, measured rather than assumed. The base model was scored six times from six separate servers. At temperature 0 on fixed weights the only remaining source of variation is batching nondeterminism, so the spread across those six is what this harness returns when *nothing* is different. It is not small: 0.4083, 0.4320, 0.4379, 0.4379, 0.4231, 0.4290 — mean 0.4280, standard deviation 0.0112, range 3.0 points, with 14 to 22 of 338 problems changing answer between two runs of identical weights. **Three of the six same-weights pairs differ significantly by McNemar**: `base_b` − `base_a` = +0.0237 with 95% CI [+0.0020, +0.0454], and both `base_c` and `base_d` against `base_a` give +0.0296 with 95% CI [+0.0024, +0.0568]. This is a sixth entry for the catalogue of Sec. 3.20: McNemar's interval models sampling error only, and greedy decoding on a batching server carries a second source it does not model, so a single paired comparison against a single base measurement will report a significant gain between two copies of the same weights at a rate far above 5%. A base control has to be *replicated*, not merely present.

The curve. Table 30 gives it. The 22 checkpoints span 0.4142 to 0.4615, a range of 4.7 points, against a same-weights band of 3.0 points. Pooled over all 22, the checkpoints average 0.4369 against the base's 0.4280, a difference of +0.89 points with a bootstrap 95% CI of [−0.56, +2.39] — consistent with zero, and tight enough to have resolved a true effect of 2.4 points. Fitting accuracy against step gives a slope of +0.13 points per 100 steps, 95% CI [−0.21, +0.49]; over the 525 steps measured that is +0.68 points with 95% CI [−1.09, +2.56]. Both intervals are computed by resampling *problems*, which is what the arms share, rather than resampling arms.

The three points that looked significant do not replicate. Three checkpoints cleared $|z| > 2$ against the base mean — steps 199, 499 and 549. At 22 comparisons that is close to what $\alpha = 0.05$ produces by chance, and none survives a Bonferroni threshold of $|z| > 2.86$, but the decisive test is to measure them again rather than to argue about multiplicity. Each was re-scored on a fresh server, together with the lowest checkpoint as a control on the direction of the effect. Every one moved back toward the base: step 199 fell from 0.4615 to 0.4320, step 549 from 0.4615 to 0.4438, step 499 from 0.4527 to 0.4467, while step 24 rose from 0.4142 to 0.4231. Averaged over both measurements no checkpoint sits more than 2.5 points above the base mean, which is inside the same-weights band. **The apparent significance was the batching noise of the preceding paragraph, caught by the control it predicts.**

What this supports, stated plainly. At the headline budget, across 525 training steps, the A0 baseline is statistically indistinguishable from the un-adapted base model. The design would have resolved a pooled effect of 2.4 points or a drift of 2.6 points over the run and it found +0.89 and +0.68 respectively, both with intervals containing zero. So the reading is *not* that the cheap probe was too blunt to see a real gain: at seven times the sensitivity and eight times the budget there is still no gain to see. What the cheap probe did get badly wrong is the level — 0.11 against 0.43, a 32-point understatement — which is exactly the distortion Sec. 3.34 built its naming convention to prevent, now measured on this run rather than argued from the general case. Combined with the weight-space trajectory, which shows the adapter growing by 46× throughout, the diagnosis is the "learning but not helping" branch of `periodic_eval.diagnose`, not the "not learning" branch:

Table 30: A0 at the headline budget: OlympiadBench `search` half, all 338 problems, 16,384 new tokens, greedy, one sample. `base` is the mean of six replicates of the un-adapted model, whose own range is 3.0 points. Δ is against that mean and z is Δ over its bootstrap standard error. Every row graded 338 of 338 with no request failures. **Unlike Table 28 these are accuracies at the headline budget and are directly comparable with the rest of this paper.**

step	accuracy	Wilson 95%	Δ vs base	z
base ($n = 6$)	0.4280	—	—	—
24	0.4142	[0.363, 0.467]	−0.0138	−1.03
49	0.4467	[0.395, 0.500]	+0.0187	+1.41
74	0.4231	[0.372, 0.476]	−0.0049	−0.35
99	0.4497	[0.398, 0.503]	+0.0217	+1.75
124	0.4379	[0.386, 0.491]	+0.0099	+0.77
149	0.4349	[0.383, 0.488]	+0.0069	+0.46
174	0.4527	[0.400, 0.506]	+0.0247	+1.74
199	0.4615	[0.409, 0.515]	+0.0335	+2.48
224	0.4290	[0.377, 0.482]	+0.0010	+0.07
249	0.4408	[0.389, 0.494]	+0.0128	+0.91
274	0.4290	[0.377, 0.482]	+0.0010	+0.08
299	0.4260	[0.374, 0.479]	−0.0020	−0.15
324	0.4172	[0.366, 0.470]	−0.0108	−0.85
349	0.4349	[0.383, 0.488]	+0.0069	+0.48
374	0.4379	[0.386, 0.491]	+0.0099	+0.83
399	0.4231	[0.372, 0.476]	−0.0049	−0.41
424	0.4349	[0.383, 0.488]	+0.0069	+0.49
449	0.4290	[0.377, 0.482]	+0.0010	+0.08
474	0.4349	[0.383, 0.488]	+0.0069	+0.51
499	0.4527	[0.400, 0.506]	+0.0247	+2.10
524	0.4408	[0.389, 0.494]	+0.0128	+0.87
549	0.4615	[0.409, 0.515]	+0.0335	+2.46

the optimiser is moving the weights steadily and none of that movement reaches OlympiadBench accuracy.

What this does not support, and what would settle it. This is 549 steps of a run configured for 12,853, so it bounds the gain *so far* and not the gain the run can eventually produce; a null at 4% of a schedule is not a null at the end of one. It is also one benchmark, and a reward that is teaching something OlympiadBench does not test would look exactly like this. Two things would separate those cases. First, the same sweep on the training reward itself: if the reward is rising while accuracy is flat, the gap is the claim and it is measurable with the artifacts already saved. Second, continuing this curve as the run proceeds — the sweep costs about ten GPU-minutes per checkpoint and the harness, merge and analysis are stored beside the scores in `gs://selfevo/evals/a0_headline/`, so extending it is re-running one script rather than rebuilding the measurement.

3.39 The interference null survives a second run, a later checkpoint, and a swept random control

Sec. 3.28 closed the pre-registered gate on one checkpoint. Three objections survive that measurement and each of them, if right, would reopen the method: that the batch was favourable, that the

effect appears only once the policy has specialised, and that a single draw of the size-matched random partition is not a null distribution. This subsection answers all three, and adds a provenance correction to Sec. 3.28 that has to be recorded because the checkpoint it names no longer holds the weights it was measured on.

Provenance correction to Sec. 3.28. The dump behind Table 23 read the adapter at `a0_math/t1/default/epoch0epochstep199globalstep199` at 09:41 on 2026-09-02. The file now at that path was written at 21:22 the same day by the currently live run, which reuses the same experiment and trial names and therefore the same directory. The tensor Table 23 measured is consequently *not* the tensor at that path, and that table cannot be re-derived from the checkpoint directory as it now stands. Nothing in the result changes — the dump, its 496 exact gradient pairs and its metadata are all still on disk — but the identity has to be carried by recorded checksums rather than by a path, and every measurement below records one.

The batch is the most readable one available, not a favourable one. A GRPO group whose rollouts all score alike carries an identically zero advantage and therefore an exactly zero gradient, so a batch of mostly unanimous groups drives every partition's mean cosine toward the same value for a reason that has nothing to do with clustering. That is the false-negative this gate was pre-registered to refuse, and the first dump taken on this project hit it: 90 of 128 groups were unanimous. The batch used here and in Sec. 3.28 (`a0_step199_probe.jsonl`, md5 `078a2ba0541a2a19b9a9c95449062ad6`) carries 140 groups at group size 8 with k-histogram {1:20, 2:19, 3:20, 4:14, 5:15, 6:17, 7:35}: **140 of 140 groups are non-unanimous**, against a pre-registered requirement of 60%. The dumps confirm it downstream — zero zero-valued GRPO sketches in every dump reported here. Whatever this null is, it is not the sparse-advantage artefact.

A second run, never previously analysed. A readable dump also exists at `globalstep49` of the `harnessT_trunc` experiment — a different run, same base (Qwen2.5-32B-Instruct), same adapter geometry (r= 32, $\alpha = 32$, on q/k/v/o) — over 153 groups, again 153/153 non-unanimous, k-histogram {1:28, 2:24, 3:19, 4:11, 5:16, 6:18, 7:37}. It had been dumped but never analysed. Table 31 gives it. The contrast changes sign across the three settings (+0.0029, −0.0004, +0.0263) and no setting resolves: the largest is still below the 0.0331 sketch resolution floor. The `mcs` = 5 row is reported for completeness but does not meet the pre-registered $K \geq 4$ requirement — it finds three clusters, which is one pair per partition.

What was run for the trajectory. Three adapters from the *live* A0 run were re-dumped on eight H200s, on the identical batch, with the identical sketch (`dim` 8192, `seed` 0) and the identical library versions as the H100 dump (torch 2.9.1+cu128, transformers 5.3.0, peft 0.18.1); the repository's own 175 cluster-LoRA tests pass on that box, including the test that pins the `hidden_states` layout the MEDS feature depends on. Identity is established the way Sec. 3.38 established it, because the path cannot carry it: md5 `b8319909729618a04f3f5a90faf9468b` at step 199, `a876c1c068274251a4001f35e8c261c9` at step 449 and `65aa994558e845bea9b3bf63b720ac70` at step 749, each 134,287,464 bytes with exactly 256 merged tensors, and Frobenius norms of the merged LoRA delta of 1.2309, 3.0912 and 4.7495 — strictly increasing, so the three lie on one run's trajectory and the adapter is demonstrably not inert across the span being tested.

One caveat, stated rather than buried. The rollouts are held fixed across the three checkpoints, so at steps 449 and 749 they are off-policy and the stored rewards, and hence the advantages, are

Table 31: A second run, never previously analysed: mean pairwise cosine between per-cluster gradients at `globalstep49` of `harnessT_trunc`, 153 groups, 153/153 non-unanimous. The hypothesis requires *behavioural* to sit below *random*. The contrast changes sign across settings and no row resolves against the 0.0331 floor. The `mcs` = 5 row finds three clusters and so does not meet the pre-registered $K \geq 4$.

min. cluster size	clusters	behavioural	random	prompt-grad.	behav. − random
2	30	+0.00282	−0.00007	+0.00069	+0.00289
3	15	+0.00719	+0.00759	−0.00124	−0.00040
5	3	+0.01375	−0.01253	−0.02562	+0.02628

Table 32: The interference null along one run's trajectory. Three checkpoints of the live A0 run (identity established by the strictly increasing merged-LoRA norms 1.2309, 3.0912, 4.7495), the same 140-group batch, the same sketch. *behav.* and *random* are mean pairwise cosines between per-cluster gradients; the random control is redrawn at five seeds and z is the behavioural-minus-random contrast in units of that control's own spread. The hypothesis requires the contrast to be *negative* and large. Nothing resolves against the 0.0331 sketch floor at any checkpoint, the sign depends on the clustering setting rather than on the checkpoint, and the contrast does not grow as the policy specialises.

step	min. clust.	K	behav.	random	behav. − random	z
199	2	25	−0.00151	+0.00019	−0.00170	1.19
199	3	17	−0.00110	−0.00071	−0.00039	0.22
199	5	8	+0.00423	−0.00102	+0.00526	1.34
449	2	26	−0.00090	+0.00049	−0.00139	0.89
449	3	18	−0.00079	+0.00002	−0.00081	0.85
749	2	24	−0.00169	+0.00059	−0.00228	1.17
749	3	18	−0.00054	−0.00001	−0.00053	0.49
749	5	7	+0.00742	+0.00119	+0.00624	1.48

those of the earlier policy. That is deliberate: holding the batch fixed is what makes the comparison across checkpoints a statement about the *parameters* rather than about two things moving at once. What it cannot rule out is an interference structure that only appears in a batch drawn from the later policy itself.

Why the random control is swept. Sec. 3.28 compares the behavioural partition against *one* draw of a size-matched random partition. A single draw is a point, not a null distribution, and the quantity that decides the gate is whether the behavioural partition sits outside the range that feature-blind labels of the same sizes produce. We therefore redraw the control at five seeds per setting and read the contrast against its own spread, which is the comparison the pre-registered rule was reaching for.

The verdict, and what it costs. Across four checkpoints from two runs, three clustering settings and five control seeds, the behavioural partition never separates gradients from a size-matched random one by more than about 1.5 standard deviations of the control's own spread, and every contrast sits an order of magnitude inside the sketch's resolution floor. At the settings that meet the pre-registered $K \geq 4$ with the most clusters the sign is nominally favourable and never significant; at the coarsest setting it is unfavourable. **The pre-registered rule closes the gate: the clusters**

are not a gradient-separating structure, and no per-cluster adapters are built on this signal. The one escape the plan had reserved — that interference grows as the policy specialises — is measured here across a 550-step span and does not occur.

3.40 Per-prompt credit has a prerequisite that the corpus, not the controller, decides

Per-prompt credit assignment pairs a routing decision with the *next sighting of the same prompt*, so the interval at which any credit becomes available is a property of the data loader:

$$\text{recurrence period} \;=\; \frac{n_{\text{prompts}}}{\text{batch size}} \text{ steps.} \tag{7}$$

This is arithmetic, and it disqualifies the configuration our own baseline runs in. A0 trains on 102,835 decontaminated DeepMath rows at a batch of 8, so its epoch is 12,853 steps and it is presently at step 852 of it. A learned-router arm launched on that corpus would emit thousands of routing decisions and receive **zero** credits for the entire run, while every logged quantity except `prompt_credit/credited` reported it healthy. That is not a null result, it is a no-op that reads as one, and it is invisible in simulation because a simulated world recurs by construction.

We therefore state it as a precondition rather than a detail: *a per-unit credit rule across time is defined only on a corpus the run revisits*, and the revisit rate, not the estimator, sets how much signal exists. A second-order term follows from the self-baseline rule, under which a prompt's first delta is withheld because a zero baseline would hand the first-credited mode the whole common training trend: the first credit therefore arrives at a prompt's *third* sighting, after two recurrence periods.

Our M8 arms consequently train on a seeded 512-row subset of the same decontaminated corpus, giving a 64-step epoch — confirmed in the run header (`Epoch 1/10 Step 1/64`) and in the ledger, whose `pending` count rises by exactly 8 per step to 512 at step 64 with zero evictions and zero same-batch skips. The subset size is a *consequence of Eq. 7*, not a tuned hyper-parameter, and it is the arms' principal limitation: they say what a learned router does when credit is available, not what it does at A0's corpus size, where by Eq. 7 it is not.

3.41 What carries the routing claim, and the instrument checked against two controls

The mode mix is not evidence. In a controlled world where a right answer exists, a batch-credited router's L_1 distance from uniform *rises* while its targeting stays at chance, because the router chooses its own assignment and the arms drift apart on that feedback alone. Results are therefore carried by **subset contrast** — half the total variation distance between the mode distribution on one subset of groups and on another, which is 0 for a router with a favourite mode however lopsided the mix becomes — with direction checked by a **per-subset targeting fraction**, so an inverted preference cannot pass.

Two subset labellings are reported and they answer different questions. *Silence* is the fixed rule's own partition (solved-silent, unsolved-silent, informative); a router that separates these is doing what the hand-written rule already does, so this is the floor and not the finding, and its targeting fraction is reported as *agreement with the rule*, never as correctness — a live run has no ground truth about which mode helps a prompt. *Covariate* is a median split on one covariate *within* the informative stratum, which holds the pass count approximately fixed, so any contrast there is attributable to something the pass count does not carry. That is the axis our own feature audit records as untested rather than refuted.

Table 33: The measurement instrument against a positive and a null control, 200 steps through the real `_compute_advantages`. Brackets are 95% bootstrap intervals over steps.

router	contrast on `mean_logprob`	on `logprob_dispersion`	silence contrast
reads `mean_logprob` only	0.9882 [0.831, 1.000]	0.2071	0.0098
feature-blind (null)	0.0616 [0.020, 0.228]	0.0440	0.0528
difference	**+0.9266** [+0.670, +0.954]	+0.1632	−0.0430

Table 34: `credit_sim` re-run on the tree that now exposes the rule from configuration. Final-quarter subset contrast, mean over 8 paired seeds, with the error bar on the *difference* and the number of seeds in which it has the reported sign.

arm	final contrast	difference	value
`batch`	0.098	`prompt` − `batch`	$+0.682 \pm 0.021$ (8/8)
`prompt`	0.779	`prompt` − shuffled	$+0.678 \pm 0.018$ (8/8)
`prompt`, credit shuffled	0.102	self-baseline − shuffled	$+0.685 \pm 0.024$ (8/8)
`prompt_self_baseline`	**0.828**	self-baseline − `prompt`	$+0.048 \pm 0.012$ (7/8)
same, credit shuffled	0.143	`prompt_centered` − `prompt`	-0.027 ± 0.010 (1/8)
`prompt_centered`	0.752		

Every number is a difference against a control, bootstrapped over *steps* rather than groups because within one step the decisions share a policy, a batch and a parameter vector. The two arms are resampled independently: rollout generation is not deterministic across inference servers, so two arms with the same seed and the same data order see different completions from step one and are independent draws from one configuration, not a matched pair.

Before any arm was read, the statistic was driven through the real advantage path for 200 steps against a router that reads one covariate and nothing else, and against a feature-blind router drawing modes from fixed proportions (Table 33). It fires at 0.99 on the covariate a router actually used and reports 0.06 for a router that used nothing, so a contrast reported below is not a property of the estimator. A covariate that is constant within a window is reported *undefined*, never 0.

3.42 The credit study reproduces exactly with the config path in place

Making `credit="prompt_self_baseline"` selectable from configuration required editing the two lines in the actor and the config dataclass that the CPU study had deliberately left alone. Table 34 re-runs that study on the edited tree: every figure it is cited by is recovered to three decimals, over the same eight paired seeds.

The last row is a negative result and is reported as one. `prompt_centered` subtracts the batch's mean delta, which is one number shared by every arm — the same class of quantity per-prompt credit exists to escape — and it measures *below* plain per-prompt credit. It remains selectable because it is the rung of the ablation that shows the distinction matters, not because it is a candidate.

3.43 The learned router on a GPU: arms, and the pre-credit window as a null check

Four arms train concurrently on 8×B200, two GPUs each (`fsdp:d1p1t1` actor + `sglang:d1p1t1` rollout; the fp32 32B actor fits one card unsharded at a measured 133.7 GB of 178.35, so there is exactly one router per arm rather than one per data-parallel worker). Everything except the routing

Table 35: Pre-credit window, steps $[0, 20)$ on every arm — 160 routing decisions per arm, 0 credits in either. Windows are matched on *step index*, i.e. policy age, because the arms were launched at different times. Brackets are 95% bootstrap intervals over steps; the two arms are resampled independently, since rollout generation is not deterministic across inference servers and the arms are independent draws from one configuration rather than a matched pair.

subset contrast	treatment	matched random	difference
silence (solved vs unsolved)	0.3966	0.2857	$+0.1108$ $[-0.072, +0.260]$
cov. `mean_response_len`	0.0864	0.0732	$+0.0132$ $[-0.230, +0.229]$
cov. `len_dispersion`	0.0682	0.1463	-0.0782 $[-0.259, +0.163]$
cov. `mean_logprob`	0.1397	0.0976	$+0.0421$ $[-0.191, +0.291]$
cov. `logprob_dispersion`	0.1383	0.0976	$+0.0408$ $[-0.177, +0.243]$
cov. `truncated_fraction`	undefined — constant within the informative stratum		

configuration is A0's: Qwen2.5-32B-Instruct, LoRA r=32 on q/k/v/o, 8 prompts × 8 samples, cap 1024, lr $= 10^{-4}$, `kl_ctl` $= 0$, `adv_norm` $=$ `null`.

arm	router	credit	role
`m8_prompt`	`contextual`	`prompt_self_baseline`	treatment
`m8_control`	`random`	(inert: no `observe`)	**rate-matched control**
`m8_shuffle`	`contextual`	`prompt_self_baseline` + shuffle	correspondence control
`m8_batch`	`contextual`	`batch`	the recorded null, re-run

The control replays the treatment's *measured* proportions, taken from the treatment's own `route/{mode}_groups` over its first 19 steps (`rl` $= 0.3882$, `sft` $= 0.2961$, `skip` $= 0.3158$) and supplied through two independent paths — the environment the router reads and a file the preflight compares it against — so the configuration cannot agree with the expectation by construction. Over the window below the control realised 0.325/0.306/0.369 against the treatment's own 0.400/0.288/0.313, inside three binomial standard errors at this sample size.

A pre-registered null, and it holds. By Eq. 7 no credit exists before a prompt's third sighting, i.e. step ≈ 128 here, and over the window reported in Table 35 the ledger's `credited` count is 0 for every step of every arm while `pending` rises by exactly 8 per step. The learned router has therefore received no feedback and its ridge models sit at initialisation for every mode, so it must *not* separate itself from a router that reads nothing. It does not: every difference against the matched control contains zero. This window is reported because it is the check that says the instrument and the control are not manufacturing a result — it is not the claim, and the window where the claim lives is stated below as still open.

What is still open, stated as such. These arms establish that the learned router is selectable, runs at 32B beside its mandatory control, and does *not* beat that control before it is taught anything. They do not yet say whether it beats it once credit flows: the post-credit window $[128, \cdot)$ opens only after two recurrence periods and is being written continuously to `gs://selfevo/runs/b200/_analysis/`, together with the correspondence control that shuffles credits across the prompts that earned them and the batch-credited arm that the identity predicts cannot learn at all. A difference reported there, and only there, would be the first evidence that a learned router decides better than the fixed rule in training.

Eq. 7 verified live, step for step. The treatment arm's ledger behaves exactly as the arithmetic requires on the running 32B job. `prompt_credit/pending` rises by precisely 8 per step and saturates at 512, the corpus size, with zero evictions and zero same-batch skips. `prompt_credit/cold_baseline_skips` becomes non-zero at *exactly* step 64 — one recurrence period — which is the first delta being withheld because a prompt has no earlier delta to centre against. The *first credit* lands at *exactly* step 128, two recurrence periods, and the credited values are genuinely per-unit rather than a shared scalar: over the first 32 they span $[-0.625, +0.750]$ with standard deviation 0.250 around a mean of -0.020. The interval between a decision and the credit that closes it is not the epoch length but scatters around it (45 steps for the first), because prompts are reshuffled within an epoch. This is the first time a learned router in this project has been credited with anything that varies by unit, and it is the precondition every result in the post-credit window rests on.

3.44 Two external self-improvement systems: one reproducible, one only reimplementable

This subsection records what we were able to recover from two published self-improvement systems, because the paper needs both a baseline we can actually run and an honest statement of what could not be reconstructed. No GPU was used for anything below; every number is CPU-side and was produced on 2026-09-02.

Ornith-1.5 can be reimplemented but not reproduced, and the distinction is not cosmetic. Its method page publishes explicit formulas, so the *method* is implementable: three sequential stages per iteration (task generation, scaffold generation, rollout), each optimised with GRPO, with $R_{\text{task}} = V(q,s)\,D(q,s,\{\tau_i\})\,N(q)$, $D = \exp[-(p-p^*)^2/2\sigma^2]$, $N(q) = 1 - \max_{q_j \in \mathcal{B}} \text{sim}(q, q_j)$, $R_{\text{harness}} = C(q,h)F(h,\{\tau_i\})H(h)$ and $R_{\text{rollout}}(\tau_i) = h(q,\tau_i)$. What is *not* available is everything needed to reproduce a run. The advertised repository holds a README, a licence, a `.gitignore` and four images across its entire history; there is no source file, configuration, log or training loop in any commit, and no task buffer, per-task outcome, harness, rollout trace, reward, advantage, ablation or compute ledger was released. Of the loop's hyperparameters only $p^* = 0.2$ is disclosed: σ, the rollout count k, the similarity function behind N, and the operational definitions of V, C, F and H are all absent. We therefore report a *reimplementation*, and every one of those gaps is an explicitly numbered choice of ours in `reimplementations/ornith_repro/AMBIGUITIES.md` rather than an inherited fact.

There is no head-to-head with Ornith and we do not construct one. Everything Ornith reports is agentic or repository-scale coding — Terminal-Bench 2.1, SWE-Bench Verified/Pro/Multilingual, DeepSWE, NL2Repo — at 128K–400K context with four-hour timeouts, two of the benchmarks judged by a proprietary model and the headline figures self-reported five-run averages with no per-run values or intervals. It reports no competition mathematics, no LiveCodeBench and no SQL, which are exactly what we report. The benchmark intersection is *empty*. Their released weights are 9B, 35B-A3B and 397B, with no 32B model, so scale matching was never possible either, and their bases were Qwen3.5 and Gemma 4 rather than ours. Any table putting our numbers beside theirs would be one we invented, so we do not build one.

What the reimplementation is, and how it is checked. The loop runs end-to-end on CPU against a deterministic stub client: one generated task, one scaffold, one rollout batch, one reward per stage, one update per stage, artifacts written to JSONL and read back with each stage's provenance digest recomputed on read. Every assertion is on observable state — a field in a record

or a value read off disk — never on a function returning true. The suite is 48 tests and runs in under a second, unchanged, on both a Python 3.12 workstation and the H100 box's Python 3.10 with `CUDA_VISIBLE_DEVICES` empty, so none of it depends on any GPU or on which base model turns out to be trainable in our stack. The base model is a constructor parameter, not a constant.

The guards are mutation-tested, and the harness found a real hole. A guard that has never been observed to fail is not evidence, so `reimplementations/mutate.py` applies twelve deliberate defects to fresh copies of the package and requires the suite to go red for each: empty-buffer novelty returning 0, the product VDN replaced by a mean, the published p^* altered, a singleton GRPO group scored as zeros rather than refused, the task inserted into the buffer before it is scored, the control-matching guard short-circuited, the degeneracy formula truncated, provenance made input-independent, and the all-degenerate-batch and context-length guards disabled. On its first run **one mutation survived**: flipping the default abort policy from "exclude" to "failure" — that is, silently grading aborted generations as wrong answers, the exact silent-zero path this project has been bitten by — changed nothing observable, because every test passed `abort_policy` explicitly and so never exercised the value that ships. Two regression tests now assert the defaults directly and all twelve mutations are caught. This is recorded because the hole was in the test suite, not in the code, and it would have been invisible to a green run.

The difficulty gate beats its size-matched control, which is not what this project's other learned components did. On a synthetic pool of 4000 tasks with known true success probability θ and an observable family label independent of θ, we score every task on a discovery block of k rollouts, keep the top 400 by D exactly as R_{task} would rank them at $V = N = 1$, and compare against 400 tasks drawn at the treatment's *own measured* family proportions. A fresh independent rollout block then re-evaluates both arms. Table 36 reports the result. Selection genuinely enriches tasks whose true difficulty is near p^*: the mean $|\theta - p^*|$ is 0.140 for the treatment against 0.351 for the matched control at $k = 8$, a difference of $+0.211 \pm 0.013$ on the difference ($z = 16.2$). Unlike the learned router and the MEDS clustering, both retired for tying with a control of this shape, this component does not tie.

But the score it reports is badly optimistic, and σ is inert for ranking. The same table shows a large winner's curse: at $k = 8$ the selected tasks score $D = 0.946$ on the block that selected them and 0.551 on a fresh one, a shrinkage of 0.395, so the difficulty a run would report for its own curriculum is inflated by roughly 72%. The curse is a pure function of k and closes only slowly, needing $k \gtrsim 149$ to fall under 0.05. Separately, the enrichment column is *identical* across $\sigma \in \{0.05, 0.10, 0.15, 0.30\}$, which is not a numerical accident: with V and N held constant, ranking by D is ranking by $|\hat{p} - p^*|$, and that ordering does not depend on σ at all. The undisclosed σ therefore cannot change which tasks a run selects, only how the reward is scaled — and GRPO's group normalisation absorbs a common positive scale. One of the two most conspicuous missing hyperparameters is thus nearly inert for selection, which materially reduces the reconstruction risk. It regains influence only through the product with a varying V and N.

The published $p^* = 0.2$ maximises the fraction of wasted groups. A binary reward makes a GRPO group constant, and therefore exactly zero-gradient, with probability $\theta^G + (1-\theta)^G$; we verify this closed form against simulation to ± 0.01. At $G = 8$ it is 0.0078 at $\theta = 0.5$ but 0.1678 at $\theta = 0.2$, a factor of 21.5. The tension this creates is visible in Table 36: as k rises the gate selects true difficulty better, and the degenerate-group fraction it produces rises *with* it, from 0.126 at

Table 36: Ornith-1.5's difficulty gate, reimplemented, on a synthetic pool of 4000 tasks with known true θ, keeping the top 400 by D against a size-matched random control at the treatment's own measured family proportions ($\sigma = 0.15$, $p^* = 0.2$, $G = 8$, seed 20260902). D_{disc} and D_{hold} are the mean difficulty reward on the selecting block and on a fresh independent block. The enrichment column is $|\theta - p^*|$ for the control minus the treatment, so positive means the gate selected genuinely-harder-to-target tasks; its uncertainty is the standard error on the *difference*. This is a synthetic instantiation of the published reward design, not a measurement of Ornith's models.

k	D_{disc}	D_{hold}	shrinkage	enrichment (ctrl−treat)	degenerate frac.
4	0.9460	0.4078	+0.5381	+0.1605 ($z = 11.0$)	0.1259
8	0.9460	0.5513	+0.3947	+0.2113 ($z = 16.2$)	0.1122
16	0.9762	0.7058	+0.2703	+0.2435 ($z = 19.3$)	0.1548
32	0.9790	0.8151	+0.1640	+0.2871 ($z = 23.5$)	0.1875
64	0.9829	0.8963	+0.0866	+0.2887 ($z = 24.6$)	0.1860
149	0.9830	0.9406	+0.0425	+0.3085 ($z = 25.7$)	0.1746
closed-form degenerate fraction at $G = 8$: 0.1678 at $\theta = 0.2$, 0.0078 at $\theta = 0.5$.					

$k = 4$ to 0.187 at $k = 32$, converging on the closed-form value at p^*. The better Ornith's difficulty gate works, the more of its rollout budget carries no reward-directed gradient. This is a property of the published constant, derived from the published equations and measured here; it is not a claim about their weights.

OpenRSI, by contrast, is genuinely reproducible, and we verified that rather than assuming it. OpenRSI (`FrontisAI/OpenRSI`, commit `1f477c48`, CC BY-NC 4.0 with a `NOTICE`; arXiv:2607.28568) ships the search harness (`OpenMLE-Evo`), the SFT and RL stages (`OpenMLE-ERL`), and a real execution sandbox (`OpenMLE-Gym`, with controller, router, workers and `docker-compose`). It installs and imports cleanly under Python 3.12, and **its own test suites pass** 119/119 — 66 in `OpenMLE-Evo/tests` and 53 in the vendored `aira-evo` tests. Both `Frontis-MA1-35B` and its base `Qwen3.6-35B-A3B` are on the Hub, and the NatureBench task data is public and ungated; a single task package downloads at 2.9 MB and contains the problem data, the evaluator *and* the ground truth, so a one-task end-to-end run needs nothing but a served model. Their declared `requires-python` is `>=3.11,<3.13`, and the suite indeed does not collect under 3.13. We note one near-miss in our own procedure: invoking their tests through a relative interpreter path fails 2 of 66, because `sys.executable` then carries `..` segments that their code correctly resolves and their assertion does not; with an absolute path it is 66/66. Their release is clean and we nearly reported a defect that was ours.

Two corrections to how that target is usually described, and what it costs. First, the headline 39.39% → 60.61% Medal Average is a *model* swap with the harness held fixed — both rows run OpenMLE-Evo — and their own table says so explicitly; it is not a scaffold-versus-no-scaffold delta, and recovering it means serving two models and running the search twice. Second, the NatureBench hidden evaluation data, task packages, evaluation service and container images are *not* redistributed inside OpenRSI; they come from the separate public NatureBench repository and Hub dataset. On cost, their protocol fixes a *wall-clock* budget per task, so a faster GPU cannot buy it down — it only changes how much search fits inside the budget, which would itself deviate from the protocol. MLE-Bench Lite at their stated protocol (22 tasks, three independent runs, 12 h per task) is 792 sandbox-hours per model–harness cell and 1584 for the base-versus-MA1 pair, plus a served

35B-A3B model throughout, plus Docker and Kaggle data. NatureBench Lite (10 tasks, four-hour budget) is about 40 hours per cell, roughly twenty times cheaper for the same claim shape, which is why the ladder we propose starts there and starts with a single-candidate smoke run.

3.45 OpenRSI rung 0: the released quickstart does not complete against its own benchmark's current HEAD

We attempted the smallest end-to-end run OpenRSI documents: its NatureBench local quickstart with `--smoke`, which generates and evaluates a single candidate on a single task. It was run on one idle H200 serving `Frontis-MA1-30B` (their released 30B post-trained checkpoint, `Qwen3MoeForCausalLM`, 48 layers, 128 experts, 8 routed per token) through SGLang at a 131072-token served context, with the candidate runtime built from `conda-forge` to their published environment spec. We used the 30B rather than the 35B because it fits one card comfortably. The GPUs running other work were not touched.

Four of five stages fired; the fifth could not. Task construction, prompt and scaffold construction, generation, and candidate materialisation all succeeded: the model returned a 5338-byte `run.py` for the counterfactual-estimation task, which is a real generation and not a template. The run then failed at the reward stage. The NatureBench evaluation service answered `POST /register` with HTTP 404, `unknown endpoint`, and the search aborted before any candidate was scored.

The cause is a version skew between two first-party repositories. It is not a misconfiguration on our side and not a missing credential. NatureBench commit `d38eb9c` (2026-08-28), "protect control endpoints with service token", put `/register`, `/start_timer`, `/resume_timer` and `/pause_timer` behind a `_has_control_access()` check that returns 404 — deliberately masking the endpoint's existence — unless the caller presents a service control token. The check is absent at `d38eb9c^`. OpenRSI at its own HEAD (`1f477c48`) has no notion of that token: its `base_task._post_json` sends only `Content-Type: application/json`, and the strings `control_token` and `eval_control_token` do not occur anywhere in `OpenMLE-Evo`. So a user following OpenRSI's documented quickstart against NatureBench HEAD, three days newer than the commit that broke it, cannot complete a single task. This does not touch the validity of their published results, which were produced before the skew; it is a statement about what the released instructions do today.

Their failure handling is correct, which we checked rather than assumed. The run's `summary.csv` reports `grade_avg@k` = 1.0 beside `success_rate` = 0.0 and `unknown` = 1. Read carelessly that looks like a harness reporting a perfect score for a run that never evaluated, which is the failure mode Secs. 3.29 and following document repeatedly in our own tooling. It is not. In their `build_submit_grade_and_medal`, grade is a normalised leaderboard rank in which *lower is better* — `submit_grade_best` is a `min` — and 1.0 with medal `N/A` is the explicit fallback returned when `submit_score is None`. The failure is recorded conservatively and flagged in three separate columns. We report this because we were one careless reading away from filing a false defect against them, for the second time in this audit: their test suite also appears to fail 2 of 66 until one notices that the failure is caused by invoking the interpreter through a relative path.

Stopped here by pre-agreement. Rung 0 was defined to require a read-back artifact carrying a per-stage reward, and it did not produce one, so rung 1 was not started. Two ways forward exist and they are not equivalent: pinning NatureBench to `d38eb9c^` reproduces the stack OpenRSI was

written against but re-opens control endpoints to model-generated code, which is the exact exposure their fix closed and is a poor idea on a shared box; alternatively OpenRSI's client can be taught to send the token, which is a one-header change but means we are no longer running their code unmodified. The choice belongs to the PI, not to us.

3.46 Correction and hardening of the p^* claim in Sec. 3.44

Sec. 3.44 stated that Ornith's published $p^* = 0.2$ "maximises the fraction of budget that carries zero gradient" and that "the better the gate works the more it wastes". Both statements are too strong, and this subsection corrects them. The underlying arithmetic is unchanged and exact; the interpretation was not.

What survives exactly. For a binary reward and group size G, a GRPO group is constant with probability $\theta^G + (1-\theta)^G$, verified against simulation to ± 0.01. This is monotone *decreasing* on $[0, \frac{1}{2}]$, so $p^* = 0.2$ is in no sense a maximiser: at $G = 8$ the rate is 0.6634 at $\theta = 0.05$, 0.1678 at 0.20, and 0.0078 at 0.50. The defensible claim is the pairwise one — targeting 0.2 rather than 0.5 costs a factor of 21.5 in constant groups — and, more generally, that any target below one half pays a monotone penalty.

The claim needs R_{rollout} to be exactly binary, and the release never says it is. This is the load-bearing assumption and it is fragile. Modelling the harness as a binary verdict plus a continuous quality term of weight w, the constant-group rate at $\theta = 0.2$ collapses from 0.1620 at $w = 0$ to 0.0000 at $w = 0.01$: a harness that returns any graded component at all essentially never produces an exactly constant group. Ornith defines p through a binary success test s, but $R_{\text{rollout}} = h$ is never stated to be binary, and the release does not resolve whether $s = h$. We therefore downgrade this from a defect of the published method to a defect *conditional on a binary harness*. One caveat cuts the other way and is a derivation rather than a measurement: in an all-failure group with small w, the normalised advantages are determined entirely by the jitter, so the gradient is non-zero but uncorrelated with anything about the task. Exactly-zero gradient and pure-noise gradient are not obviously ranked.

Selection usually reduces degeneracy relative to no selection, which is the opposite of what we implied. Repeating the selection under four priors on θ, the kept set's mean constant-group rate against the unselected pool's is 0.128 vs 0.231 (uniform), 0.134 vs 0.165 (Beta$(2,5)$), 0.032 vs 0.168 (Beta$(5,2)$), and 0.663 vs 0.663 (bimodal at 0.05/0.95). In three of four the gate *improves* matters, because an unselected pool is full of tasks near $\theta = 0$ or 1 that are far more degenerate than anything near 0.2. The rising degeneracy with k in Table 36 is convergence from below onto the closed-form value at p^*, not evidence that the gate destroys signal a no-gate baseline would have kept. The correct comparison is between choices of p^*, never between gate and no gate.

The multiplicative structure does not rescue it; it mildly worsens it. Asked whether $V \cdot D \cdot N$ blunts the effect by admitting tasks away from p^*, we find the kept set's constant-group rate *rises* with factor variation, 0.1307 at $V = N = 1$, 0.1842 under mild variation and 0.1876 under strong variation, while $|\theta - p^*|$ barely moves. The product admits some candidates whose θ is more extreme, which costs rather than saves.

Correlated rollouts make the penalty larger, so the arithmetic is conservative. With a shared per-group shift of half-width 0.2 — the effect of sharing one prompt and one policy — the rate at $\theta = 0.2$ rises from 0.176 to 0.273, and at $\theta = 0.5$ from 0.008 to 0.022. The independence assumption understates the cost.

And there is a real defence of $p^* < \frac{1}{2}$, which we state because it nearly wins. A task at $\theta = 0.5$ yields the most informative groups now but is closer to being solved and leaves the useful band sooner. Scoring the informative fraction per step against a lifetime proportional to $1 - \theta$ gives 0.666 at $\theta = 0.2$, 0.660 at 0.3 and 0.496 at 0.5: on this model 0.2 and 0.3 are effectively tied and both beat 0.5. The model is ours, invented to steelman their choice, and Ornith publishes neither a task lifetime nor the staleness dynamics that would settle it. **Net verdict:** the per-step signal cost of $p^* = 0.2$ is exact, large, and worsened by both correlation and the product form; whether it is the *wrong* target is not established, because it turns on a task-lifetime quantity the release does not disclose. Stated that narrowly, the criticism holds. Stated as "0.2 maximises waste", it does not.

Everything in both subsections remains CPU-side and, for the p^* work, synthetic. No claim here has been reproduced on a real model, and the label stays until one is.

Reporting convention adopted for the gate result. Wherever the difficulty gate is reported from here on, the *fresh-block* figure is the headline and the selecting-block figure is labelled an artefact of selection. At $k = 8$ that means the gate's difficulty is 0.551, not 0.946. The two differ by 0.395, and quoting the selecting block is precisely how a curriculum is made to look like it works. Both numbers stay in Table 36 so the gap is visible rather than quietly resolved.

The σ-inertness result, stated as a finding with its proof. Among the loop's undisclosed hyperparameters, σ cannot affect which tasks a run selects. Proof: with V and N constant across candidates, $R_{\text{task}} \propto D = \exp[-(\hat{p} - p^*)^2/2\sigma^2]$, which is a strictly decreasing function of $(\hat{p} - p^*)^2$ for every $\sigma > 0$. Ranking candidates by D is therefore ranking them by $|\hat{p} - p^*|$, an order independent of σ; ties are resolved identically because $\hat{p}$ takes only $k + 1$ values. Selection is thus invariant to σ, and GRPO's group normalisation additionally absorbs the common positive scale σ induces in the reward. This is confirmed empirically: the enrichment column is identical across $\sigma \in \{0.05, 0.10, 0.15, 0.30\}$. σ regains influence only through the product with a *varying* V and N, where it sets the exchange rate between difficulty and the other two factors. The practical consequence for anyone reimplementing Ornith is that one of the two most conspicuous missing constants carries far less reconstruction risk than it appears to.

3.47 Rung 0, second attempt: the skew is authentication *and* an API change

Following Sec. 3.45 we took the patch route rather than pinning NatureBench to its pre-fix commit, because pinning re-opens the control endpoints to model-generated code and the box in question also runs other people's jobs. The security argument is unchanged and we still endorse it. What changed is the size of the patch.

The modification we made, declared. Against OpenRSI commit `1f477c48` and NatureBench commit `cd9de65`, we add to `examples/nature_bench/base_task.py` a helper `_load_naturebench_control_token(` and make `_post_json` attach the header

```
X-NatureBench-Control-Token:  <token>
```

read from NATUREBENCH_CONTROL_TOKEN or the file named by NATUREBENCH_CONTROL_TOKEN_PATH, which the service auto-creates at NatureBench/eval_logs/eval_control_token. It is 60 diff lines, verified to apply cleanly to a pristine 1f477c48, bracketed in the source by LOCAL MODIFICATION, NOT UPSTREAM, and a no-op when no token is configured. It is kept as a patch file, not a vendored fork, because OpenRSI is CC BY-NC 4.0.

It closes the authentication half and no more. After the patch the service logs POST /register 200 and POST /start_timer 200, where both were 404 before. POST /evaluate then returns 400, missing eval_token: NatureBench's /evaluate now takes an opaque per-task eval_token bound at registration, while OpenRSI still posts the pre-hardening body {task_name, batch_name, output_dir}. A git log -S eval_token places that API change in the same two commits as the gate (d38eb9c, 5304604).

We stopped rather than widen the patch, and the reason is a silent-wrong-number risk. Under the new API the service evaluates <registered out_dir>/workspace/output from state bound at registration, and no longer honours a per-call output_dir. In our run those are different paths: the registered directory was .../workspaces/_eval_service/rung0_smoke_patched/s42256-023-00611-x while the attempt actually wrote to .../workspaces/s42256-023-00611-x_validation_1/output. OpenRSI registers once per task but evaluates once per attempt. Adding an eval_token and dropping output_dir would therefore score a fixed directory the candidate never wrote to, and it would do so *successfully*: a returned number, of the wrong directory. Making it correct needs per-attempt re-registration with a constructed out_dir and a token threaded through the attempt lifecycle, roughly 30–50 lines of semantics we would be inferring rather than reading. That is where a wrong number would enter, so the scope decision goes back to the PI rather than being taken here.

The reproducibility observation, stated as such. At current HEADs of two first-party repositories — OpenRSI 1f477c48 and NatureBench cd9de65, three days after the commit that broke compatibility — OpenRSI's documented NatureBench quickstart cannot evaluate a single task. It fails first at /register on a missing control-token header and then, once that is supplied, at /evaluate on a changed request contract. Ornith's absent code cannot be run at all; this is the milder but more insidious case, where a genuinely complete and well-tested release stops working because a security hardening in the benchmark it depends on has not yet been mirrored in the client. **Their published results predate the skew and are untouched by it**, and their own test suites still pass 119/119. We record this as a reproducibility observation about released systems, not as a criticism of the work: it is precisely the kind of quiet breakage that stops third parties reproducing a paper without anyone noticing.

3.48 The group structure was wrong, and it is the algorithm

Our first reimplementation sampled *flat*: one scaffold per task, one rollout group, and — worse — it passed the rollout advantages to all three stages, so the scaffold and task stages formed no groups of their own. We record this plainly because GRPO's advantage is group-relative, so the group structure is not a detail of the algorithm, it *is* the algorithm, and a flat implementation is a different estimator rather than a simplified one.

The source contradicts itself, which is why this was missable. Ornith's release figure shows each task producing several scaffolds and each scaffold producing several rollouts. The

Table 37: Advantage energy delivered per rollout, on realised distributions rather than at the nominal target. Synthetic, CPU-side, $G = 8$, $k = 8$, $\sigma = 0.15$.

selector	mean θ	dead-group fraction	energy / rollout
gate, realised	0.304	0.1206	**0.8794**
size-matched uniform	0.481	0.2485	0.7515
idealised at $p^* = 0.2$	0.200	0.1678	0.8322
idealised at 0.5	0.500	0.0078	0.9922

prose does not: it says the policy produces "a solution rollout", singular, while the difficulty term references a set $\{\tau_i\}$ for the success rate. So the nesting rests on the figure alone and the text is silent or arguably contradictory. We treat nested as faithful, because a figure showing structure is more specific evidence than prose that omits it, and keep flat as an ablation (`NestedConfig.sampling`). This is ambiguity A16; the two counts are A17 and neither is disclosed.

Nested creates two comparison levels where flat creates one. Rollouts compete within a scaffold on R_{rollout}; scaffolds compete within a task on $R_{\text{harness}} = CFH$, whose F is an aggregate over that scaffold's own rollouts. Arms must therefore be matched on total rollouts, $n_{\text{scaffolds}} \times n_{\text{rollouts}}$, not on steps: nested at 3×8 spends 24 rollouts per task where flat at 1×8 spends 8. The implementation is 16 mutations deep now, four of them new and specific to the nesting — a silently collapsed scaffold level, a $\hat{p}$ taken from one scaffold instead of the task's whole budget, a singleton scaffold group accepted, and a malformed nesting zipped away rather than refused. All 16 are caught and the suite is 55 tests.

The winner's curse recurs one level up, by construction. A scaffold is scored on the rollouts it happened to draw, so a scaffold with lucky rollouts scores well for reasons that have nothing to do with the scaffold. This is the identical hazard we measured for task selection, at the scaffold level, and it takes the identical remedy: score on one rollout block, re-score on an independent one, and report the fresh block. The nested runner takes a holdout block for exactly this, and a test asserts that discovery and holdout scaffold rewards actually differ — if they did not, the scaffold score would not depend on drawn rollouts and there would be no curse to correct.

3.49 Does the gate improve learning? What is settled, and what a simulation cannot settle

The outcome decomposes as improvement per unit budget = (task quality) × (budget efficiency), and the two halves have different epistemic status.

Budget efficiency is settled exactly, with no learning assumptions. Under binary-reward GRPO every non-degenerate group carries advantage energy exactly G and every degenerate group exactly 0, so energy per rollout is precisely the non-degenerate fraction. We verified the identity rather than quoting it: mean energy 7.999962 against $G = 8$, maximum absolute deviation 4.8×10^{-5} (our denominator is $s + \epsilon$, not s). On the gate's *realised* difficulty distribution this gives Table 37.

Two corrections to the waste figure, including to our own earlier statement of it. The realised dead-group fraction is 0.1206, **not** 0.1678: the 17% number is the idealised value *at* $p^* = 0.2$ and the gate does not hit its target, its realised mean θ being 0.304. Quoting 17% overstates the waste by about 40% relative. And against *no selection at all* the gate is more efficient, 0.879 against 0.752,

because an unselected pool is full of near-0 and near-1 tasks that are deader than anything near 0.2. The waste claim is only ever a comparison between targets (0.879 against 0.992), never between gate and no gate.

Task quality cannot be settled here, and we demonstrate that rather than assert it. Running five defensible learning rules against four controls gives two results. First, **the control that was specified is vacuous in a difficulty-only simulation**: against a control resampled to the treatment's own realised θ distribution, the largest absolute difference across all five rules is 0.0115, necessarily so, because θ is the entire task representation in simulation and matching it exhausts the task. In a real run it is not vacuous, because there the gate also selects on validity and novelty and the control does not. Second, **the sign flips with the assumed rule**: the gate beats a uniform control under learnability, frontier, zone-of-proximal-development and lifetime, but *loses* under a rule where gain rises with success rate, by -0.0911 against uniform and -0.1547 against a crude band filter. Which rule holds is exactly what is unknown, so a cheap simulation returns the answer it was built to give. The outcome question must be run on a real model.

One substantive hint the simulation does license. A one-line filter keeping $0.1 < \hat{p} < 0.9$, with no target and no kernel, recovers much of the gate's advantage and beats it outright under two of the five rules. If that survives on a real model, the difficulty kernel and its target are unsupported complexity. That comparison is pre-registered as arm C2 in `PREREG_gate_outcome.md`, alongside the matched-difficulty control C1, with stop rules fixed in advance: if the gate does not separate from C1 beyond the noise band it works and does not matter, which given the router and the clustering is a third such component and is publishable as one.

3.50 OpenRSI: routes to their published numbers that avoid the broken endpoint

Asked to find a path to OpenRSI's published results that does not go through the failing `/evaluate`, we checked three and report all three outcomes.

- **A pre-skew tagged release: does not exist.** `git tag` on OpenRSI is empty; there are no releases. So there is no version predating the NatureBench hardening that we could run unmodified, and the pin option is closed on availability as well as on security.
- **Shipped reference outputs or expected scores: none.** No expected, reference, golden, or baseline result files exist anywhere in the tree, so there is nothing to compare a local run against short of rerunning their protocol.
- **An offline grader: yes, and it is the MLE-Bench path.** Their MLE-Bench evaluation runs through `dojo/tasks/mlebench/evaluate.py`, whose `evaluate_submission(submission_path, data_dir, competition_id, results_output_dir)` grades against the `mlebench` package's registry and leaderboard with no HTTP service at all. The NatureBench eval-service skew therefore does not touch their headline benchmark.

The catch is cost and comparability, not plumbing. `mlebench` is an external dependency needing prepared Kaggle competition data per task, and Ornith publishes only aggregate Medal Average over the full 22-task Lite split at three runs, with no per-run or per-instance values, so a subset run is not comparable to anything they report. Verifying their published numbers by this route is the 1584-sandbox-hour path in Sec. 3.44 and needs a deliberate decision. For NatureBench specifically the token-plus-assertion route remains the only option.

Reported upstream. We opened `FrontisAI/OpenRSI` issue #12 describing the skew: both commit hashes, the control-token gate on `/register`, the simultaneous change of `/evaluate` to require an opaque per-task token and to stop honouring a per-call `output_dir`, and the resulting hazard that a minimal fix would score the wrong directory successfully rather than erroring. It states that their suites still pass 119/119 and that their published results predate the change.

3.51 A fifth of A0's negative gradient pushes down on rollouts that were on their way to a correct answer

The mechanism, and it is not rare. The reward is `math_reward_fn`: `float(verify(parse(completion), parse(gold)))`, with a bare `except` returning 0.0. It never inspects a finish reason. A rollout that hits the 1024-token generation cap stops mid-derivation, offers the verifier no answer to parse and scores 0 — the same 0 a confidently wrong answer scores, and after `reward_bias`= −0.5 and `reward_scaling`= 10 the same −5. `overlong_reward_penalty` is `false`, but that only disables an *additional* penalty; the implicit one, through the extractor, is unconditional. On A0's own rollout dumps (154,856 generations over 1,419 steps; over the restart-free region from step 520, 57,272 rows in 7,159 complete groups of eight) **11.4**% of generations reach the cap and 10.6% reach it and grade wrong, on either window. That is **26.3**% of every wrong answer over the restart-free region and 26.8% over the whole dump set; every figure below is the restart-free one. The rate is flat: per 100-step block it moves between 10.2% and 12.2%, sloping −1.8 points per thousand steps. The dumps reproduce the trainer's own accuracy to four decimals (0.5982 against `task_reward/avg` = 0.5986 over the same steps), so they are the batch the actor trained on and not a sample of it.

What it costs the update. With `adv_norm=null`, `kl_ctl`= 0, no critic, $\gamma = \lambda = 1$, GAE gives every response token of a row the same advantage, and group normalisation makes that $(r_i - \bar{r}_g)/(\sigma_g + \epsilon)$. The surrogate is a token mean, so a row's share of the gradient magnitude is $|A_i|$ times its response length. Replaying that pipeline on the dumps: truncated rows are 11.4% of rows, 17.5% of response tokens and **21.8**% of the batch's $|A|\times$token gradient mass — 2.18× the mass of an average terminating row, because the length that got them truncated also weights them. Of that, 19.0 points are negative advantage, which is **36.6**% of all negative-advantage mass in the run. Better than a third of the downward push in A0 sits on rows whose label was never checked.

They are unfinished, not degenerate, and 39.9% **of them would have been right.** Of the 6,167 truncated-and-wrong rollouts after step 520, 92.8% carry no `\boxed{}` at all, show no repeating block covering more than half their tail, and compress no better than 4×: the budget ends them, not a loop. What that costs was then measured rather than argued. Every one of the 293 truncated-and-wrong rollouts drawn in steps 1375–1424 was re-fed to the policy as its prompt *plus its own* 1024 *truncated tokens* and allowed to run on for up to 2,900 more, four continuations each at the same temperature 1.0 the rollouts were drawn at, on the LoRA checkpoint at `globalstep1399` (the adapter moves by at most 2.4% in relative Frobenius norm across that window, so one checkpoint covers it). This is the exact counterfactual and not a resample: a fresh draw from the same prompt is a different trajectory, while a continuation is the same one with more room.

99.9% **of the continuations terminated**, needing a median of 154 and a mean of 209 extra tokens — these rollouts were a paragraph short, not a chapter. The recovery rate is **0.3985**, 95% CI $[0.3473, 0.4488]$ by a cluster bootstrap over the 293 rows. The free upper bound stated in advance was 0.4610, the rate at which these rows' *untruncated* siblings solve the same prompt, and the interval lands inside it. Row by row, 49.5% recover at least one continuation of four and 31.4% recover all four.

On the rows that carry gradient, recovery is higher, and that is the defect. A row contributes nothing unless its group disagrees with itself, so the rows that matter are the 165 truncated-wrong rows with a strictly negative advantage — and those sit on prompts a sibling did solve, which are the recoverable ones. Weighting each row by the gradient mass it actually carries, the recovery probability on them is **0.6439**, 95% CI [0.5628, 0.7174]. Those rows hold 32.7% of the negative-advantage mass in this window (36.6% over the longer restart-free window). Multiplying through per row rather than multiplying two aggregates: **21.1**% of A0's negative-advantage gradient mass, and **11.0**% of its total gradient mass, is a push *away* from a rollout that was on its way to a correct answer. That figure needs no assumption about learning. It is a defect in the training signal whether or not it explains anything about the curve.

Raising the cap buys 2.8 accuracy points of training signal for 1.4% more generation time, and that is measured, not extrapolated. The same 400 prompts the trainer drew in that window — every question in it, not only the ones that truncated, because the cost of a bigger cap is a property of the whole mix — were redrawn four times at a 1,024-token cap and four times at 3,900. The 1,024 arm reproduces what the trainer saw, which is the check that this is the same policy on the same data: accuracy 0.6331 against the window's own 0.6422, hit-cap 0.1113 against 0.1000, mean 660.3 generated tokens against 653.3. At 3,900 the hit-cap rate is **0.0000** — not one of 1,600 draws reaches it — accuracy rises by $+\mathbf{0.0275}$ $[+0.0031, +0.0519]$, and the mean response grows from 660.3 to 669.7 tokens, a factor of 1.014. On the prompts that truncate at all the gain is $+0.0938$ $[+0.0385, +0.1490]$ for +7.6% tokens.

The parametric relation quoted earlier, that a mean length of 667 grows by $0.114 \cdot L$, was right; the guess at L was not. Measured, the rollouts that exceed 1,024 tokens need about 84 more on average — $0.114 \times 84 = 9.6$ tokens against the 9.4 observed — not the 250 to 2,000 the illustration spanned. The distribution does *not* broadly lengthen when the cap is lifted: the 90th percentile at a 3,900-token cap is 1039 tokens, so the 1,024 cap was cutting at about the 89th percentile of a distribution that essentially ends there. Nearly all of the cost of a bigger cap is paid on the tenth of rollouts that wanted it.

What is launchable is 1,536**, not** 3,900**, and it is enough.** The cap cannot simply be raised on the box A0 runs on, and the preflight is right to refuse. A sequence longer than `actor.mb_spec.max_tokens_per_mb`= 2048 cannot be split and forms its own oversized microbatch, against a measured 9.95 GB transient pool and a 3.37 GB margin, so `train_dataset.max_length`+`gconfig.max_new` must not exceed 2048 — which at `max_length`= 1024 pins the generation cap at exactly 1024. Asked for 1536, 2048 or 3072 it refuses all three, and it should.

The slack is in the prompt half of that sum, which nothing was using. Over 8,656 sampled rollouts the longest prompt the trainer has ever served is 676 tokens and the 99.9th percentile is 590, so `max_length` can come down to 512 at the cost of 0.28% of prompts (to 768 at the cost of none) and hand the difference to generation. Redrawing the same 400 prompts four times at each of three caps:

cap	accuracy	paired Δ vs 1024	hit-cap	mean gen. tokens	×1024
1024	0.6306	—	0.1081	657.6	1.000
1280	0.6538	+0.0231 [+0.0006, +0.0456]	0.0319	672.7	1.023
1536	0.6825	+0.0519 [+0.0275, +0.0769]	0.0100	673.4	1.024

Both intervals exclude zero, and the two caps cost the same 2.4% in tokens because the extra budget is only ever spent by the tenth of rollouts that wanted it. So `MAX_PROMPT_LEN=512`

`MAX_NEW_TOKENS=1536` removes 91% of the truncation, buys 5.2 points of training-signal accuracy, costs 2.4% more generated tokens and 0.28% of prompts, and passes the preflight at exactly 2048. That configuration is the arm we would run; it was verified to start and was not started, because launching it is a decision about days of GPU rather than about this measurement.

Excluding is not free, and on this run's own dumps zeroing dominates it. Two repairs are available: give the truncated row no advantage of its own but leave the group baseline alone (*zero*), or additionally drop it from the group mean and standard deviation (*exclude*). Exclude is the more principled — an unverified row should not set the bar its siblings are judged against — and it is the more expensive, because it shrinks the group, and a group whose members agree contributes exactly zero. Counterfactually on the same 7,159 groups: 52.1% are already unanimous and silent; excluding truncated rows drives a further **7.8**% of all groups to unanimity (16.3% of the groups that currently carry any signal) and empties 2.0% outright, leaving 68.4% of the gradient mass. Zeroing leaves 78.2% and makes no group newly unanimous, because every surviving row's advantage is bit-identical to the baseline's. Neither is asserted better, and on the measurement above neither is the first thing to reach for: a bigger cap removes most of the population these two policies govern, for a couple of percent in tokens. The ordering we would defend is raise the cap first and set `truncated_advantage=zero` for whatever still truncates — it preserves 78.2% of the gradient mass against exclude's 68.4% and kills no groups — rather than re-weighting a defect that can simply be removed.

No run in this project had a truncation instrument, and the ready-made flag rests on the same broken predicate. `ppo_actor/no_eos_ratios` compares a sequence's length to the *padded batch width*, so it flags whichever rows happen to be longest. On A0 it averages 0.0835 against a true cap rate of 0.1138 and its per-step correlation with the true rate is $r = 0.04$ over 895 steps: the mean is close enough to be mistaken for the quantity and the series carries none of it. `mask_no_eos_with_zero`, the configuration flag that already exists for exactly this repair, is driven by that same predicate, so switching it on would zero a nearly unrelated 8% of rows. The correct predicate — response length against the cap — existed in the trainer as `_truncated_rows` but was computed only when group routing was enabled, which A0 is not.

Two corrections to the record, both of them ours. Both were stated in this project's own reporting before being checked, and the inflated versions have already been passed on, so they are retracted here explicitly rather than quietly superseded.

First: token budget is not worth 32 *accuracy points, and we do not train at a sixteenth of the evaluation budget in any sense that costs accuracy.* The claim came from setting the headline OlympiadBench number at 16,384 tokens, 0.43–0.47, beside the in-training trend score at 2,048, 0.11–0.14. Those are different problem sets. The periodic evaluation takes the first 64 problems of the OlympiadBench search half, and the headline run itself scores **0.125** on those same 64 indices at 16,384 tokens against **0.141** for the periodic evaluation at 2,048 — a one-problem difference in the other direction. The budget does not bind at either setting: 1 of 64 generations stops on `finish_reason=length` at 2,048 and 7 of 675 at 16,384, and greedy responses average about 2,230 characters in both. The 0.47 is the mean over all 675 problems and the 64 are a hard prefix of them. Nothing about the evaluation budget was ever the story.

Second: the mean length of an incorrect answer is 870 *tokens, not* 982. The 982-versus-802 pair that motivated this investigation was a single step's reading of `ppo_actor/{correct,incorrect}_seq_len`. Averaged over the run those are **869.9** and **719.2** tokens of *sequence* — 760 and 607 of generated

response once the 109-token mean prompt is removed. `incorrect_seq_len` does reach 982, at 26% of steps. The qualitative claim that wrong answers are much longer survives; the specific numbers were a snapshot quoted as an average.

So the mechanism is real and we do not claim it is the null. Three things argue against truncation explaining the flat curve. The truncation rate is constant across 1,419 steps and so is response length (870 tokens of sequence for a wrong answer against 719 for a correct one, both flat to within 2% end to end), which is not what a run optimising against length looks like. 52.1% of groups are unanimous and contribute exactly zero whatever is done about truncation. And a perfect repair moves at most the 21.8% of gradient mass identified above. What can be said, and is now measured rather than argued, is that this is the largest identified source of label noise in A0, that about a third of its negative gradient rests on rows the verifier never checked, and that a fifth of that negative gradient has the *wrong sign*. Whether repairing it moves the curve is a question for the arm, not for this measurement, and we do not pre-announce the answer.

Three checks that had to pass before any of this counted. The grader used throughout is the trainer's own `math_reward_fn`, and it reproduces *all* 3,200 stored rewards in the window, so a recovery measured with it is measured with A0's own notion of correct. Decoding a dumped rollout back to text and re-tokenising it round-trips exactly on 293 of 293 truncated rows — prompt length and response length both — so a continuation resumes from the token state its rollout was actually in. And the adapter had to be shown live: sglang lists `a0_v1399` in `/v1/models` and answers requests naming it with 200 and fluent mathematics, but those completions are **bit-identical to the base model** over 144 greedy tokens. The adapter is applied only when it is passed as the request's `lora_path` field, where it moves the logprobs by up to 0.061. A run that had trusted the model name would have regenerated with the base model and reported the result as A0's policy, with nothing in the response to say otherwise; the client now refuses to generate unless that difference is non-zero.

What was built, and what it does by default. `PPOActorConfig` gains `truncated_advantage` $\in$ {`keep`, `zero`, `exclude`}, defaulting to `keep`, which is bit-identical to every prior run; a misspelling is refused rather than defaulted, so an arm cannot report the fix and run the baseline. The cap-based truncation rate is now emitted on every step as `ppo_actor/truncated_ratios`, beside `no_eos_ratios` rather than instead of it, so old and new runs are not two readings of one name. Fourteen tests pin all three settings against hand-computed group statistics, and seven mutations of the seam are all killed; the first version of the harness found a guard that could not fire, which was removed, and a dry run found that the field is on the dataclass but not in the composed YAML, so the override needs hydra's append form. The preflight refuses a launch whose resolved config does not carry the arm the launcher asked for. Separately, `run_a0.sh` and `preflight_a0.py` — which between them decide which arm runs — lived in a directory that is not a git repository; they are now in the tree, and the preflight sha256s the live file against the committed one and refuses the launch when they differ, so an unversioned edit stops a run instead of quietly changing an experiment. Nothing was changed in the running baseline: no config, no restart, and its logs, dumps and checkpoints were read only.

3.52 The post-credit window: one covariate survives both controls

All four arms ran to the end of their ten epochs, 640 steps, and the per-prompt arms received exactly 4096 credits each — 512 steps $\times$ 8 groups, i.e. every step from 128 on, which is what Eq. 7

Table 38: Post-credit window, steps $[128, 640)$, 4096 decisions per arm. Each column is a *difference* against a different control and they answer different questions: the random router isolates *does the router use the features at all*, the shuffle isolates *is the credit attached to the prompt that earned it.* Brackets are 95% bootstrap intervals over steps. Bold marks a difference whose interval excludes zero against *both.*

subset contrast	− rate-matched random	− correspondence shuffle	− batch credit
cov. `mean_logprob`	**+0.180** [+0.113, +0.232]	**+0.156** [+0.096, +0.210]	−0.056 [−0.120, +0.007]
cov. `mean_response_len`	+0.193 [+0.125, +0.244]	+0.061 [−0.006, +0.128]	+0.065 [−0.006, +0.132]
cov. `logprob_dispersion`	+0.095 [+0.031, +0.132]	−0.079 [−0.136, −0.018]	−0.201 [−0.252, −0.139]
cov. `len_dispersion`	+0.013 [−0.041, +0.074]	−0.025 [−0.087, +0.039]	+0.041 [−0.017, +0.090]
silence (solved vs unsolved)	+0.153 [+0.094, +0.203]	+0.065 [−0.004, +0.135]	+0.107 [+0.052, +0.158]

predicts and not an approximation to it. Table 38 reports the window $[128, 640)$: 4096 routing decisions per arm, made while credit was flowing.

What the two controls separate, and why only one row is bold. Against the rate-matched random router the learned arm separates the covariate subsets on three of four covariates, the largest being `mean_response_len` at +0.193. Taken alone that reads as targeting. It is not: the correspondence control — same ledger, same pairings, same multiset of credit values, only the prompt-to-credit correspondence destroyed — reproduces almost all of it, leaving +0.061 with an interval containing zero. So the separation on `mean_response_len` comes from the router receiving a *varying* signal at all, not from that signal being attached to the prompt that earned it. On `logprob_dispersion` the shuffled arm is *higher* than the treatment.

One covariate survives both: `mean_logprob`, +0.180 against random and +0.156 against its own shuffle. That second number is the one that carries the claim, and at 5.3 standard errors it clears a Bonferroni threshold of $|z| > 2.94$ for the fifteen comparisons in this table. It is the first evidence in this project that a learned router targets on a covariate rather than on the pass count, and it is one covariate out of five, from one seed.

Three cautions that belong with the number. First, *the batch-credited arm is not inert on this statistic*: it beats the treatment on `logprob_dispersion` by 0.201. Subset contrast measures differential treatment of subsets, which any context-dependent router produces; it is evidence of targeting only in the difference against the shuffle, which is exactly why that control exists and why the L_1 signature was retracted as a validator. Second, *the router's mixture moves once credit arrives*: the treatment realised `rl` = 0.4471 over the whole run against `rl` = 0.3372 inside the post-credit window, so a control matched to the whole-run mix would be matched to a regime the comparison never uses. The control reported here is matched to the window's own measured mix. Third, *agreement with the hand-written rule falls* on two of three silence subsets (0.267 vs 0.305 on solved, 0.419 vs 0.623 on unsolved), so the learned router is not reproducing the rule; whether its disagreement is an improvement is a benchmark question these arms do not answer.

Replication in flight. A second seed of the treatment, its matched random control and the correspondence shuffle is running, together with the hand-written rule arm that the first round never launched. One covariate out of five surviving in one seed is a result to replicate before it is leaned on, and the comparison that must replicate is the `mean_logprob` column against the shuffle.

Table 39: Any statistic that is a function of $\hat{p}$ alone cannot separate the gate from a control matched to its own $\hat{p}$ histogram. Beta(1.5, 3.5) pool, $k = 8$, $N = 400{,}000$, top decile by D.

selector	mean $\hat{p}$	energy / rollout	gradient norm $\sqrt{\hat{p}(1-\hat{p})}$
gate, top decile by D	0.2500	1.0000	0.4330
C1, matched to the gate's histogram	0.2500	1.0000	0.4330
C2, band $0.1 \le \hat{p} \le 0.9$	0.3595	1.0000	0.4281

3.53 Running the gate's outcome experiment: two feasibility tests, and the operating point that passes them

Sec. 3.49 left the outcome question to a real model and `PREREG_gate_outcome.md` fixed the design in advance. This section reports what happened when we took it to `Qwen3.8-27B` on eight H200s: why the arms were not started at the default operating point, and the measurement that identifies one where they can be. **Nothing here is an outcome for arm T, C1 or C2, and no stop rule has fired.** The pre-registration stands unamended. **And nothing here is a test of Ornith-1.5's loop:** what we measured is their difficulty criterion applied to a *fixed* benchmark pool, whereas their published method *generates* its tasks. §3.53 states that scoping in full, and it matters most exactly where our numbers look worst for them.

First, a correction that binds the design: C1 ties on every secondary outcome as arithmetic, not as a simulation artefact. The pre-registration records that the matched-difficulty control is vacuous "in a difficulty-only simulation", because there θ is the whole task. That is true and weaker than the situation warrants. Advantage energy per rollout is exactly $1 - \Pr(\text{unanimous})$, a function of the selected tasks' $\hat{p}$ distribution and of nothing else, so a control resampled to the treatment's own $\hat{p}$ histogram matches it identically — on a real model as much as in simulation. We re-derived Part A's arithmetic independently before reading it and recover its idealised rows exactly (0.8322 at $p^\star = 0.2$, 0.9922 at 0.5, $G = 8$); the tie is then exact in a 400,000-task draw (Table 39).

The consequence is procedural. The pre-registration's *secondary* outcome — "advantage energy actually delivered per token in each arm" — cannot separate T from C1 even in principle, so a flat primary outcome must not be read as partially rescued by it. Only held-out capability can carry the comparison. Two smaller corrections belong with it. Under the gradient-norm measure this programme adopted in place of energy, C2 does *not* dominate the gate (0.4281 against 0.4330), so the case against the difficulty kernel also has to be made on held-out capability. And `gate_outcome.py` draws $\theta \sim \text{Uniform}(0, 1)$, so Table 37's "size-matched uniform" row describes that synthetic pool rather than a benchmark: a real pool at a strong model's operating point is skewed toward $\theta = 1$, where almost every group is unanimous, so the true no-selection row is *worse* than 0.7515 and the gate's margin over no selection is understated by the synthetic table. That correction favours the gate, and we record it for the same reason we recorded the ones that did not.

Second: the held-out venue offers 25 movable items, and the pipeline's own demonstrated effect is +0.89 points. Counting from the committed greedy baseline rows rather than from a percentage, and separating the failures no policy could win (Table 40), the report half offers 25 items — 7.42 points of total possible movement, and that is the ceiling on the *whole treatment*, not on the difference between two selection rules. Set against it, Sec. 3.38 measured this programme's full pipeline on this same benchmark at +0.89 points, 95% CI [−0.56, +2.39], with the instrument

Table 40: OlympiadBench decomposed by the committed split, from a greedy baseline at a 57,344-token cap (`olymp_qwen38_20260903T170519Z`, 675/675 graded, dataset md5 `77315ce5...` verified on load). Its pooled accuracy 0.8148 sits between the 32,768 and 65,536 rows of Table 18, as the cap series predicts. A truncated generation is a budget artefact and an unextractable one a formatting artefact; neither is capability.

half	n	solved	wrong, truncated	wrong, no box	**improvable**
search	338	272	36	2	28 (0.0828)
report (held out)	337	278	25	9	**25** (**0.0742**)
all	675	550	61	11	53 (0.0785)

able to resolve a true effect of 2.4 points, over 549 steps across which the adapter grew 46×. That null was obtained at a base scoring 0.43, with more than fifty points of headroom available. Moving to a base with seven cannot improve the situation, and a T-versus-C1 contrast is a second-order difference on top of a first-order effect that is already indistinguishable from zero.

Third, and not anticipated by the pre-registration: at the rollout sampling distribution most generations never terminate. Every OlympiadBench figure in this paper is *greedy*. A GRPO rollout is not. Measured at the rollout distribution — temperature 1.0, top-p 0.95, the same 57,344-token cap, sixteen samples — the truncation rate is **0.625** and the harness declares the run `cap_limited`. One problem resolved six of eight samples; the other resolved *none*, its eight completions running 163k–192k characters without ever closing the reasoning block.

This is not only a cost problem, it changes what the gate selects. A task whose rollouts all truncate scores $\hat{p} = 0$ and is assigned $D = \exp\big(-(0-0.2)^2/(2\cdot 0.15^2)\big) = 0.411$, a substantial weight. **At the default thinking budget Ornith's difficulty gate preferentially selects tasks that fail to terminate**, not tasks of the intended difficulty, and shortening the rollout budget toward anything an RL loop could afford makes that worse rather than better. The gate's input is a budget measurement before it is a difficulty measurement — a defect of the published mechanism as applied, not of our reimplementation of it.

A silent zero that would have looked like a difficulty distribution. The harness sends a fixed `--seed 0` unchanged on every request. Had the server honoured it per-request, all sixteen samples of a problem would have been identical and every $\hat{p}$ would have been exactly 0 or 1 — a plausible-looking, entirely artefactual difficulty distribution in which every group is unanimous. It does not: all sixteen completions per problem are textually distinct, confirmed separately by two identically seeded requests returning different text. We record the check because its failure mode is invisible in the output.

The operating point that passes both tests, and it is the thinking budget rather than the benchmark. `Qwen3.8-27B`'s chat template exposes `reasoning_effort` ∈ {`xhigh` (default), `medium`, `low`}. Re-running the same problems and the same sampling distribution at `low` (Table 41) improves both at once without fixing either completely: truncation falls from 0.625 to 0.210, median rollout length falls by more than an order of magnitude to a budget an RL loop can afford, and accuracy falls away from ceiling, which *enlarges* the improvable set the first test was short of. The two requirements looked opposed — the benchmarks this base does not saturate are the ones on which it does not terminate — and the thinking budget separates them.

Table 41: The same benchmark and the same sampling distribution (temperature 1.0, top-p 0.95) at two thinking budgets. The `xhigh` row is 16 generations over two problems, enough to establish that the median rollout *is* the cap and not enough for a difficulty distribution; the `low` row is 5408 generations over 338 problems of the committed search half. `tokens` are counted with the model's own tokenizer.

`reasoning_effort`	cap	truncated	median tokens	min tokens
`xhigh` (default)	57,344	0.625	57,344 (the cap)	35,035
`low`	16,384	0.210	1,955	236

Table 42: The two-block difficulty measurement at the terminating budget, on the committed search half. Each problem is sampled 16 times and split into an independent selecting block and fresh block of $k = 8$; `dead` is the fraction of groups that are unanimous and therefore carry exactly zero GRPO advantage. This is the real-model counterpart of Table 37, whose pool was $\theta \sim \text{Uniform}(0, 1)$.

	mean $\hat{p}$	dead-group fraction	energy / rollout	$\hat{p} \in [0.10, 0.35]$
real pool, no selection (block A)	0.8622	0.8095	0.1905	3.7%
synthetic uniform pool (Table 37)	0.481	0.2485	0.7515	—

At the default budget the *median* rollout is the cap itself and even the shortest of the sixteen ran to 35,035 tokens, so the model does not finish its reasoning on these problems at all; at `low` the median falls to 1,955 tokens, a factor of 29×, and truncation falls from 0.625 to 0.210. That is a real improvement and not a cure: it remains more than twice the 0.093 the *greedy* baseline shows at a cap three and a half times larger, so a fifth of rollouts still end at the cap and every figure below is computed on resolved samples only, with the truncated ones treated as unknown rather than wrong. Accuracy falls too, which is the point: it moves the base off the ceiling that left the held-out half with 25 movable items.

The prediction made above is confirmed and its size is larger than expected: an unselected real pool is far deader than the synthetic uniform one, 0.8095 against 0.2485, because a strong model's difficulty distribution piles up at $\hat{p} = 1$ rather than spreading over $(0, 1)$. Two consequences follow for the pre-registered arms, both of which change how they must be run rather than what they will find. The gate's budget-efficiency margin over no selection is much larger on a real pool than Table 37 suggested. And the pool the gate actually selects from is small: only 3.7% of the search half sits in the kernel's effective support $\hat{p} \in [0.10, 0.35]$, so the arms must be sized against that handful rather than against the 338-problem half. **The gate's difficulty on a real model is scarcity, not noise.** The simulation diagnosed the gate as a fine kernel applied to a statistic too coarse to resolve it, and predicted that selecting-block difficulty would regress hard on a fresh block ($0.946 \to 0.551$ at $k = 8$). On this pool the opposite holds: difficulty is close to bimodal, so $\hat{p}$ carries little binomial noise, the split-half reliability of a $k = 8$ measurement is 0.871, and the realised selecting-to-fresh shift on the gate-selected tasks is -0.0123 rather than the large positive regression the simulation predicted — and that same bimodality is exactly why there is almost nothing at the target difficulty to select. **The gate's problem on a real model is scarcity, not noise**, and the two call for different repairs: shrinkage fixes noise and does nothing for scarcity.

Retracting the repair we proposed. The companion analysis of the gate's estimator recommended one change to Ornith's gate and predicted it would help: replace $D(\hat{p})$ with $D(\tilde{p})$, where $\tilde{p}$ shrinks $\hat{p}$ toward the pool mean by a James–Stein factor, on the reasoning that the kernel is fine and

Table 43: Published against substituted. Ambiguity ids are those of `ornith_repro/AMBIGUITIES.md`. Nothing in the right column is a criticism of their method; it is a statement of what a reader of their method page can and cannot pin down.

quantity	published?	ours
target success rate $p^\star$	**yes**, 0.2	0.2
kernel width σ	no (A2)	0.15
rollouts per task k	no (A1)	8, and 16 here to give two blocks
GRPO group size G	no (A3)	8
similarity behind $N(q)$	no (A6)	Jaccard over tokens
C, F, H of R_{harness}	named only	frozen judges, never trained (A11)
scaffolds/rollouts per task	figure only (A16, A17)	nested, 3×8
stage update order	no (A10)	solver $\rightarrow$ harness $\rightarrow$ proposer
base model	Qwen3.5, Gemma 4	`Qwen3.8-27B` (A14)
benchmark	agentic, repository-scale	OlympiadBench

the statistic it reads is coarse. **That repair addresses a problem this pool does not have.** Shrinkage buys accuracy when $\hat{p}$ is noisy; here the split-half reliability of a $k = 8$ measurement is 0.871 and the selecting-to-fresh shift is -0.0123, so there is almost no noise for it to remove, and shrinking toward a pool mean of 0.8622 would drag every candidate *away* from the target rather than toward it. We stated in advance that if shrinkage did not help, the shrinkage thread should be dropped for tasks. It does not, and we drop it.

What this measures, and what it cannot show. Ornith's task reward is a product of three factors, $R_{\text{task}} = V \cdot D \cdot N$: a validity gate, the difficulty kernel, and novelty against a buffer of tasks the system has already produced. **On a fixed curated pool two of those three are inert by construction.** Every OlympiadBench problem is valid, so $V \equiv 1$; there is no generation buffer, so N is the constant the implementation returns against an empty archive. What we exercised is D alone, and we applied it to a difficulty distribution the method never chooses.

Ornith does not select from a pool. It proposes tasks "that go beyond what the model has already solved". A generator is not bound by the benchmark's difficulty distribution: it can aim at the mixed region directly, and the novelty term exists precisely to push it somewhere the buffer is not. So the scarcity in Table 44 is **a property of pool-based selection, not a refutation of Ornith-1.5**. Read the other way round it is the strongest argument in this paper *for* their design: if only

3.7%

of a curated frontier-mathematics half sits where the kernel has weight, and 0.7130 of it the model already solves every time, then a loop that selects from such a pool is starved, and generating tasks is the sensible response rather than an ornament. Our reimplementation supports generation; this measurement simply does not use it.

Which parts of this are theirs and which are ours. $p^\star = 0.2$ is the only hyperparameter Ornith-1.5 publishes. Everything else in Table 43 is a choice of ours standing in for one of theirs, and a reader is entitled to see the list rather than infer it. Two entries do real work in the numbers above: σ sets how wide the "gate's support" column is, and the rollout count k sets how finely $\hat{p}$ is resolved at all.

The reasoning budget cuts the same way, and also in their favour. These figures come from `reasoning_effort=low`, chosen because the default does not terminate. That budget also

Table 44: Informative groups are rare, and group size is a bounded lever. Exact probabilities from the measured outcomes of the 230 problems that still hold all sixteen samples once truncated generations are dropped as unknown: for a problem with c of n resolved samples correct, a group of G drawn without replacement is unanimous with probability $[\binom{c}{G} + \binom{n-c}{G}]/\binom{n}{G}$. "Informative" is one minus that, which by the energy identity is also the advantage energy delivered per rollout.

group size G	informative	dead	in the gate's support $[0.10, 0.35]$
2	0.0689	0.9311	0.0000
4	0.1284	0.8716	0.0193
8	0.1928	0.8072	0.0226
16	0.2565	0.7435	0.0217

lowers accuracy — and lower accuracy means *more* mixed problems, not fewer. So the mixed fraction reported below is the optimistic end: at the default thinking budget the model is stronger, more problems become always-solved, and informative groups would be rarer still. Every scarcity number here is a lower bound on the difficulty of pool-based selection.

The scarcity, quantified, and it is not a property of the rollout budget. A group contributes gradient only when its members disagree. Table 44 gives the exact hypergeometric probability of that, per group size, computed from each problem's own sixteen resolved outcomes rather than from a fitted difficulty model.

An always-solved problem cannot produce a disagreeing group at any G, so the ceiling on informative groups is the fraction of the pool that is genuinely mixed — here 0.2565, against 0.7130 the model solves every time out of sixteen. Raising G from 2 to 16, an eightfold increase in cost per task, moves the informative fraction only from 0.0689 to 0.2565, and leaves the share of groups landing in the gate's support at 0.0217. Group size is a real lever on wasted rollouts and a strictly bounded one: it cannot manufacture a task the model sometimes solves.

Three measurements, one shape. This is the third independent count in this paper of training signal that does no work, and they do not overlap in method. Sec. 3.10 shows the router's credit signal is uninformative *by construction*, the per-arm counts cancelling so that every arm converges to the same vector. Sec. 3.51 finds 21.1% of A0's negative-advantage gradient mass pushing *away* from rollouts that were on their way to a correct answer. And here, at the operating point a self-improvement loop would actually run at, 0.8095 of groups carry exactly zero advantage. A companion reachability analysis adds a fourth from the weight side: the displacements actually applied are first-order capability-null, while a data-derived direction of identical norm separates at a factor of 442. **If informative groups are this rare, the binding constraint on this literature is manufacturing them, not choosing among them** — which would explain why a learned router and a clustering method both sit on the same flat trunk. We do not extend that list to Ornith's gate: we never ran it as specified, and the one factor of it we did run was applied to a pool its authors would not have selected from. We state that as the reading these four measurements invite, not as a result they establish: none of them was designed to test it, and the experiment that would is a different one from any run here.

What we did and did not commit GPU-days to. We did not start T, C1 and C2 at the default operating point. A run there would have returned a difference smaller than the instrument, and the pre-registration's stop rule — "if T versus C1 does not separate beyond the noise band, the

gate works and does not matter" — would have fired for want of an instrument rather than for want of an effect. That is the false negative this programme has already documented, in which "the method does not work" and "there was no room left" produce the same table; firing a pre-registered stop rule on it would have converted a measurement failure into a published finding. What we did run is the two-block difficulty measurement the pre-registration's matching rule 2 requires and which did not previously exist for any real model: every problem in the committed search half sampled sixteen times at the terminating budget, split into an independent selecting block and fresh block of $k = 8$, so that the control can be matched to fresh-block difficulty rather than to the inflated selecting-block value.

3.54 Running the pre-registered arms on eight B200s: the mixed subset, four silent traps, and what moved

Sec. 3.53 declined to start T, C1 and C2, on the grounds that a run at the default operating point would return a difference smaller than the instrument. This section reports the run, on a machine that removes the constraint that had been mistaken for an engineering one. Two things changed.

The hardware. Eight B200s at 183 GB each. A 27B policy in bf16 is 52 GB, so a LoRA trainer needs no sharding at all and a rollout server on a second device still has $\sim$100 GB of KV cache. The four arms therefore run *concurrently*, one trainer and one server each, on eight identical devices — rather than in sequence on 80 GB boxes where the same experiment needs FSDP and the arms cannot share a hardware generation. Every number below comes from that one machine in one session.

The pool. The arms train on the **mixed subset**, not on the search half. A group contributes gradient only when its members disagree, and an always-solved problem cannot disagree with itself at any group size; training on a pool that is mostly always-solved is the condition under which "the method does not work" and "there was no room left" produce the same table. The subset is established below from this box's own rollouts. We did not inherit a list.

The rollout cap is measured, not assumed, and the measurement is the finding. Four budget failures in this programme have the same shape: a cap chosen for the length of the visible answer while the model emits its reasoning first, so the answer never arrives and the failure reads as a refusal. So the whole committed search half was sampled sixteen times at `reasoning_effort=low`, temperature 1.0, top-p 0.95, at a deliberately generous 16,384-token cap: 5408 generations, zero harness errors. The distribution is not long-tailed, it is *bimodal in termination* (Table 45). Half the generations finish inside 1277 tokens; and 31.7% never finish at all. Doubling the cap from 8192 to 16,384 buys 7 points of truncation for 66% more tokens per rollout.

The model does not get these problems wrong; it fails to stop. Accuracy over the resolved generations is **0.9480**, against 0.6477 once truncated generations are scored as the benchmark scores them. The gap is the whole story: at this thinking budget the binding constraint on OlympiadBench for `Qwen3.8-27B` is termination inside the token budget, not mathematics. That single measurement decides the pool, the reward and the outcome measure, and it is why both truncation conventions are reported everywhere below.

The mixed subset, established here, and how large it is depends on one convention. A problem is in the pool when at least one of its sixteen samples is scored correct and at least one is not. Whether a truncated sample counts as *wrong* or as *unknown* changes that count by a factor of two and a half (Table 46).

Table 45: Completion length at the rollout sampling distribution, from 5408 generations over the 338 committed search-half problems. Quantiles are over the 3695 *resolved* generations only: a truncated one's length is the cap, not its length, and mixing them in makes every upper quantile equal the cap. The right two columns are what each candidate cap would have cost, computed from the same samples.

statistic	tokens	cap	truncated	mean tokens/rollout
median	1,277	4,096	0.4601	2,580
p90	8,301	8,192	0.3874	4,293
p95	11,912	12,288	0.3482	5,792
p99	15,360	16,384	0.3168	7,147

Table 46: The mixed subset of the 338-problem search half, under both truncation conventions, from the same 5408 samples. "informative" is the exact hypergeometric probability that a group of G drawn from a problem's own sixteen outcomes disagrees, averaged over the half — which by the energy identity is also the advantage energy delivered per rollout.

convention	always solved	never solved	**mixed** (pool)	informative at $G =$ 2	4	8	16
truncated = wrong, cap 16384	180	94	**64**	0.0533	0.0981	0.1416	0.1893
truncated = wrong, cap 8192	159	120	**59**	0.0453	0.0842	0.1265	0.1746
truncated = unknown, cap 16384	201	8	**25**	0.0184	0.0357	0.0640	0.1106

The arms train under *truncated = wrong*, and the reason is not a preference. Under *unknown* the gate's effective pool — the inverse participation ratio of its own sampling weights — is **2.7** *problems*, and its matched control C1 lands on the same 2.7; there is no experiment there, only three problems memorised four hundred times. Under *wrong* at the training cap the effective pools are 14.4 (T), 17.9 (C1) and 42.0 (C2) out of 59. The convention also aligns the reward with the outcome the arms are judged on, and with the measurement above: what this model has to learn here is to finish. Its cost is stated rather than hidden — $\hat{p}$ then carries the token budget as well as the mathematics, which is exactly the contamination that made the *default* thinking budget unusable in Sec. 3.53. The mixed-subset filter removes the worst of it, because a problem that never terminates is never mixed and is excluded by construction.

Restricting to the mixed subset is the whole headroom, and it is a factor of five. At the training cap and $G = 8$, **0.8735** of groups drawn from the unrestricted search half carry exactly zero advantage. Measured over the run itself, the fraction of groups that carried advantage inside the mixed subset was **0.80** over the first ten steps of both T and C1 — a factor of **6.3** — falling to 0.40 (T) and 0.65 (C1) averaged over the whole run as the arms exhaust a 59-problem pool. That ratio is the difference between an experiment with headroom and one without, and it is available for free: it costs one pass of rollout statistics over the pool, which every one of these systems already produces and discards. The decay is the other half of the same fact — a pool this small is spent in a few dozen steps, which is an argument for *generating* tasks rather than selecting them, and is exactly what Sec. 3.53 says Ornith's design does.

One agreement worth recording. The split-half reliability of a $k = 8$ difficulty estimate on this pool is **0.878**, against the 0.871 measured independently on other hardware in Sec. 3.53. The two runs share no machine, no server process and no sampling seed, so this is a genuine replication of the claim that $\hat{p}$ on a real model is scarce rather than noisy.

Four silent traps, all of which fire and none of which is visible in a loss. Each was found by a check written to be able to fail, and each would have produced a complete, well-formed, gate-passing run.

1. **The adapter reaches 16 of 64 layers.** `Qwen3.8-27B` sets `full_attention_interval` = 4: 16 layers carry `self_attn.{q,k,v,o}_proj` and the other 48 carry a gated-delta-net `linear_attn` block with different module names. The `target_modules` inherited from the older Qwen match 64 modules on 16 layers — and additionally the multi-token-prediction head, giving 17 distinct layer indices in the checkpoint's own naming. Adding the MLP triple reaches all 64 layers but still 0 of 48 linear-attention modules. Adding `linear_attn.out_proj` gives 304 modules, 16/16 full-attention, 48/48 linear-attention, 64/64 layers. The coverage assertion counts *layers reached per family* and fires on both inherited configurations; it is reported firing, not merely passing.

2. **Naming the adapter in the `model` field is inert.** On the same prompt string, `/v1/completions` with `model=` *adapter* returns HTTP 200 and text byte-identical to the base route, while `/generate` with `lora_path=` *adapter* differs. Every arm here routes through `lora_path`.

3. **A "does the adapter change the output" probe is scale-dependent, in both directions.** At LoRA-B magnitude 0.02 only the MLP family moved a greedy completion; the full-attention family needed 0.05 and the linear-attention family 0.2. A fixed small probe therefore reports *not applied* for a family that is applied — our first run of it accused two of three families. The check now sweeps magnitude and reports the smallest one at which each family responds. In the other direction, comparing logprobs for *equality* is too fine: routing through the LoRA path adds BAx that is exactly zero for a null adapter, but the extra GEMM changes the bf16 reduction order, so a provably null adapter still shifts logprobs. The usable form is a live adapter measured against a null adapter on the same server.

4. **Gradient checkpointing was silently inert.** `from_pretrained` returns a model in `eval` mode and transformers skips checkpointing entirely unless `self.training`; the enabling call is accepted with no warning. Measured at the same micro-batch: 175.9 GiB allocated and an out-of-memory error, against **82.5** GiB after `model.train()`. The trainer now runs one worst-case forward and backward *before* the first rollout is paid for and refuses to start without headroom.

A fifth, which is ours rather than the stack's, and which the one-sample run caught. The pool was built with *truncated = wrong* while the advantage was computed by dropping truncated rollouts. On the first smoke step that combination reported accuracy 1.000 and an informative fraction of 0.00 — on a pool every member of which is mixed by construction — because a task selected precisely because it sometimes fails to terminate had its failures deleted and its survivors all agreed. The trainer now refuses to start when the convention it is given differs from the one recorded in the pool file.

And a sixth, in the serving stack. `/unload_lora_adapter` deadlocks on a server still holding requests whose client was killed mid-generation: it logs "Start unload Lora adapter", never returns, and sits at 0% GPU, which is indistinguishable from a slow step. On a clean server the unload/load pair takes 0.0 s and 0.2 s. Both calls are now bounded, so the failure aborts the arm instead of stalling it silently.

Table 47: What each selector actually draws from the 59-problem mixed subset. "effective pool" is the inverse participation ratio of the selector's own weights, i.e. the number of problems it behaves as though it had. The C1 row is the control working: it matches T on the independent block ($\hat{p}_B$) and differs on the block the gate reads ($\hat{p}_A$), so it is a different rule that lands on the same difficulty rather than a relabelling of T.

arm	rule	mean $\hat{p}_A$	mean $\hat{p}_B$	effective pool
T	kernel at $p^\star = 0.2$, $\sigma = 0.15$	0.175	0.219	14.4
C1	matched to T's $\hat{p}_B$ histogram	0.221	0.220	17.9
C2	band $0.1 \le \hat{p} \le 0.9$	0.637	0.688	42.0
C3	T's draws, reward $\sim$ fair coin	0.175	0.219	14.4

Micro-batching is a rewrite of the measured quantity, so it is shown to be a no-op. Right padding lets several rollouts share one forward, which is what makes a 27B on-policy loop affordable; on a model whose 48 recurrent layers could consume padding it is also a way to change every logprob the gradient is built from. A fixed tolerance cannot decide this and trying one first is what showed why: two identical calls agree *exactly* (0.0), but scoring a sequence alone versus in a batch of two *copies* of itself — equal lengths, no padding at all — already moves a token's logprob by 0.1032. The drift is a shape-dependent bf16 reduction, and any absolute tolerance rejects it whatever padding does. Padding is therefore measured against that control: padding alone to length 64 gives 0.1023 and to 256 gives 0.0469. It does not exceed the no-padding control and it does not grow with the amount of padding, which is what a recurrence consuming pad tokens would do.

The arms, and what each one actually selected. All four read the same 59-problem pool file and differ only in the rule (Table 47). T weights each task by Ornith's kernel $D(\hat{p}) = \exp(-(\hat{p} - p^\star)^2/2\sigma^2)$ with $p^\star = 0.2$, $\sigma = 0.15$, read on the *selecting* block. C1 is uniform-random resampled to T's realised *fresh*-block difficulty histogram, which is pre-registration matching rule 2 and is also why the control is not vacuous here: it reaches the same true difficulty by a different rule, so it agrees with T on $\hat{p}_B$ and disagrees on $\hat{p}_A$. C2 keeps $0.1 \le \hat{p} \le 0.9$, no target and no kernel. C3 selects *exactly* as T — the same seed, the same draws — and replaces the reward with an independent fair coin, so that the only difference between T and C3 is what the optimiser is told.

And under the other convention the comparison does not exist. With truncated samples dropped as unknown, T's effective pool is **2.7** problems and C1's is the same 2.7; the two arms would then be the same three problems seen four hundred times each, and any difference between them would be memorisation. That is the sharpest statement of the scarcity this paper has: at a strong model's operating point, the difficulty gate applied to a curated frontier-mathematics pool has essentially nothing to select.

A first attempt collapsed, and the control that identified why was C3. The arms were first run with a *token-level* loss normaliser (DAPO's, dividing by the batch's total completion tokens) at $\eta = 2 \times 10^{-4}$. Three of the four arms destroyed themselves within a dozen steps (Table 48): rollout accuracy went to zero and stayed there. **The random-reward arm did not.** C3 sees the same tasks, the same group structure, the same optimiser and the same normaliser, and differs only in that its reward is a fair coin — and it was still generating and still solving at the last step.

The mechanism, and it is arithmetic rather than a tuning accident. Under *truncated = wrong*, a rollout that hits the cap carries a negative advantage and 8192 tokens, while a correct one carries a positive advantage and a median of about 1,200. A token-level normaliser weights

Table 48: The aborted first attempt, token-level normalisation at $\eta = 2 \times 10^{-4}$ (`runs/*_lr2e4_tokennorm_collapsed`). "first"/"last" are means over the first and last three steps of each arm. The three arms with a real reward collapse; the one with a random reward does not, which is what locates the cause.

	rollout accuracy		truncation		generated tokens/step	
arm	first	last	first	last	first	last
T	0.243	**0.014**	0.702	0.271	343k	202k
C1	0.250	**0.063**	0.674	0.062	349k	112k
C2	0.743	**0.007**	0.243	0.222	238k	185k
C3 (random reward)	0.174	0.292	0.764	0.604	353k	320k

each rollout by its length, so the negative mass outweighs the positive by roughly seven to one, *systematically*, on every group. The policy is pushed away from long continuations until it stops producing them, and on a benchmark where the model is right 94.8% of the time *when it finishes*, that is fatal. Under a random reward the same length weighting has no systematic direction — long rollouts draw positive advantage as often as negative — so C3 drifts and does not collapse. This is the same defect Sec. 3.51 measured as "21.1% of A0's negative-advantage gradient mass pushes away from rollouts on their way to a correct answer", at the operating point where truncation is 0.39 rather than a fifth of that, and it is therefore the dominant term rather than a correction.

The arms were restarted with the vanilla GRPO normaliser — average within a rollout, then across rollouts, so every rollout counts once whatever its length — at $\eta = 10^{-4}$, and a guard was added that halts an arm when its mean generated tokens per step falls below a quarter of its opening window, so a collapse is dated and cheap rather than a flat tail nobody reads. **We report the collapsed run rather than discarding it**: it is the sharpest evidence in this paper that a loss-normalisation choice, not a reward or a selector, can be the whole difference between a run that learns and a run that dies, and it was identified by a control included for an unrelated reason.

The held-out outcome, and a correction to how we first scored it. Every checkpoint was scored on the *report* half — 337 problems no arm trained on — at the training operating point (`reasoning_effort=low`, temperature 1.0, top-p 0.95, cap 8192), eight samples per item, adapters served by one process in one session so the pairing is exact. Intervals are on the paired per-item DIFFERENCE, never per arm.

We first scored these files with a convention that was ours rather than this paper's, and it inflated every treatment effect. It is withdrawn here. Our analysis forced any generation that hit the token cap to zero. The grader this paper uses does not: it takes the last *balanced* `\boxed{}`, so a generation cut off after committing an answer is still scored on that answer — a rule Sec. 3.29 adopted after measuring that discarding those flips 0 items for the base model and 21 of 23 for the two most degraded checkpoints, i.e. that it penalises precisely the checkpoints under argument.

The two readings differ by the share of generations cut off after committing, and that share is a property of the arm: 0.055 for the base model, which truncates 41% of the time, against 0.000 for C1, which truncates 0.1% of the time. **So the discarded convention charges the most to whichever arm truncates most, and the base model truncates most of all.** It moved T's endpoint gain from +3.1 to +7.9 points and C2's from +8.0 to +13.2, in each case by penalising the baseline rather than by crediting the arm. Table 49 is on the paper's own grader.

Table 49: Held-out report half, avg@8, 337 paired items, runs `eval_final_*`, `eval_matched_*` and `eval_extra_*`, scored with the same grader as every other number in this paper. "matched" is each arm's last checkpoint at or below 6.95M generated tokens — the largest budget every arm reached, set by C1's halt; "final" is each arm's last checkpoint. "trunc" is the truncation rate at evaluation. C1's two columns are one measurement, not two: it halted *at* the common budget, so its final adapter is its matched checkpoint and the file is byte-identical (md5 `7a1dcf26...`).

	matched ($\leq$ 6.95M tokens)			final			
arm	avg@8	Δ base	95% CI	avg@8	Δ base	95% CI	trunc
base	0.6205	—	—	0.6205	—	—	0.410
T	0.7226	**+0.102**	[+0.075, +0.129]	0.6513	+0.031	[+0.011, +0.050]	0.216
C1	0.5593	**−0.061**	[−0.084, −0.038]	0.5593	−0.061	[−0.084, −0.038]	0.001
C2	0.6558	+0.035	[+0.021, +0.050]	0.7007	**+0.080**	[+0.053, +0.108]	0.039
C3	0.6295	+0.009	[−0.001, +0.019]	0.5909	−0.030	[−0.043, −0.017]	0.405

C3: the reward was independent, and the arm did not gain. Two questions, both answerable from the artifacts. *Was the reward actually random?* The trainer computes the reward as `float(rng.random() < 0.5)` for C3 and `float(correct)` otherwise, so the C3 expression never reads correctness; the measurable consequence is that C3's distribution of rewarded-members-per-group must centre on 4 rather than on 8× its accuracy. It does. Over 234 groups C3's mean rewarded count is **4.21** while 8× its measured accuracy is 2.56, and its histogram fits a fair coin ($\chi^2 = 14.8$ on 8 d.o.f.) far better than it fits its own accuracy ($\chi^2 = 656$). The same statistic on T, whose reward *is* correctness, gives a mean rewarded count of 6.612 against 8× accuracy of 6.612 — equal to three decimals. The reward was independent of correctness.

Did C3 nonetheless move the benchmark? +0.9 points at the matched checkpoint, with a 95% interval [−0.1, +1.9] that crosses zero, and −3.0 points [−4.3, −1.7] at its own endpoint, where it is significantly *below* the untrained base. An arm that is a point above baseline at one checkpoint and three points below at another, with the largest token budget of the four (12.3M against T's 10.6M), is oscillating around zero rather than gaining. **The +3.0-point C3 lift in our first analysis was an artefact of the withdrawn convention**, which credited C3 for truncating as often as the base model while charging the base model for it.

Charging C3 as a floor anyway — subtracting its gain wherever that gain is positive — every treatment effect survives: T +0.093 net at the matched budget (6.6 standard errors above the floor) and +0.031 at the endpoint (3.1), C2 +0.026 (3.6) and +0.080 (5.7). **The pre-stated failure condition does not obtain.** We would have reported it if it had: it is stated in `PREREG_gate_outcome.md` and in the operating-point finding that added C3 to the control set, precisely because spurious rewards are known to lift Qwen-family maths scores and this base is in that family.

The two evaluations disagree because both arms' held-out curves are non-monotone, and they cross. This is not two instruments measuring different things: it is one instrument applied to different checkpoints of the same arms. Neither is contaminated by the selection that produced it — the report half was never trained on and was never used to choose a checkpoint — but the arms do not improve monotonically, so *which* checkpoint is scored moves the answer by more than the difference being tested.

The pre-registration nominates the **matched** evaluation: rule 3 says to match total token budget, not step count, because the arms differ in generation cost. On that reading T leads C2 by 6.7 points.

Table 50: The same arm at several checkpoints, same grader, same 337 items. T loses 11.8 points between step 50 and step 61; C2 dips and recovers. T is ahead of C2 at every budget we measured except the last.

arm	step	generated tokens	avg@8	Δ base
T	30	6,487,508	0.7226	+0.102
T	50	8,958,062	**0.7697**	**+0.149**
T	61	10,618,909	0.6513	+0.031
C2	20	3,998,567	0.6792	+0.059
C2	40	6,810,872	0.6558	+0.035
C2	60	10,555,169	0.7007	+0.080

But the budget it is matched *at* was not pre-registered — it is 6.95M because that is where C1's length guard halted, a threshold we chose, and the guard misfired (below). At the endpoint, where T and C2 are matched to within 0.6% on tokens without any correction, C2 leads by 4.9. T is ahead at 6.5M and at 9.0M and behind at 10.6M, having lost 11.8 points in its last eleven steps.

So we do not fire stop rule 2, in either direction, and we do not nominate a winner between T and C2. An endpoint comparison between arms with crossing trajectories reports the wall clock; a best-checkpoint comparison reports whoever we looked at hardest — and we evaluated T's step 50 only *after* seeing that its step 30 beat its step 61, so quoting +0.149 as T's result would be selection on the outcome. What this run can support is that both arms move held-out capability by far more than the instrument resolves, and that deciding between them needs a stopping criterion fixed in advance on a held-out signal. Ours was a wall clock. That is a design defect in this run, not a finding about the gate.

What does survive. Stop rule 1 fires against the null: T beats C1 at every budget, +0.163 at the matched budget and +0.092 at the endpoint, and C1 is the one arm here that was actively *damaged* — −6.1 points below the untrained base, with truncation driven to 0.001 and accuracy among finished generations to 0.560. That is not a broken evaluation: it is the same terse-and-wrong endpoint its training curve reaches, and the identical numbers in both columns are one measurement copied, which we should have labelled and now have. A control matched to the gate's own realised fresh-block difficulty does not merely fail to keep up; on this pool it destroys capability while the gate builds it. This is the first learned selection component in this programme to beat its matched control rather than tie with it.

And the headroom claim is established, which was the point of the exercise. At the matched budget alone the arms span **16.3** points (0.7226 for T against 0.5593 for C1), against a 2.4-point instrument and against this programme's previous full-pipeline result of +0.89 points with a 95% interval of $[-0.56, +2.39]$. The change that bought it was not the hardware and not the selector: it was training on the mixed subset, where 0.80 of groups carry gradient against 0.13 on the pool these systems normally draw from.

A guard that fired on the wrong thing. C1 was halted at step 36 by a watchdog that stops an arm when its mean generated tokens per step falls below a quarter of its opening window. C1's fell to 0.24. But on this task *learning to terminate is* a fourfold reduction in tokens — it is the mechanism by which T and C2 gained — so the threshold cannot separate the improvement from the pathology, and it stopped the arm that most needed to be watched while letting T run into its own late collapse unremarked. The guard should key on held-out capability, or on tokens *and*

accuracy jointly, not on length alone. It is the reason the common budget sits at 6.95M rather than somewhere chosen in advance, and therefore a contributing cause of the unresolved comparison above.

3.55 Ornith's three-stage loop, executed: the optimiser it never had, and what the loop costs before it can learn anything

Everything this paper has measured of Ornith-1.5 so far is its *task-selection criterion*. The reproduction in `reimplementations/ornith_repro` has sixteen modules and **nothing in it trains**: there is no optimiser, no backward pass and no parameter update anywhere in the package, and `loop.apply_update` is a seam that records what would be updated. The central claim — that a model improves itself by co-evolving tasks, scaffolds and weights — had therefore never been executed. This section adds the missing half and runs it.

What was added, and what was reused unchanged. Group-relative optimisation on all three stages, each against its own published reward: $R_{\text{task}} = V(q,s)\,D(q,s,\{\tau\})\,N(q)$ with $D = \exp(-(\hat{p}-p^{\star})^2/2\sigma^2)$ and $p^{\star} = 0.2$; $R_{\text{harness}} = C(q,h)\,F(h,\{\tau\})\,H(h)$; and $R_{\text{rollout}} = h(q,\tau)$, the scaffold being the rollout's reward function. Every one of those, the reward propagation between them and both within-task comparison levels are computed by `nested.run_iteration_nested`, which is imported rather than rewritten; the one group it cannot form is the *task* group, because it scores a single task per call, so that is formed across the tasks of an iteration and is the only new GRPO group here. Three optimiser steps per iteration on one shared adapter, in the package's own `stage_order` (solver, harness, proposer): one policy improving itself, not three policies. The scaffold constructors and the proposal parser are the package's, called through a replay client so that batched asynchronous generation does not fork their logic.

Three configuration traps, all of which report as something else.

1. **Dynamic adapter loading forbids data parallelism.** sglang refuses `load_lora_adapter` with `dp_size must be 1 or dp attention must be enabled`, as an HTTP 500. Serving the arms in Sec. 3.54 never hit this because those adapters were loaded once at boot; a loop that rewrites its adapter every iteration cannot use data parallelism at all and must run tensor-parallel.

2. **The server's context length reports as the generation cap.** With a 6144-token context every proposer completion stopped between 5359 and 5520 tokens and the server returned `finish_reason: length` — indistinguishable from hitting a generation cap of 16384 that was nowhere near reached. Read at face value it says the proposer rambles to any budget; it actually says the window was 6144.

3. **The stage prompts are not chat-templated by the package.** Sent as raw strings to an instruction-tuned model the proposer does free-form continuation rather than instruction-following. Every stage here is rendered through the model's chat template at the same `reasoning_effort=low` the rest of this paper uses.

The loop's cost, measured before its budget was fixed, and it is the finding that constrains everything else. The proposer does not terminate. At a 16,384-token cap two thirds of attempts never close their reasoning block; raising the cap to 22,528, essentially the whole context window, leaves three quarters unterminated and the share that parses as a well-formed task

Table 51: What one turn of the published loop costs on `Qwen3.8-27B`, measured on the B200 before any budget was chosen. The benchmark row is the same model, same thinking budget, solving OlympiadBench (Table 45) and is the comparison.

stage	cap	median tokens	never terminated	usable output
proposer	16,384	at the cap	8/12	4/12 parse as a task
proposer	22,528	at the cap	9/12	3/12 parse as a task
scaffold	16,384	3,649	0/8	—
solver, *generated* task	16,384	**15,325**	7/16	12/16 boxed
solver, OlympiadBench (Sec. 3.54)	16,384	1,277	0.317	—

unchanged at a quarter to a third. The extra budget buys nothing: the completions simply fill whatever cap is set. In the first live iteration this cost 383,722 generated tokens across 24 attempts to obtain **two** usable tasks, a yield of 0.083.

And its output is expensive to use. A task the model proposes takes a median of **15,325** tokens to attempt — *twelve times* the 1,277-token median of an OlympiadBench problem for the same model at the same thinking budget — with 7 of 16 attempts never finishing at 16,384. At the 4096 cap the loop was first configured with, 15 of 16 rollouts are cut off, and the package's own guard G1 correctly refused the task for carrying 1 gradeable rollout of 18. That refusal is the loop working, not failing; but it means a faithful iteration cannot be run at a rollout budget an RL loop would normally use.

What the loop did, against conditions fixed before it ran. `PREREG_ornith_loop.md` states the primary quantity (informative rollout groups sustained across iterations), the secondary ones, and five degeneration conditions, and was written before any iteration statistic was read; its budget was amended once, before the run, because the measurement above showed the loop costs twelve times what the benchmark does. Ten iterations were run across three runs. No update ever fired between iterations that produced none, so the policy was unchanged and the iterations are independent draws.

One of ten iterations produced a gradient step at all. The other nine could not assemble a task group of two. Table 52 decomposes where they were lost.

Which degeneration, and the k-vectors say it unambiguously. "All groups degenerate" can mean the proposer made a task the solver always solves or one it never solves, and those are opposite failures with opposite repairs, so the refusal path records the success count of every scaffold group it refuses. Across the 30 scaffold groups of the refused tasks, **27** were unanimously *correct* ($k = 4$ of 4), 2 unanimously wrong, and 1 mixed. The two tasks that survived to be scored had $\hat{p} = 0.9167$.

The proposer, instructed in Ornith's own prompt to aim at a problem the model solves "about one time in five", produces tasks it solves essentially every time. That is degeneration condition 1 of the five declared in advance, and it is the one that matters because the difficulty kernel is built to punish it: at $\hat{p} = 0.9167$, $D = \exp(-(0.9167-0.2)^2/2\cdot 0.15^2) = 1.1\times 10^{-5}$, so $R_{\text{task}} = VDN$ is 1.07×10^{-5} and the task reward is numerically dead. The gate is doing its job; there is simply nothing in its support to select.

This is not an artefact of our group size. The package's own `predicted_binary_degeneracy` gives 0.706 degenerate groups at $\hat{p} = 0.9167$ with $G = 4$ and 0.499 at $G = 8$: a larger group would have halved the loss, not removed it, and the observed all-correct rate of 27/30 is higher than even

Table 52: Ten iterations of the three-stage loop, pooled (`runs/ornith_v2`, `ornith_run`, `ornith_smoke`). "never terminated" is a proposer attempt that filled the 16,384-token cap without closing its reasoning block. The refusals are the package's own guards firing, correctly, on tasks it had accepted as valid.

quantity	value
iterations run	10
iterations that formed a task group and trained	**1**
proposer attempts	168
never terminated	126 (0.750)
accepted as a well-formed task	27 (yield 0.161)
proposer tokens generated	2,442,356
per accepted task	90k
per task that survived to be scored	**407k**
accepted tasks refused: all rollout groups degenerate (guard G4)	20
accepted tasks refused: too few gradeable rollouts (guard G1)	1

the $G = 4$ prediction because many of these tasks are at $\hat{p} = 1$ exactly. At Ornith's published $p^\star = 0.2$ the same formula gives 0.411, so the target itself concedes a substantial degenerate fraction; the proposer is nowhere near it.

The one iteration that trained, reported in full because it is the only one. All three stages fired and all three took a real gradient step on one shared adapter: solver 8 rows / 111,999 tokens / grad-norm 0.0260; harness 6 rows / 61,668 / 0.0395; proposer 2 rows / 26,152 / 0.0146. Both nested comparison levels formed — 2 of 6 rollout groups informative, 2 of 2 scaffold groups informative, task group informative — the adapter's LoRA-B moved from exactly 0 to 16,156, and the served adapter was verified against a null adapter on the same server at 2.33× the null-routing numerical floor. The iteration took 868 seconds and generated 734,658 tokens. Novelty was healthy ($N = 0.966$) and validity perfect ($V = 1.0$); neither the novelty buffer nor the validity gate is what stopped this loop.

What this establishes, and what it does not. It establishes that the loop *runs*: the optimiser this reproduction never had is now attached to all three stages, the nested group structure is exercised rather than merely present, and every stage takes a measurable gradient step. It also establishes the process answer this run was for, and the answer is negative in a specific and diagnosable way. **On this base, the loop does not sustain a supply of informative groups because it cannot manufacture a task at the difficulty its own reward demands.** Nine iterations in ten produced no gradient at all, and 407k generated tokens were spent per task that reached scoring.

It does not establish that Ornith-1.5's method fails. Three limits bound the claim, and all three are ours. The proposer is a *prompted* stage of an untrained policy: the loop is supposed to train it, and it never got the chance because the task group needed two survivors to form. A group size of 4 costs signal that a group of 8 would partly recover. And ten iterations is a small number, forced by a per-iteration cost we measured rather than chose. What the run does show is that the first turn of the loop is the hard one, and that the published design has no mechanism to get through it: the difficulty reward can only rank tasks a proposer already produces, and on a strong base an untrained proposer produces tasks at $\hat{p} \approx 1$. That is the same shape as this paper's other three counts of training signal that does no work, arriving one level up — and it is an argument for seeding the

loop from measured difficulty, which is exactly what the mixed subset of Sec. 3.54 provides and what the arms there used.

3.56 OpenRSI's fix lands, and how we checked it: a patch can pass its own tests and still return the wrong number

Sections 3.45 and 3.47 left the only genuinely reproducible external system in this paper blocked at its reward stage, on a version skew between two of its authors' own repositories. We reported it as FrontisAI/OpenRSI issue 12. It was closed on 2026-09-04 with "we have pushed a new PR for this problem, please try it."

What they changed. The fix is commit `ce3ba53` ("Fix current NatureBench evaluation contract"), merged as `215bad9` from branch `fix/naturebench-eval-contract`; OpenRSI HEAD moves from `1f477c4` to `215bad9`. It touches five files and adds 585 lines, closing both halves of the skew we reported and also the third hazard, the one we declined to patch ourselves:

- **The control-token gate.** `base_task` now attaches `X-NatureBench-Control-Token` to exactly the four control endpoints (`register`, `start_timer`, `resume_timer`, `pause_timer`), reading the token from an `eval_control_token_file` config key or the `NATUREBENCH_EVAL_CONTROL_TOKEN_FILE` environment variable. The launcher starts the evaluator with `--control-token-file` pointed at a private file under the experiment output, so the token is never exposed to model-generated candidate code.
- **The `/evaluate` contract.** The client now generates its own opaque `eval_token`, persists it under the eval-service output directory, and sends it at both `register` and `evaluate`, rejecting a token that collides with the control token.
- **The directory-binding hazard.** Under the token contract each attempt's workspace becomes `<attempt>/workspace` and its output `<attempt>/workspace/output`; `_post_evaluate` re-registers before every evaluation with `out_dir` set to `<attempt>`, so the service's fixed `<out_dir>/workspace/output` resolves to the directory that attempt actually wrote. `force` is sent only on the first registration, so re-registration takes NatureBench's timer-preserving "re-register without force" branch.

Their own suite goes from 66 passed to 70 passed and 1 skipped, the four new tests asserting on the register/evaluate call sequence and the per-attempt output paths.

Why we did not stop at a green suite. This is worth stating carefully, because it generalises beyond OpenRSI and is not a criticism of them. The bug we reported was specifically one whose *wrong* outcome is a plausible number rather than an error: a client that evaluates a stale directory gets a score back, and the score is of the wrong directory. A test can only catch that if something in it depends on *which* directory was read. Their four new tests assert against a mocked `_post_json`, so they constrain the client's call sequence — exactly the thing that was wrong — but no real evaluator ever runs, and no assertion depends on the contents of any output directory. A patch of this shape can therefore pass its own tests and still return the wrong number. The check that closes the gap is cheap: drive the real client against the real service with two candidates whose outputs differ in quality, so that a wrong-directory bug shows up as *identical* scores.

That is what we ran, on task package `s42256-023-00611-x` against NatureBench HEAD `2bd59cf`, with attempt 1 predicting a constant and attempt 2 fitting a logistic regression. No upstream file was modified; the task class was imported from the upstream tree.

stage	observable	result
control endpoint	`POST /register`	`200` (was `404`)
timer	`POST /start_timer`	`200`, fired *once*
reward	`POST /evaluate`	`200` (was `400`)
re-binding	service-logged `out_dir`	`..._validation_1`, then `..._validation_2`
scored the right dir	`aggregate_improvement`	-0.835758 vs -0.588490
artifacts	`submissions.jsonl`	one per attempt root, `attempt`= 1, 2

The two attempts received different scores, ordered as the written predictions imply, with per-instance F_1 values recorded for all four instances. **The contract is repaired, and repaired correctly.** OpenRSI's documented NatureBench quickstart evaluates a task again.

One gap in the fix, reported for completeness. Only the quickstart launcher `run_naturebench_local.py` wires the token; no configuration under `tts_search/configs` sets `eval_control_token_file`, so the *formal* Lite-v2 path (`run_naturebench.sh`) still sends the pre-hardening contract unless the operator exports `NATUREBENCH_EVAL_CONTROL_TOKEN_FILE` by hand. Because `base_task` reads that variable, the formal path works with no patch, but an operator following `benchmarks/naturebench_lite_v2/RUNNING.md` verbatim would hit the same 404 we reported. This costs one line of documentation upstream.

3.57 The cheap benchmark cannot settle its own claim, and what our run is therefore for

We priced the reproduction from their configuration files rather than their prose before spending on it. The result changes what a run of their cheap benchmark can be presented as, so we state it before the numbers.

The headline finding of this section. NatureBench Lite-v2 is *ten tasks at one sample each*, and its published delta is Match-SOTA 50% → 70% — that is, 5/10 → 7/10, a two-task difference. Fisher's exact test gives $p = 0.65$. Worse, at $n = 10$ against a 5/10 baseline, **no outcome short of** 10/10 **reaches** $p < 0.05$: 8/10 gives $p = 0.35$, 9/10 gives $p = 0.14$, 10/10 gives $p = 0.033$. At its own sample size the published NatureBench Lite delta is not merely uncertain, it is *unfalsifiable*. **No run we perform on this benchmark can confirm or refute their claim, and we do not present our run as a reproduction of it.**

What such a run *can* establish is separate and, given the last two weeks, not small: that their released pipeline executes end to end on current hardware, at their own search budget, on their own published data, and produces ordered, directory-correct scores. A week ago their documented quickstart could not evaluate a single task (Sec. 3.47). Demonstrating that it now runs a ten-task benchmark to completion is a statement about the artifact, not about the effect, and that is how we report it.

Two corrections to how the target has been described, including by us. First, the 39.39% → 60.61% Medal Average figure is **MLE-Bench Lite**, not NatureBench. NatureBench Lite carries two separate published claims: Match-SOTA 50% → 70% with the framework fixed and the model swapped, and 20% → 50% with the model fixed and the harness swapped. Second, the MLE-Bench cost has been quoted too low. `openmle_evo.yaml` sets `time_budget: 43200` (12 h) but `model_plus_sandbox_time_budget: 64800` (18 h), and it is the latter that the harness

enforces. At 22 tasks × 3 runs that is **1,188 model-plus-sandbox hours per cell**, or 2,376 for a base/evolved pair. The familiar "∼1,584 sandbox-hours for the pair" is computed on the 12 h agent budget and is therefore an *underestimate of the enforced cost*, not an approximation of it. For contrast, `naturebench_scm_lite_v2.yaml` sets both budgets to 14,400 s over ten tasks at one sample: 40 model-plus-sandbox hours per cell, 80 for the pair.

The headline benchmark is the one with statistical content, and it is blocked on a credential rather than on compute. 39.39%, 60.61% and 71.21% of 66 are exactly 26, 40 and 47, confirming 22 tasks × 3 runs as the denominator; 26/66 → 40/66 gives $p = 0.023$. That is a real separation, though 66 trials are 3 seeds over 22 tasks and clustering by task pulls the effective n back toward 22. Running it requires a Kaggle API token to fetch the competition data. **The obstacle is a credential, not GPU time.** The two routes therefore trade off against precisely the thing we care about: the affordable benchmark cannot resolve its own claim, and the benchmark that can resolve its claim needs 1,188 hours per cell and an account.

The model-swap delta is not measurable for an arbitrary base model. A reader will ask why we do not repeat their headline comparison on a current strong base such as `Qwen3.8-27B`. We cannot, and the reason is structural rather than budgetary. Their delta is *base model → Frontis-MA1*, with the harness held fixed, and Frontis-MA1 is a trained artifact: there is no evolved counterpart of `Qwen3.8-27B`, and producing one would mean running their SFT and RL pipeline, which is a different and much larger undertaking than an evaluation. The comparison that *is* defined for an arbitrary base is their other published claim, the harness swap at 20% → 50% with the model fixed, and in their release it is a clean configuration ablation: `search= airaevo_naturebench` against `airaevo_naturebench_original_tts`, which differ only in the `experience` block (prompt-score sanitization, parent selection and prompt memory on or off) and in `fresh_draft_prob` (0.2 against 0.0). That is the comparison we run on a current base. The same power arithmetic applies to it: 2/10 → 5/10 gives $p = 0.35$, so it too is a statement about whether the mechanism runs and moves scores in the expected direction, not a significance test.

It is, however, the *better-powered* of their two NatureBench claims, and that is a second reason to prefer it beyond mere measurability. Because its baseline is 2/10 rather than 5/10, the separation needed for significance is reachable in principle: 8/10 gives $p = 0.023$ and 7/10 gives $p = 0.070$, where the model-swap comparison against a 5/10 baseline requires a perfect 10/10. So a harness-swap run that landed well above its published 5/10 could actually say something, whereas no attainable model-swap result on this benchmark could. We do not expect that outcome and do not design around it; we record it so that the run is interpreted against a threshold fixed in advance rather than one chosen after the scores are in.

3.58 Correction: the vendored framework is Meta's aira-dojo, but it does not give us a shared measurement surface with Meta

The `aira-evo` tree inside OpenRSI is Meta's `aira-dojo` (arXiv:2507.02554, "AI Research Agents for Machine Learning: Search, Exploration, and Generalization in MLE-bench"), CC BY-NC 4.0. Pulling OpenRSI therefore delivers part of a second published system whether or not one intends it. **We previously recorded that this gives us a shared measurement surface with Meta's research-agent line via MLE-Bench. That is wrong, and we correct it here.** Four facts bound what is actually there.

- **It is a fork, not a copy.** Against upstream HEAD `c795d86` (2025-09-26, 15 commits), the

vendored tree differs in **29** files and adds a further **38** paths, including the entire NatureBench adapter and the experience-memory operators (`draft_experience`, `improve_experience`, `crossover_experience`, `rich_memory_summary`) that carry OpenMLE-Evo's contribution. OpenRSI's numbers are results *of their fork*, and any comparison to "aira-dojo" has to say which one.

- **The tests are FrontisAI's, not Meta's.** Upstream `aira-dojo` ships *no* test suite — zero `test_*.py` files. The 53 tests we previously credited to "the vendored aira-evo" are six files FrontisAI added. This is a point in FrontisAI's favour and a correction to our earlier note.
- **Upstream implements exactly one task,** `mlebench`, and running it needs a Kaggle API token, so it is blocked by the same credential as Sec. 3.57.
- **Meta's current research-agent work does not report on MLE-Bench at all.** "AI Research Preference Models" (arXiv:2608.13940, v2) is evaluated on **AIRS-Bench** — twenty tasks built from SOTA papers, with its own normalised score — not on MLE-bench, which it cites once as related prior work. The MLE-bench overlap is with the older Toledo et al. paper only. Nor is there a code release for it: the `aira-dojo` repository's last push predates the paper by four months, and its tree contains no file matching `rpm`, `preference`, `pilot` or `airs`.

So the shared, runnable surface is with OpenRSI alone. Meta's newer work is prior art to credit, not a system we can run or compare numbers against.

What we must credit anyway. Their inference-only variant builds a preference model from a frozen pretrained LLM that reasons over candidate plans, code and previously executed solutions with their validation scores — which is the frozen-LLM router over retrieved prior executions that this project had written down, and it is now published first. Their agentic variant spends a small pilot experiment before committing expensive compute, which is the same logic as our verifier checking a key before spending 48 rollouts on it. What remains distinct is granularity: they choose among candidate research plans, we act on training data and parameter updates. It is worth recording that their own uncertainty is thin. The reported lift is 0.684 → 0.711 and 0.729, but no numeric confidence interval is printed for any of the three anywhere in the text, and the only quantified statement about the differences is an rliable probability of improvement of 0.5923 and 0.5913 with 95% CI lower bounds of 0.5066 and 0.5018 — and the agentic variant, which has the *higher* mean, has the *lower* probability of improvement.

3.59 Rung 0 passes on both cells, and the ten-task run executes

With the contract repaired we re-ran the stop-rule gate from Sec. 3.45: one task, one candidate, one reward, artifacts written and read back. Machine: one `8×B200` node; GPUs 0–1 served the evolved model, 2–3 the base, via SGLang 0.5.19 on `torch 2.13.0+cu130`; all eight cards were verified usable by allocation and a bf16 matmul rather than by `device_count`, which has lied to us before. Command: their `scripts/run_naturebench_local.py --smoke` on `s42256-023-00611-x`, upstream unmodified at `215bad9`.

stage	Sec. 3.47	now
task construction	OK	OK
eval service + control token	—	OK, token auto-created `0600`
prompt / scaffold	OK	OK
generation	OK	OK, real `run.py` (LightGBM)
candidate execution	—	OK, ran 52.3 s, exit 1
evaluation / reward	**404 on /register**	**200; a real score**
artifacts	—	journal, summary.csv, submissions.jsonl

We ran the gate on both cells of their published comparison: the evolved `Frontis-MA1-35B` and its base `Qwen/Qwen3.6-35B-A3B`, served side by side from separate GPU pairs and verified to be distinct weights (identical prompts at temperature 0 produce different completions). Both complete the contract and write a reward:

cell	`aggregate_improvement`	`success_rate`	instances scored
evolved, `Frontis-MA1-35B`	−1.000	0.0	0/4 (crashed)
base, `Qwen3.6-35B-A3B`	+0.0026	1.0	4/4

This is not evidence against their claim and we do not report it as any. It is one task, one candidate, two steps and a thirty-minute budget — a hundredth of the search their protocol grants — and the evolved model's loss is a single feature-alignment bug (`LGBM_GetLastError`: trained on 23 features, predicted on 22) that more search would likely have debugged away, since debugging is what the loop does. LightGBM was installed and executed, so this is a modelling error by the agent and not a missing dependency. We record it because it is the honest content of the artifact, and because it instantiates Sec. 3.57: at one sample per task a single crash moves a ten-task Match-SOTA score by a full ten points, which is half their published effect.

The ten-task run, and the deviations it carries. Both cells then ran the ten-task Lite-v2 manifest concurrently on the same node. Their search protocol is reproduced exactly — `step_limit`= 160, 80 generations × 2 individuals, 14,400 s budget, one sample per task — because `naturebench_local_quick` already carries the same search parameters as `naturebench_scm_lite_v2`; only concurrency and the sandbox differ. Three deviations must be declared, all recorded in `RUN_NOTES.txt` beside the outputs.

1. **Local process execution, not their Docker sandbox.** This is forced, not chosen: their released `Dockerfile.base` pins CUDA 11.8 and `torch 2.6.0+cu118`, which cannot target `sm_100`, so on a B200 their own container cannot use the GPUs at all. Their documentation calls the local path "dependency isolation rather than a security sandbox" and unsuitable for formal submission.
2. **Candidate threads capped** at 4 for `OMP/MKL/OPENBLAS/NUMEXPR/LOKY`. See below.
3. **Sandbox concurrency** 5 **per cell** (10 across both cells on one host), against their 10 per cell on a cluster where each task gets its own container.

We installed every package their prompt advertises to the candidate (16/16 verified importable) so that candidate failures are attributable to the agent rather than to our environment.

An operational note worth a paragraph, because it cost us an hour. Running ten candidates concurrently on one 224-core host without resource containment is catastrophic, and the failure is silent until the machine stops making progress. Candidates call scikit-learn with `n_jobs=-1`, which spawns one `loky` worker per core, and each worker then opens an OpenMP pool sized to the whole machine. We measured **26,483 Python threads across 228 orphaned worker processes and a load average above 21,000** — roughly 100× oversubscription. Because their protocol fixes a *wall-clock* budget, that contention is not merely slow: it is spent directly out of the search budget, so both cells silently get less search than the protocol grants. Capping the five thread-pool variables (verified in a live candidate's `/proc/<pid>/environ`, not assumed) brought the same workload to 1,706 threads and a load average of 31. Anyone reproducing a per-task-container benchmark on a single fat node should set these before the first run.

3.60 A silently truncated download produced a valid-looking benchmark score, and the check that caught it was not the check we had written

We discarded the first ten-task run and restarted it. The reason is worth a subsection, because the failure produced no error at any stage, the run completed tasks normally, and the resulting numbers were plausible.

What the metric actually is. First a correction that matters for reading any of these numbers. NatureBench's `_compute_improvements` defines the per-instance score as $g = \text{direction} \times (\text{agent} - \text{sota})/|\text{sota}|$, aggregated as a mean over primary instances with a fixed -1.0 penalty for an instance that produced no finite score. So g is measured *relative to the paper's SOTA*, not on a baseline-to-SOTA scale: $g = 0$ means the agent matched SOTA and $g < 0$ means it fell short. Their scorer defines **Match-SOTA as the fraction of tasks with** $g \geq 0$ and **Surpass-SOTA as the fraction with** $g > 0.1$, each over the track's official task count, so an unscored task counts against the denominator. We score with their `compute_scores.calculate_metrics` rather than reimplementing it, supplying case metadata restricted to the ten Lite-v2 tasks because their command line validates only the official 90- and 25-task tracks.

The failure. Two of the ten task packages were incomplete on disk, and nothing said so. `s43588-025-00842-5` was missing nine files including `train_data.csv`, `x_test.csv` and `zeolite_priors_0.pkl`; every one of its eleven attempts in both cells died on `FileNotFoundError` and scored the -1.0 floor. `s41467-025-65557-7` was worse and quieter: all 399 files were present, but 188 of them were *truncated*, at roughly 29 kB against an upstream 400 kB. Candidates therefore ran to completion on about seven per cent of the data, wrote a well-formed `predictions.csv`, and were scored against the paper's SOTA as though the run were real. **That task carried the evolved cell's best score of** $g = 0.2145$ **and its only clear Surpass-SOTA.** The interim standing — evolved Match-SOTA 50% and Surpass-SOTA 20% against base 40% and 0% — was thus contaminated in the one cell and the one direction that would have flattered the published claim.

The check we had written could not have caught it. Our download verification asserted that each package had a `metadata.json`, an `evaluation/evaluator.py`, a `ground_truth/` directory and a `problem/data` directory. All four were true of both broken packages. The verification tested *existence*, and the failure was *completeness*; a guard of that shape passes exactly the case it is meant to refuse. What found it was comparing every local file against the upstream tree by name *and by size*, which reported 188 mismatches on one task and nine missing files on the other. That audit is now a hard preflight in the run script: it runs before the model is contacted and aborts the run on

any discrepancy, and we tested that it fires by truncating a known-good file to 1 kB and confirming the launcher refused to start.

Why it happened, and the general lesson. The downloads reported success. `hf download` with an `--include` pattern exits zero whether it fetched everything, something or nothing, and with `HF_HUB_ENABLE_HF_TRANSFER` set we also had a transfer path that left short files without raising. Refetching without `hf_transfer` and with fewer workers produced byte-exact copies, and all ten packages now match upstream exactly.

The lesson generalises past this benchmark and is the same one this paper keeps rediscovering in other forms. **A dataset that is partially present is more dangerous than one that is absent**, because absence raises an error and partial presence produces a number. Any reproduction that downloads its own evaluation data should verify it against the publisher's manifest by size or checksum before the first GPU-second is spent, and should treat an exit code of zero from a downloader as carrying no information about completeness.

3.61 Seeding the loop from measured difficulty: the cold start was not the problem

Sec. 3.55 located the loop's failure at its first turn — a prompted proposer emits problems the base already solves, so the difficulty gate has nothing in its support — and scoped that as a statement about the cold start rather than about the method. The obvious repair is to supply the first turns from measurement instead of from a prompt, and this section runs it. `PREREG_seeded_loop.md` fixes the claim, the primary quantity, the degeneration conditions and, explicitly, the reading of the outcome in which seeding changes nothing; it was written before either arm started.

Two arms, both run fresh at matched settings, concurrently so that a truncated run leaves them with the same number of iterations. **COLD** takes every task from the proposer, as published. **SEED** draws iterations 0–3 from the *mixed subset* — the benchmark problems this base sometimes solves and sometimes does not, established from its own rollouts — and hands over to the proposer at iteration 4. Same three rewards, same nested structure, same group sizes, same caps. The control is re-run rather than compared against Sec. 3.55's numbers, whose group size and iteration count would otherwise be confounded with the seed.

The answer is that seeding does not rescue the loop. SEED formed a task group in 0 of 6 iterations — including 0 of its 4 seeded ones — against COLD's 1 of 4 (Table 53). Both numbers are near zero and n is small, so the honest statement is not that seeding hurt but that it did not help: the pre-registered outcome in which seeding changes nothing is the one that occurred.

Seeding did move the per-task number, just not far enough. Three seeded tasks survived scoring across four seeded iterations against COLD's two across four, but never two in the same iteration, and a task group needs two. The refusals say why: across SEED's 15 refused tasks (45 scaffold groups) 27 groups were unanimously correct and 18 unanimously wrong, with *none* mixed. In SEED's first seeded iteration the two refused tasks carried the k-vectors $[0, 0, 4]$ and $[4, 4, 4]$ at $G = 4$ — tasks drawn from the mixed subset *because* the base does not always solve them, solved under the loop every time or not at all.

Nor did seeding change the proposer when it took over. On proposer-driven iterations SEED's proposer never terminated on 18 of 24 attempts (0.750) at a validity yield of 0.250, against COLD's 47 of 60 (0.783) at 0.167 — indistinguishable at this n, and neither near usable. Four

Table 53: COLD against SEED, run concurrently at matched settings (3 tasks per iteration, 3 scaffolds, $G = 4$ rollouts, caps 16384, $\eta = 10^{-4}$), runs `runs/COLD` and `runs/SEED`. A task group needs two survivors, so an iteration with one scored task still trains nothing.

arm	iteration	task source	accepted	scored	formed a group
COLD	0	proposer	3	0	no
COLD	1	proposer	2	0	no
COLD	2	proposer	3	2	**yes**
COLD	3	proposer	2	0	no
SEED	0	seed	3	1	no
SEED	1	seed	3	0	no
SEED	2	seed	3	1	no
SEED	3	seed	3	1	no
SEED	4	proposer	3	0	no
SEED	5	proposer	3	0	no

Table 54: A generated scaffold does not preserve the difficulty a seed was selected for. Base model, eight tasks drawn from the mixed subset, three scaffolds each, four rollouts per scaffold, run `out/SCAFFOLD_DRIFT.txt`. "pool $\hat{p}$" is the difficulty measured by this paper from bare prompting over sixteen samples; "scaffolded" is the rate under the loop's own harness stage.

task	pool $\hat{p}$	scaffolded	k per scaffold (of 4)
26	0.812	0.583	$[4, 1, 2]$
374	0.938	1.000	$[4, 4, 4]$
184	0.812	1.000	$[4, 4, 4]$
529	0.938	1.000	$[4, 4, 4]$
315	0.875	1.000	$[4, 4, 4]$
209	0.562	1.000	$[4, 4, 4]$
487	0.938	1.000	$[4, 4, 4]$
132	0.312	0.500	$[0, 4, 2]$

seeded iterations of solver and harness updates, carried to the proposer through the shared adapter, did not visibly change what it writes.

Why: the scaffold stage destroys the difficulty the seed was chosen for. The loop never solves a task bare. The harness writes a scaffold and the solver is conditioned on it, so a seed's measured $\hat{p}$ is a property of a different prompt from the one the rollouts see. Measured directly on the base model, on eight mixed-subset tasks, three generated scaffolds each, four rollouts per scaffold (Table 54):

Mean absolute drift from the measured value is 0.169 and the signed mean is $+$**0.112**: scaffolds make tasks *easier*. Six of the eight went to $\hat{p} = 1.000$ exactly. 21 of 24 scaffold groups were unanimous, and **six of the eight tasks would be refused by the package's own guard G4** — the same refusal, at the same rate, that the proposed tasks earn. A seed selected for difficulty arrives at the solver as a task that is no longer difficult.

And the seed pool was never at the target difficulty to begin with. "Mixed" means not unanimous over sixteen samples, which on a strong base mostly means solved thirteen or fourteen times out of sixteen. The mixed subset's own $\hat{p}$ has mean 0.671 and **median** 0.812; 57.6% of it

sits above 0.75 and only **13.6**% lies in the gate's effective support $[0.10, 0.35]$. So the two effects compound: a seed pool whose median task is already solved four times in five, pushed a further $+0.112$ by the scaffold. The mixed subset is the right pool for *extracting gradient* — that is what Sec. 3.54 used it for, and there it moved held-out accuracy by up to fourteen points — but it is not a supply of tasks at $p^\star = 0.2$, and this experiment is what distinguishes those two uses.

What this settles, and it is the stronger negative. The pre-registration named this outcome in advance: if the seeded arm's behaviour matches the cold-started one, then the cold start was not the problem and the fault lies in the proposer stage or the design, not in how the loop is started. That is what happened, and the drift measurement says which: **it is not the proposer either.** Even when the task is supplied from measurement, the loop's own harness stage moves it off the difficulty its own task reward demands, and the rollout-level guard then refuses it. The difficulty gate is being asked to select on a statistic that a later stage of the same loop overwrites.

One structural observation, which is the sharpest thing in this section. The refused seed with $k = [0, 0, 4]$ is not an uninformative task. Two of its scaffolds never solved it and one always did: at the *scaffold* comparison level — the level the nested structure exists to create, and the one R_{harness} is defined on — that task carries a large, clean signal. It is discarded because guard G4 is applied to the *rollout* groups alone. The nesting creates a second comparison level and the batch guard then refuses batches without consulting it, so the loop throws away exactly the tasks whose signal lives at the level the nesting was introduced for. That is a defect of the guard's placement, ours to fix rather than Ornith's, and we record it as a correction to Sec. 3.55's reading of the same refusals: some of those twenty refusals will have been scaffold-informative too.

The cost, which replicates. The one iteration that trained generated 703,186 tokens for two tasks that reached scoring, i.e. **352k per scored task**, against the 407k measured in Sec. 3.55 on a different configuration. COLD spent 938,924 tokens on the proposer alone across four iterations. Whatever else is true of this loop, a task that survives to be scored costs on the order of 4×10^5 generated tokens, and three quarters of the proposer's spend buys generations that never terminate.

Scope. Eight tasks in the drift measurement and four to six iterations per arm; the drift direction is large and consistent but its magnitude is not tightly estimated. The seeded arm gives the proposer no gradient of its own during seeding — it did not write those tasks — so it benefits only through the shared adapter, which is stated in the pre-registration as the mechanism under test and leaves a behaviour-cloning variant untested. And the whole section speaks to `Qwen3.8-27B` at `reasoning_effort=low` on OlympiadBench-derived tasks; a weaker base would put more of any pool inside the gate's support.

3.62 Measuring difficulty after the scaffold: the fix our own criticism implies, tested

Sec. 3.61 left a criticism with no remedy attached: the difficulty gate selects on a success rate measured by prompting the model with the task alone, while the loop's solver sees a scaffolded prompt, so the gate selects on a number describing a prompt nobody runs. The implied repair is a change in *where* the statistic is computed, not in the reward, the kernel or the target. `PREREG_postscaffold.md` fixes the primary quantity, the cost accounting and the outcome in which the fix changes nothing; it was written before either arm started.

Two arms, run fresh and concurrently at matched settings, 8 iterations each, six candidate tasks per iteration, three selected, three scaffolds per task, $G = 4$. **PRE** selects the top

Table 55: PRE against POST, runs `runs/PRE` and `runs/POST`, eight iterations each. Tokens are generated tokens, split by what generated them. POST's probe is the block it must pay for in order to measure difficulty under the scaffold; it cannot be reused as the scored block without scoring the selection on its own draw.

arm	task groups formed	informative rollout groups	tokens	per scored task	probe share
PRE	4/8	0.486	1,521,051	**109**k	—
POST	4/8	0.611	4,074,971	**370**k	56%

three candidates by $D(\hat{p})$ with $\hat{p}$ measured bare, as published. **POST** scaffolds *every* candidate, rolls an independent probe block, and selects by $D(\hat{p})$ with $\hat{p}$ measured under those scaffolds; the block the loop then scores is fresh, so a task is never scored on the draw that selected it. Neither arm uses the proposer — both draw from the mixed subset — which isolates where difficulty is measured from whether a prompted proposer can hit a target at all.

The primary outcome: it changes nothing. PRE formed a task group in 4 **of** 8 **iterations; POST in** 4 **of** 8 (Table 55). The informative rollout-group fraction within those iterations was 0.486 for PRE and 0.611 for POST, on four iterations each. This is the outcome the pre-registration named in advance, and it is reported as a result rather than as an inconclusive run.

The cost, which is half the result. POST costs **2.68**× PRE per iteration and **370**k generated tokens per task that reaches scoring against PRE's **109**k — and 56% of POST's entire spend is the probe block, which the loop consumes for nothing but the estimate. At equal tokens PRE would run 2.7× as many iterations. **So the fix costs roughly** 3.4× **per scored task and buys no additional task groups.** It is worth noting what the 109k figure means on its own: the ∼350k-per-scored-task figure that replicated twice in Secs. 3.55–3.61 was dominated by the *proposer*, and removing that stage alone drops it by a factor of three. POST lands back at 370k for an entirely different reason.

Why it changes nothing, and it corrects our own earlier claim. POST's selection works on the statistic it measures: it picked tasks whose *scaffolded* $\hat{p}$ averaged 0.301, close to the published $p^\star = 0.2$, where PRE's picks averaged 0.510 on the bare statistic. But on the fresh scored block the realised rates were 0.696 for POST and 0.736 for PRE — indistinguishable, and both far above the target. **A four-sample scaffolded estimate does not predict the next four samples on the same task-scaffold pair.**

The reason is that the scaffold does not shift difficulty, it *re-randomises* it, and this corrects the reading in Sec. 3.61. Over 48 candidates measured here the signed drift from bare to scaffolded $\hat{p}$ is $\mathbf{-0.099}$ with mean absolute drift **0.362**; the two smaller measurements that preceded it gave +0.112 and +0.039. **The signed drift is not stable across measurements and is not distinguishable from zero; the absolute drift is large and consistent.** Our earlier statement that "scaffolds make tasks easier" does not survive its own replication and is withdrawn. What survives, and is stronger, is that a scaffold changes a task's difficulty by roughly 0.2–0.36 in an unpredictable direction. Selecting on the extreme of an estimate that noisy is selecting on noise, and the regression from 0.301 to 0.696 is exactly the winner's curse this paper has documented at two other levels.

The guard, decided deliberately. Guard G4 is applied to the rollout groups only, which discards tasks whose scaffolds disagree — signal at the level the nesting exists to create. **We left it as the package has it**, so the primary numbers describe their design, and recorded every refusal's k-vector so the alternative could be priced. It is cheap: of PRE's ten refused tasks one carried scaffold-level signal and of POST's thirteen none did, so a scaffold-aware guard would have added one task group to PRE (5/8) and none to POST (4/8). On this configuration the guard's placement is not what is limiting the loop, which is worth knowing before anyone spends effort changing it.

A defect in our own harness, found in the logs and matched across arms. Every iteration reports `harness rows=0, grad_norm=0`: the update loop iterates over `("solver", "harness")` but only ever populates rows for the solver, so **the harness stage never took a gradient step in either arm**. Both arms carry the defect identically, so the PRE-versus-POST comparison stands, but this run trained the solver alone and is not a test of the full loop. The smoke check missed it because it asserted that *some* stage stepped with a non-zero gradient norm rather than that *each* did — the third instance in this programme of a guard that passes exactly the condition it was written to refuse, and this one is ours.

What this settles. Both pre-registered outcomes were on the table and the null one occurred: moving the difficulty estimate onto the prompt the solver actually receives does not make the loop form more task groups, at 3.4× the price per scored task. **The problem is therefore deeper than where difficulty is measured.** It is not that the gate reads the wrong prompt; it is that on this base a task's difficulty under a scaffold is not a stable quantity to read at all — it varies by 0.36 across scaffolds and is not reproduced by a fresh sample of the same pair. A selection rule needs a statistic that predicts the block it is selecting for, and at these group sizes no cheap estimate does. That is a constraint on the whole family of difficulty-targeted self-improvement loops, not on Ornith's kernel.

Scope. Eight iterations per arm and four groups formed per arm, so the equality of the two rates is 4/8 against 4/8 and not a tight bound; a probe larger than four rollouts would estimate difficulty better and cost more still, and that trade was not swept. Truncation was re-measured on this configuration rather than inherited and is not a confound: 0.073 for PRE's scored blocks, 0.087 for POST's and 0.049 for its probe, against the 0.317 that made the default operating point unusable.

3.63 A task-source layer, and why its headline number could not be measured with a string-matching verifier

Three sources — self-generated, retrieved, and teacher-distilled — behind one interface, so that everything downstream is identical and a difference between sources is a difference in what survives the same path: contamination check, then *one* shared novelty buffer, then the verifier. The layer exists to test a sharp prediction. Self-generated keys are refuted 6.56% of the time among decided cases against 2.55% for professionally curated ones, and retrieved problems carry keys written by someone other than the model being trained, so they should not show that failure at all.

What was built, and what an audit was allowed to attack. 694 lines across six modules, with eleven tests. Passing tests are not evidence, so each of the four properties an auditor would go for was *broken on purpose* in a copy and the suite required to notice; all five mutants were caught (`code/mutate_tasksource.py`):

Table 56: Twenty candidates per source through the identical path, run `out/tasksource/table.json`, 162 s. "contam" is rejection for overlap with the held-out report half at a deliberately conservative threshold; "dupe" is rejection by the shared novelty buffer. **The refuted column is not interpretable at this configuration** and the next paragraph is why.

source	candidates	contam	dupe	novelty rej.	decided	refuted	accepted
generated	20	7	3	0.150	10	2	8
retrieved	20	3	0	0.000	5	2	15
distilled	20	2	9	0.450	9	2	7

- the shared novelty buffer replaced by one buffer per source — caught: a retrieved near-duplicate of a generated task must lose to it, and the rejection must name the source that already held it;
- the contamination check degraded to exact matching — caught: a reworded held-out problem must still be rejected, which is why similarity is the *maximum* of token Jaccard and character 5-gram cosine rather than either alone;
- provenance dropped on the way to the artifact — caught: source, origin and licence must survive every transformation, and a record refuses to exist without them;
- a source returning zero tasks recorded as a success — caught, in both shapes (an explicit failure and an empty success), because a silent zero is the failure mode this programme has hit five times;
- a refuted key accepted anyway — caught: it must enter neither the buffer nor the artifacts.

The prediction could not be tested, and finding that out is the result. Retrieved tasks came back with the *highest* refuted-key rate of the three, 2 of 5 decided. Curated keys from MATH, AIME and AMC are not wrong two times in five, so that number measures the verifier. Retrieval costs no generation, so the curated corpora were used as a calibration set — their keys are right by assumption, so every refutation on them is a false one. On 60 curated problems the verifier decided 38 and **refuted 17: a false refutation rate of 0.447, Wilson** 95% $[0.301, 0.603]$, against the 2.55% published for the same class of problem. The instrument is off by a factor of about eighteen and cannot resolve a 6.56%-versus-2.55% difference.

Reading the refutations individually — the same method that established the generated-key figure was real — **eight of eight were formatting, not arithmetic**: key `29.0` against computed `29`; `101.68` against `\$101.68`; `$C=12/5$` against `\frac{12}{5}`; `$-\ln|\cos x|+C$` against `\ln|\sec x|+C`; `$\log(e^{\ln(100)})=2$`, where the corpus stores the whole equation and the model answers `2`. Not one was a wrong key. Fourteen of the seventeen came from `college_math`, whose answers carry prefixes and LaTeX wrappers.

And the confound is not neutral across sources, which is what makes it fatal rather than merely noisy. A generated key is written by the same model that later solves the problem, so it is already in that model's answer format; a retrieved key is written in its corpus's format. A string-normalising comparison therefore refutes retrieved keys more often *for reasons that have nothing to do with correctness*. Running this table as it stands would have appeared to refute the prediction — retrieved worse than generated — by exactly the mechanism that guarantees the

wrong answer. Source comparison of key correctness needs a format-invariant decision (symbolic equivalence, or the execution backend the package's cascade puts first), not the single consensus backend used here.

What the run does establish. Self-generated tasks are a contamination risk, and a larger one than retrieval. Seven of twenty generated candidates were rejected for overlapping the held-out report half, against three of twenty retrieved and two of twenty distilled. At a threshold chosen so that a false reject costs one candidate while a false accept costs the evaluation, that is an upper bound rather than a count of memorised problems — but the ordering is the opposite of the expectation that retrieval is where contamination comes from. A model asked to invent problems reproduces its own evaluation set.

The shared buffer does real work, and it is asymmetric. The distilled source lost nine of twenty candidates as near-duplicates and the retrieved source lost none. With three buffers under one name both would have been admitted; the collisions are recorded with the source that already held the near-duplicate, so a cross-source duplicate is visible rather than merely subtracted.

Cost, and the teacher backend. The teacher is provider-agnostic and was tested against the local `Qwen3.8-27B`; **no paid API call was made**, and the hosted backend refuses to make one unless an explicit environment authorisation is set, while remaining priceable without contacting anyone. From this run's measured counts — 721 output tokens per call and 11.4 calls per accepted task, that ratio being the parse and novelty yield rather than an assumption — a hosted teacher at illustrative \$3/\$15 per million tokens would cost **\$13 for 100 accepted tasks and \$130 for 1000**. That is the estimate to take to the PI; nothing has been spent. The collusion risk is recorded in provenance rather than assumed away: the teacher here is the *same model* as the solver, which is the hazard already logged for the judges, and a different-family teacher is exactly what the paid option would buy.

Scope. Twenty candidates per source and five to ten decided cases each: the table is a working demonstration of the layer, not a measurement of the sources, and it is reported as such. The calibration that matters — 17 false refutations in 38 decided curated cases — is the number this section stands on, and it says the next thing to build is a format-invariant key check, not more tasks.

3.64 Corrections to Sec. 3.63: both of its numbers were instruments, and both are withdrawn

Sec. 3.63 reported two findings and flagged one of them as uninterpretable. On re-measurement *both* turn out to be properties of the measuring apparatus rather than of the task sources, and both are withdrawn here. The methodological point that section made — that a format-sensitive comparison confounds a source comparison — survives, and is strengthened by what follows.

Correction 1: the 0.447 false-refutation rate was the comparator, not the verifier. That calibration compared the verifier's computed answer with the asserted key using `answers_match`, the string normalisation the live grader uses, rather than `symbolic.symbolic_compare`, which strips LaTeX, splits answers as unordered sets, uses exact rationals and *refuses* inexact decimals. Re-running the identical 60 curated problems and the identical 300 solutions through both comparators, so that only the comparator differs (Table 57):

So the instrument is not eighteen times off; the earlier run used the weaker comparator. The paired breakdown is unambiguous: 19 verified under both, 0 refuted under both,

Table 57: The same 60 curated problems and the same solutions under two comparators, run `out/tasksource/recalibration.json`. Every one of the sixteen string-refutations dissolves: three are symbolically equal, thirteen are unparseable and become UNVERIFIABLE, none survives. The exact comparator never contradicts a string verification.

comparator	decided	refuted	false-refutation rate (Wilson 95%)
`answers_match` (string)	37	16	0.432 $[0.287, 0.591]$
`symbolic_compare` (exact)	22	**0**	**0.000** $[0.000, 0.149]$

3 string-refuted but symbolically *equal*, 13 string-refuted but symbolically *indeterminate*, and 0 string-verified but symbolically different. The last count matters as much as the first: the exact comparator introduces no refutations of its own.

Two caveats that must travel with the 0.000. It rests on 22 decided cases, so its upper bound is 0.149 — consistent with the published 2.55% but not on its own establishing it. And soundness is bought with coverage: decided cases fall from 37 of 60 to 22 of 60, because declining to parse is how the comparator avoids refuting a key it cannot read. That is the right trade for a key check and the wrong one for a training signal, which is why the loop grades with `answers_match` and the verifier should not.

And this makes Sec. 3.63's methodological warning sharper rather than weaker. A generated key is written in the solver's own answer format; a retrieved key is written in its corpus's. Under the string comparator that asymmetry produced a 0.432 refutation rate on keys that are not wrong, and it would have fallen hardest on the retrieved source — so the source comparison would have appeared to refute the prediction by exactly the mechanism that guarantees a wrong answer. The fix is not more tasks; it is that any comparison of key correctness *across* sources must use a format-invariant decision. With `symbolic_compare` in place, that comparison can now be run.

Correction 2: the contamination ordering does not survive a negative control, and is withdrawn. Sec. 3.63 reported that 7 of 20 self-generated candidates overlapped the held-out half against 3 of 20 retrieved, and called it the strongest result in the section. Widened to 40 model-written and 200 retrieved candidates the ordering persists but shrinks — rejection at the 0.45 cut of 0.200 (generated), 0.200 (distilled) and 0.075 (retrieved). The top-scoring "overlaps", however, are plainly unrelated problems: a derivative of $-3/(4x^2 - 2x + 1)$ scored 0.625 against an arctangent identity.

So the measure was given the control it should have had first: 400 curated problems from *other* corpora scored against the same held-out half, where a high score is a false positive by construction (Table 58).

The 0.45 cut has a 9.5% false-positive rate overall and 14.2% on statements under sixty characters, falling to zero above 160. Model-written statements are short and formulaic; retrieved ones are longer and more varied. The observed rejection rates — 0.200 for model-written against a floor of 0.13–0.14 in their length band, and 0.075 for retrieved against a floor of 0.095 — are therefore consistent with the measure's own length-dependent false-positive rate and establish nothing about memorisation. **The claim that self-generation is a larger contamination risk than retrieval is withdrawn.**

What survives is the filter's designed purpose. It was built so that a false reject costs one candidate while a false accept costs the held-out set, and at 9.5% false positives it is doing exactly that job. Its rejection rate is an upper bound and must never be quoted as a contamination estimate. Testing the question properly needs a measure whose false-positive rate does not depend

Table 58: False-positive rate of the contamination cut on unrelated curated problems, by statement length, run `out/tasksource/similarity_control.json`. The measure is max(token Jaccard, character 5-gram cosine); short statements have few content words, so both components inflate.

statement length (chars)	n	mean similarity	false positive @0.45	@0.55
0–60	113	0.313	0.142	0.044
60–100	124	0.340	0.129	0.065
100–160	89	0.259	0.067	0.011
160–260	53	0.215	0.000	0.000
> 260	21	0.163	0.000	0.000
all	400	0.275	**0.095**	0.035

on statement length — exact long-span n-gram overlap, the standard decontamination method, rather than a similarity score.

Two instruments, two withdrawals, one pattern. Both corrections have the same shape: a number was produced by an apparatus that had never been calibrated against a case whose answer was known in advance. The comparator was never run on keys known to be right; the similarity measure was never run on problems known to be unrelated. Each control cost minutes — the similarity control needed no GPU at all — and each overturned the finding it tested. The audit discipline that caught five deliberately planted defects in the layer's code did not extend to the measurements the layer produced, and that is the gap worth naming.

3.65 A permanent task store, the source comparison run properly, and the finding that none of the three sources supplies gradient

With the comparator fixed (Sec. 3.64) the source comparison can finally be run, and its output is worth keeping rather than printing. This section reports both: the store the layer now writes to, and the comparison read back out of it.

The store. One record per task, append-only JSONL, on `/extra/zhanglab1/pengchx3/selfevo/taskstore` and mirrored to `gs://selfevo/taskstore`; 52 records, 87,668 bytes, identical by MD5 in all three places. The schema version appears both in the filename and in every row, and the reader is shipped beside the data so the store can be read without the training box. The record *refuses to be constructed* without: a content hash as well as an id; per-source provenance (teacher model and version for distilled, corpus, row and licence for retrieved, prompt version and exemplars for generated); a verdict with its backend, its comparator, and — for a refutation — the witness that refuted it; a success rate together with the prompt it was measured under; three separate deduplication readings, each with its threshold and the false-positive floor for that statement's length band; and cost split into what it took to produce the task and what it took to score it. Sixteen tests cover these refusals and eleven deliberately planted defects are all caught, including the one that matters most: a record written under an older schema is refused loudly rather than reread with today's meanings.

The store's own discipline caught two more instruments, and one of them changed an answer. Both have the shape named at the end of Sec. 3.64.

Table 59: The three sources, read back out of the store (`taskstore/tasks.v3.jsonl`, 52 records). Keys are decided by `SolverConsensus` at k=5 against `symbolic_compare`; the solver never sees the key it is checking. Coverage travels with every rate: a source whose keys are mostly unparseable is not a source whose keys are mostly right. $\hat{p}$ is the success rate over the same five samples, measured under the bare solution prompt and recorded with it.

		verdict				tokens / accepted			
source	n	ver.	ref.	unver.	coverage	produce	verify	refuted rate (Wilson)	mean $\hat{p}$
generated	16	14	1	1	0.938	1859	**1685**	0.067 [0.012, 0.298]	0.938
retrieved	24	9	0	15	**0.375**	**0**	7563	0.000 [0.000, 0.299]	0.817
distilled	12	10	0	2	0.833	2580	4265	0.000 [0.000, 0.278]	1.000

Schema v1: a boolean named for the floor and computed from the threshold. The dedup reading carried one flag, `above_floor`, whose value was in fact score $\geq$ threshold. The two answer different questions — whether an overlap triggers the reference set's action, and whether it is distinguishable from the chance overlap between unrelated problems of that length — so an overlap that was real but not actionable was recorded as if it were nothing. v2 records both.

Schema v2: a per-record cost field holding a batch mean. `cost.score_tokens` was the mean over every task scored in the run, written into each row as though it were that row's own cost. It read as a measurement and was not one, and the cost ranking built on it compared three identical numbers, 4218, and returned list order. v3 attributes verification cost to the task's own samples. **The fix inverted the result**: verification costs 1685 tokens per accepted task for generated, 4265 for distilled and 7563 for retrieved, so retrieval is simultaneously the only source that is free to produce and, by a factor of 4.5, the dearest to trust. Under the collapsed column that inversion was invisible.

Neither old file was deleted. Both were retired beside the store, and the current reader was run against both to show it refuses them by version rather than reinterpreting them — the guard demonstrated firing, not asserted.

The comparison (Table 59), with the coverage caveat carried. The refuted rates are 0.067, 0.000 and 0.000, and their Wilson intervals all reach past 0.27: on nine to fifteen decided cases per source *no ordering of key reliability is established*, and the routing plan says so rather than presenting the sort order as a result. What the table does separate cleanly is coverage. The verifier decides 15 of 16 generated keys and 10 of 12 distilled ones, but only 9 of 24 retrieved — because retrieved keys are written in their corpus's format and the exact comparator declines to parse what it cannot read. That is the soundness-for-coverage trade of Sec. 3.64 landing unevenly across sources, and it is a property of the checker, not of the keys.

The single refutation is a real one and its witness makes it checkable, which is the entire reason the field is mandatory. The generator wrote *"the determinant of the 2×2 matrix with first row* $[2, 3]$ *and second row* $[5, 7]$*"* and asserted -11; the witness is -1, and $2{\cdot}7 - 3{\cdot}5 = -1$. **A model writing its own tasks writes some of its own keys wrong, and the layer catches it and preserves the evidence.**

The result that matters is not in the key column. Three of 52 accepted tasks fall in the mixed band, and all three came from retrieval (Table 60). **Both model-written sources produced zero gradient-bearing tasks between them**: every one of the twelve distilled tasks

Table 60: Difficulty of the accepted tasks, over the same five samples used for verification. The mixed band $0 < \hat{p} < 1$ is where a group-relative objective has any gradient at all; outside it every rollout in the group agrees and the advantage is exactly zero.

source	n	$\hat{p} = 0$	$0 < \hat{p} < 1$	$\hat{p} = 1$	gradient-bearing share
generated	16	1	0	15	0.000
retrieved	24	3	3	18	0.125
distilled	12	0	0	12	0.000
all	52	4	**3**	45	**0.058**

was solved five times out of five, as were fifteen of sixteen generated ones. This is the unanimous-group problem arriving one stage earlier than we have been looking for it. We have been treating task supply and difficulty targeting as separable — build the sources, then aim them — and at this model's competence they are not separable, because a model asked to write a problem writes one it can already solve. A source layer that triples throughput and leaves 94% of its output outside the informative band has not moved the constraint.

Two caveats bound this. First, $\hat{p}$ is measured on five samples, so $\hat{p} = 1$ is not $p = 1$: a task with a true p of 0.85 reads 5/5 44% of the time. The mixed count is therefore a *lower* bound, and the honest statement is that the mixed share is under 12% even for the best source, not that it is exactly 0.058. Second, difficulty here is measured under the bare prompt and recorded with it, for the reason established in Sec. 3.62: the same task measured after the harness stage is a different number, so the two must never be pooled.

Three reference sets, three rates, and one of them is not a rejection. Candidates are checked against the held-out half (contamination), the running novelty buffer (repetition) and the training pool (redundancy). Over 45 candidates per source, rejections were 7/14/0 (generated), 3/5/0 (retrieved) and 8/20/0 (distilled) for contamination, repetition and redundancy respectively. Two of these three readings must be quoted against their floors, and only one may be quoted at all:

- **Contamination, at** 0.45**, rejects.** The costs are asymmetric — a false reject loses one candidate, a false accept loses the held-out set — so rejecting on suspicion is correct even at a 9.5% false-positive rate. But the floors (0.092 and 0.081 in the two shortest bands, 0 above 160 characters) are why the contamination *ordering* was withdrawn in Sec. 3.64, and the same caution applies to the numbers above.

- **Repetition, at** 0.60 **against the buffer, rejects — and its counts are an upper bound.** The buffer's false-positive floor is 0.292 for statements under 60 characters and 0.140 for 60–100. Model-written statements fall *entirely* in those two bands (median length 69 and 85 characters, against 104 for retrieved), so the higher repetition rejection rates for generated (0.31) and distilled (0.44) sit exactly where the measure is least trustworthy. **We do not report that model-written sources repeat themselves more than retrieval**; that would be the withdrawn contamination claim wearing a different hat.

- **Redundancy, at** 0.60 **against the training pool, only flags.** Here the floor is 0.000 in every length band — unrelated curated problems never reach 0.60 against the pool — so this reading is clean, and it is null: 0 of 52 stored tasks were flagged. The decision not to reject on it is deliberate and survives the clean floor. Redundancy is symmetric in cost, and a redundant task is not an invalid task: it is one the pool already covers, which is a scheduling

fact rather than a validity fact. It is recorded per task so a scheduler can act on it, and it never silently removes a candidate.

Routing, not a winner. The plan the layer emits ranks the sources per need and refuses to rank on a field nobody measured, marking a tie as a tie. Retrieval is the only source free to produce, the only one whose writer does not collude with the solver, and the only one that produced a gradient-bearing task — and it is the dearest to verify, cannot be aimed at a difficulty, is licence-constrained, and is a fixed pool. Generation is unbounded, aimable, and the cheapest to verify, and it is the only source observed to write a wrong key. Distillation as run here is *not* evidence about distillation: the teacher was the same checkpoint as the solver, which is why it produced the twelve tasks the solver solved twelve times out of twelve, and why its provenance records the collusion explicitly. A different-family teacher is the experiment this row does not perform.

Cost. The store was produced for 320,375 tokens on the B200 — 60,704 to produce 52 tasks and 259,671 to score them, four fifths of the bill spent on trusting the tasks rather than on making them. **No money was spent**; every arm, teacher included, ran on the local checkpoint, so the dollar axis is unmeasured here rather than zero, and the routing plan declines to rank on it.

3.66 OpenRSI phase 1, executed on verified data: the pipeline runs, and the published model-swap delta does not reappear

The ten-task Lite-v2 comparison completed on byte-exact task packages (Sec. 3.60), both cells, one sample per task, seed 42. Scores are computed by NatureBench's own `compute_scores.calculate_metrics`: Match-SOTA is the fraction of tasks with $g \geq 0$, Surpass-SOTA the fraction with $g > 0.1$, both over the ten-task count.

cell	Match-SOTA	Surpass-SOTA	search nodes	wall clock
evolved, `Frontis-MA1-35B`	60% (6/10)	40% (4/10)	563	4 h 23 m
base, `Qwen3.6-35B-A3B`	70% (7/10)	20% (2/10)	574	4 h 18 m
their published pair	70% vs 50%	—	—	—

What this says, stated at the altitude the sample size allows. The pipeline executes: twenty task-cells ran to their four-hour budget, 1,137 search nodes were created and evaluated, every task returned a directory-correct score, and both cells produced artifacts that read back. That is the claim this run was for, and it is a real change from Sec. 3.47, where the same released quickstart could not evaluate a single task.

On the published effect, the run recovers nothing. Their reported pair is base 50% to evolved 70%; we measure base 70% and evolved 60%, so the Match-SOTA difference has the *opposite sign* to the claim. Surpass-SOTA moves the other way, 40% against 20%, in the claimed direction. **None of these differences is distinguishable from none.** Fisher's exact test gives $p = 1.00$ for 6/10 against 7/10 and $p = 0.63$ for 4/10 against 2/10; our base is not distinguishable from their base either (7/10 against 5/10, $p = 0.65$). This is exactly the regime Sec. 3.57 predicted in advance, and it is why we neither claim a failure to reproduce nor treat the sign reversal as a finding. A single task moves Match-SOTA by ten points, and the difference we observe is one task.

Three checks that the comparison is not confounded. *Throughput matched.* An early reading suggested the evolved cell was searching far faster, which under a wall-clock budget would

confound quality with speed. It was a startup transient: the completed run gives 563 nodes against 574, a 2% difference, so both cells received effectively the same amount of search.

The task both cells fail is genuinely hard. `s43588-025-00842-5` scored the −1.0 floor in both cells. With the data now complete, its failures are agent coding errors — `KeyError: 'SMILES'`, `np.mode`, `os.dirname`, InChIKey strings cast to float — and not missing files. It is a task neither model can do, which is a legitimate benchmark outcome and costs both cells equally.

The one asymmetry runs against the base cell, not for it. The base cell lost `s42256-023-00627-3` after two attempts when the harness assembled a prompt of **555,006 tokens** and the server refused it; the evolved cell hit this zero times. That is an OpenMLE-Evo robustness issue rather than a limit of our serving configuration — no context window would accommodate it, including the model's own 262,144 — and it is worth reporting upstream: their prompt assembly has no cap, so one pathological debug cycle can terminate a task. Because it truncated the *base* cell's search on a task the base cell nonetheless matched, the base 70% should be read as a floor.

What we are running to bound the noise. A single seed cannot say whether a one-task difference is signal or scatter, so a second independent replicate at seed 43 is running for both cells. Its purpose is not to improve the estimate of their effect — at $n = 10$ nothing can — but to measure the run-to-run spread against which the one-task gap should be read. We report the pair, not the better of the two.

3.67 The three-stage loop, actually run: a repaired harness stage, and two caps that were reading the budget rather than the method

Sec. 3.62 compared difficulty measured before and after the scaffold stage. That comparison stands — both arms carried the defect identically — but the run behind it trained one stage of three. This section repairs the defect, rebuilds the check that hid it, re-measures every cap, and runs the loop.

The defect. The update loop iterated over both stages and built rows only for the solver:

```
for stage in ("solver", "harness"):
  rows = []
  if stage == "solver":
    ... append one row per non-degenerate rollout...
```

so the harness optimiser was handed an empty list every iteration, took the `grad_norm 0.0` branch, and logged cleanly. The scaffold generations it needed were already being fetched and were discarded at the call site with _. The repair builds harness rows from the scaffold advantages the nested scorer already returns, indexed by the same list that subsets the scaffolds, so a scaffold's advantage cannot be attributed to another's tokens.

The check that hid it, rebuilt. The old check asked whether *any* stage had stepped. It is now per stage and on three observables: rows populated, gradient norm non-zero, and the adapter fingerprint — summed |LoRA-B| over the whole adapter, exactly zero at initialisation — actually moved. A step function returning normally is not evidence; parameters moving is. The check is itself tested against a stage deliberately handed no data, which it must fail, because a guard never shown to fire is the failure mode this project has now logged seven times (Table 61).

Table 61: Each stage exercised separately on real generations in that stage's own prompt shape, run `out/smoke_stages.json`. Advantages are assigned rather than earned: this tests whether data of each shape reaches the optimiser, which is a different question from whether the loop produces such data, and conflating the two is how the defect survived.

stage	rows	grad norm	fingerprint Δ	verdict
proposer	2	0.0188	8964.7	PASS
harness	2	0.0205	3978.6	PASS
solver	2	0.0061	3271.0	PASS
starved control (must fail)	0	0.0000	0.0	**FAIL, as required**

Table 62: Every cap re-measured on this configuration at a ceiling chosen not to bind, run `out/budget_3stage.json` and `out/budget_proposer_40k.json`. Two of the three caps in use were wrong, in opposite directions.

stage	ceiling	p50	p90	max	at ceiling	usable yield
proposer	16384	16384	16384	16384	0.938	0.063 parsed
proposer	**40960**	30903	37666	40960	0.083	**0.917** parsed
harness	16384	2164	5837	9625	0.000	—
solver	8192	4390	8192	8192	0.125	0.906 answered

Re-measuring the budget overturned the configuration for the fifth time, twice in one pass. Both errors were caps inherited rather than measured, and they pushed in opposite directions (Table 62).

The proposer does terminate. At the inherited 16384 cap it ran to the ceiling in 15 of 16 attempts and 1 proposal in 16 parsed, which this project had recorded as "the proposer never terminates 75% of the time". At 40960 it terminates in 11 of 12 and **11** parse — a yield of 0.917 against 0.063, from nothing but the cap. The median proposal costs 30,903 tokens. **The proposer's non-termination was an artefact of a budget roughly half of what it needs**, and the earlier finding is withdrawn.

The rollout cap was below the median rollout. The loop had been running at 4096 while the median scored rollout is 4390 tokens, so more than half of every group was truncated and the degeneracy statistics were partly reading the cap. At the measured p90 of 8192, 90.6% of rollouts answer.

And the harness was paying the proposer's bill. The scaffold stage shared `gen_cap`: it was allocated 16384 tokens for a stage whose longest generation in 16 was 9625 and whose p90 is 5837. It now has its own cap at 6144.

The loop ran, and at iteration 8 all three stages took a gradient step in the same iteration. That had not happened before: iteration 8 trained the proposer (3 rows, Δ1570), the harness (9 rows, Δ1712) and the solver (30 rows, Δ1997), with both nested comparison levels formed and the across-task group informative (Table 63). **The repair is also confirmed in the file the defect lived in**: a `pre`-mode iteration of `postscaffold_train.py` now reports harness rows 6, gradient norm 0.077, fingerprint Δ2086, where every iteration of the run behind Sec. 3.62 reported 0, 0.0 and no movement.

Table 63: The three-stage run, `runs/THREE/iters.jsonl`. A stage counts as having trained only if it received rows, produced a non-zero gradient norm, AND moved the adapter fingerprint; Δ is that movement. Iterations 0–6 draw tasks from the mixed subset, so the proposer correctly receives nothing and is marked n/a; 7–8 run the proposer live. Blank rows are iterations where fewer than two tasks survived scoring, so no group formed.

			proposer		harness		solver		informative	
it	source	scored	rows	grad	rows	grad	rows	grad	roll.	scaf.
0	seed	1				*no task group*				
1	seed	2	n/a	n/a	6	0.053	24	0.020	0.667	1.000
2	seed	2	n/a	n/a	6	0.045	30	0.015	0.833	1.000
3	seed	1				*no task group*				
4	seed	1				*no task group*				
5	seed	1				*no task group*				
6	seed	2	n/a	n/a	6	0.069	12	0.042	0.333	1.000
7	proposer	1				*no task group*				
8	proposer	3	**3**	**0.054**	9	0.059	30	0.018	0.556	1.000

Does the loop sustain informative groups? Four of nine iterations formed a task group at all, and *every* one that did was informative at all three levels: the across-task group was non-degenerate every time, the scaffold group was informative in 1.000 of cases in all four, and the rollout groups in 0.333–0.833. So the failure is not that groups go flat as the loop runs; it is that a group forms less than half the time. Of 22 refused tasks the k-vectors are almost always extreme: 12 were solved by every rollout under every scaffold, 9 by none, and exactly 1 was anything in between. **The scaffold stage polarises difficulty.** These tasks were drawn from the mixed subset — selected precisely for $0 < p < 1$ measured on the bare model — and conditioning the solver on a harness scaffold moves most of them to one extreme or the other. That is the post-scaffold drift of Sec. 3.62 seen from the other side, and it is the reason the loop's yield is what it is.

Does harness training change anything? We cannot say, and four updates is why. The harness stage now trains, and its gradient norm is consistently *larger* than the solver's — mean 0.056 against 0.023 over the same four iterations, a factor of 2.4. That is a fact about the signal, not about the outcome. Four updates cannot support a claim that the stage changed the loop's behaviour, and the mean $\hat{p}$ over those four iterations (0.833, 0.870, 0.500, 0.368, falling toward the $p^\star = 0.2$ the reward targets) is exactly the kind of four-point trend this project has already been wrong about twice. Powering it properly is arithmetic: at the observed 0.44 groups per iteration, twenty harness updates per arm needs about 45 iterations, and a harness-on/harness-off comparison at matched budget is therefore roughly 18M tokens and ten hours on this box. That is the experiment this run does not perform, and it is pre-registered here rather than approximated.

Guard G4 keeps its rollout-group scope, and now for a measured reason. G4 refuses a task whose every rollout group is unanimous, which also removes it from the harness and proposer stages before scaffold advantages are computed. Widening it was priced at one extra task group in eight. On every refusal the scaffold rewards were recomputed with the package's own judges and reward function, changing nothing about the run: **of 22 refused tasks, 0 had an informative scaffold group.** Their scaffold rewards were not merely equal but exactly $[0, 0, 0]$ in all 22 cases — because reward fidelity is computed against the task's own rollouts, so when every rollout agrees,

every scaffold scores the same. The guard discards nothing that the harness stage could have used, and the decision is to keep it exactly as the package has it.

Cost, and the one thing that does not fit. The run scored 14 tasks for 1,788,713 tokens: 127,765 **tokens per scored task**, 106,387 in the seeded iterations and 181,208 with the proposer live. The proposer is now 16.9% of the bill rather than the dominant term it was when it was being retried into a truncating cap. Rollouts are 75%.

The run ended at iteration 9 on a fatal out-of-memory: a single proposer row of ∼41,000 tokens needs 38.2 GiB for its float32 logits in one backward, and the device has 178 GiB with the model and optimiser already resident. This is a real constraint on the method rather than an accident of this box: the proposer stage's median generation is 30,903 tokens, and a sequence that long cannot be backpropagated whole at this vocabulary size. Training it needs the loss accumulated over sub-sequences, which nothing in the reproduction does today. **So the honest statement about the three-stage loop is that it runs, that all three stages train, and that the proposer stage trains only when its proposal happens to be short.**

And the rebuilt check had the same disease one level up. Made per stage, it was still evaluated on the *last* iteration only. The validation run's final iteration scored one task and formed no group, so every per-stage assertion was skipped and the check returned without testing anything — a vacuous pass, from a check written specifically to stop vacuous passes. Its `return 1` also sat outside the `if probs` test, so it could never report success either. The check is now asserted over the whole run — at least one task group must have formed, and each stage must be shown to have trained in at least one iteration that formed one — and it was lifted out of `main` so it can be tested. Against records mutated from a real run it passes the repaired run, passes a sound run whose last iteration happens to be empty, and fails all three of: a harness stage starved every iteration, a harness step that moved no parameter, and a run in which no group ever formed (`code/test_smoke_verdict.py`). **That is the eighth guard in this project to pass the condition it was written to refuse, and the fifth found by whoever wrote it.** The repaired `pre` run trains the harness stage in three consecutive grouped iterations — rows $6, 6, 9$, gradient norms $0.077, 0.052, 0.035$, parameters moving every time — against 0, 0.0 and no movement in every iteration of the run behind Sec. 3.62.

3.68 Phase 2, declared before its numbers exist: what a null there could and could not mean

The harness comparison of Sec. 3.57 — OpenMLE-Evo's `airaevo_naturebench` against the original AIRA-Evo TTS baseline `airaevo_naturebench_original_tts`, differing only in the `experience` block and `fresh_draft_prob`, with the model held fixed at `Qwen3.8-27B` — is running as this is written. We state its principal limitation now, while the numbers do not yet exist, because stating it afterwards would be worthless.

The limitation. Under their wall-clock budget this pair completes roughly **ten to fifteen search nodes per task**, against the 160 their `step_limit` permits and the ≈ 56 per task that phase 1 achieved. The treatment arm's entire mechanism is experience memory and parent selection operating over a search population: it retrieves up to three related cards, three ancestors and three siblings, and ranks parents within an island sized at 160. At a dozen nodes per task that population barely exists, and the machinery under test has almost nothing to rank or retrieve from.

Therefore a null result in phase 2 cannot be read as evidence against the mechanism. It would be substantially explained by the mechanism having nothing to operate on, and that

explanation is available independently of whether the mechanism works at the depth its authors measured it at. We pre-register that reading here. What phase 2 can still support is the weaker and honest claim that the two harnesses were exercised end to end on identical models, budgets and data, and what their scores were at this depth. The comparison remains internally fair — same model, same budget, same task packages, same containment, differing only in the config block under test.

Update, still before the numbers: it is worse than shallow. Mid-run measurement sharpens this. The treatment arm has produced 29 nodes of which 8 are non-buggy; the control arm 64 nodes of which 4 are non-buggy. So the experience memory is not merely retrieving from a small population, it is retrieving from a population of roughly *eight successful nodes spread across ten tasks*. The node-count gap between the arms is also not what it appears: the control is not out-searching the treatment, it is failing faster (94% buggy at 0.3 s mean execution against 72% at 1.2 s), so its extra nodes are cheap dead ends rather than extra search. Node count is therefore not a budget measure here and we do not use it as one. The one signal visible so far runs in the mechanism's favour — the treatment's candidates run about four times as often — but on eight successful nodes that is an observation, not a result.

We considered and rejected buying depth. The available levers are raising `individuals_per_generation` or switching to the asynchronous steady-state profile, and both alter the search protocol that phase 2 exists to measure.

3.69 A wall-clock evaluation budget silently advantages terse models over reasoning models

This is a property of OpenRSI's evaluation design rather than of any model, it is measured rather than argued, and it affects anyone who runs their benchmark.

Their protocol fixes `model_plus_sandbox_time_budget` at 14,400 seconds per task. It does not fix the number of search steps, so the amount of search a model receives is whatever its own verbosity permits inside four hours. Measured on the same node, same harness, same ten tasks:

model	architecture	search nodes per wall-clock hour
`Frontis-MA1-35B`	MoE, 8 of 256 experts active	128
`Qwen3.6-35B-A3B`	MoE, 8 of 256 experts active	133
`Qwen3.8-27B`	dense, 64 layers	**26**

The dense model receives roughly **a fifth** of the search. It is not serving speed: its server sustained *higher* aggregate token throughput than either mixture-model server (694–704 tokens/s at batch 12–13, against 323–479 at batch 2–5).

Correction: we cannot attribute this gap to verbosity, and an earlier draft of this section did. We first wrote that the difference is tokens emitted per search node, a verbose model spending its budget on output. Checking that claim rather than restating it shows it is not established, because the two families are not in the same regime. A search node costs generation *plus* candidate execution, and those differ by two orders of magnitude here:

cell	buggy nodes	non-buggy nodes	mean execution / node
`evolved` (MoE)	36%	359	132.8 s
`base` (MoE)	33%	385	89.4 s
`h_treat` (dense)	72%	8	1.2 s
`h_ctrl` (dense)	94%	4	0.3 s

The phase-1 cells were **execution-bound**: their candidates actually trained models, at one to two minutes apiece, and node throughput was limited by sandbox slots. The phase-2 cells are **generation-bound**: their candidates fail in under a second because most of them do not run at all. Comparing nodes per hour across those two regimes does not isolate output length, and the dominant measured difference between the families is not verbosity but *candidate validity* — 33–36% buggy against 72–94%.

What survives is the weaker, still useful statement: **under a wall-clock budget the amount of search a model receives is an outcome, not a constant**, and it varied fivefold across the models we ran. Which model property drives it — output length, candidate validity, or the execution cost of the code produced — is not identified by this measurement, and separating them would need a controlled run we have not done. Anyone reporting a harness or model comparison on NatureBench under a wall-clock budget should publish search-node counts, buggy fractions and execution times beside the scores, because the scores alone do not reveal which regime the run was in. That is why we do.

The practical consequence for us is unchanged: our phase-2 depth limitation (Sec. 3.68) is not a configuration mistake. Anyone reporting a harness or model comparison on NatureBench under a wall-clock budget should publish search-node counts beside the scores, which is why we do.

A correction to a framing we were carrying. We had been considering proposing that OpenRSI's task supply be sourced from retrieval rather than generation, on the strength of our own measurement that model-written tasks carry no gradient. An audit of their code retires that proposal: **OpenRSI does not generate tasks anywhere, at any stage**. Their training supply is already a fixed on-disk manifest of quality-gated Kaggle-derived tasks walked uniformly at random, and the only generation in their loop is of candidate *programs* for a task read off disk. The improvement we were going to propose is already their design, and we record it as a corrected framing rather than a contribution. Two facts from that audit do bear on our own work and are taken up elsewhere: that substituting their RL task supply is a single path change with no call sites touched, because every consumer reads task identity from sample metadata and none constructs it, so the real cost of any substitution is *evaluator* construction rather than plumbing; and that their RL loop runs a group-relative objective at sixteen samples per group over a uniformly sampled fixed task set with no dynamic-sampling filter, leaving the upstream framework's `check_reward_nonzero_std` at `None` in all four launchers, so unanimous groups are retained and contribute exactly zero gradient.

3.70 The proposer stage was silently selecting on length, and the fix is arithmetic

Sec. 3.67 ended on an out-of-memory failure: a single proposal of ∼41,000 tokens needs 38.2 GiB for one backward pass, so the run died at iteration 9. That is worth more than an engineering note, because of *how* it fails. A row that does not fit is not an error the loop reports; it is a row that is absent from the batch. The proposer's median generation is 30,903 tokens and its distribution is wide, so **the proposer stage was training on its short proposals and dropping its long ones** — a selection on exactly the variable the stage is supposed to be learning to control.

Table 64: Chunked loss accumulation, `out/chunked_loss_test.json`. The gradient difference is judged against the same-path run-to-run floor rather than a tolerance chosen to pass: in bf16 with gradient checkpointing, two runs of the identical code already differ, and chunking must not add to that. It does not — it is *below* the floor.

	chunked	unchunked
gradient norm, same batch	0.472435	0.472478
same batch, unchunked run again	—	0.471950
max relative gradient difference, chunked vs unchunked	5.91×10^{-3}	
max relative gradient difference, unchunked vs itself	7.27×10^{-3}	
ratio to the same-path noise floor	**0.81**	
peak memory, 20,000-token row	**73.8** GiB	128.2 GiB
peak memory, 41,000-token row	**97.9** GiB	**did not fit**

Why it is the method's constraint and not this machine's. The peak is set by the log-softmax: a [tokens × 248320] tensor in float32, which is 0.99 MiB per token before anything else. At the observed median proposal that is 29 GiB for one row, on any device, at any batch size, because a sequence cannot be split across a forward pass. Their published description of the loop specifies the three stages, the rewards and the nested sampling, and says nothing about this; **anyone reproducing the loop at a realistic proposal length will hit it**, and will hit it as a silent length filter rather than as a crash if their harness happens to catch the allocation.

The fix. The loss is a sum over positions of $-w_t \log p(x_t)$, so it decomposes exactly over any partition of the positions; what does not decompose is the transformer forward, since every position's activations depend on the whole prefix. So the trunk runs once, its hidden states are detached into a leaf, each chunk of positions materialises its own logits and backpropagates into that leaf, and a single backward then carries the accumulated hidden-state gradient through the trunk. Positions whose weight is zero — the prompt and the padding — are skipped rather than multiplied by zero, which is most of the saving on a short-prompt, long-generation row. The chunk is 2048 positions.

Verified three ways, because any one of them passes a no-op. Equal gradients alone are what an unapplied optimisation produces; a lower peak alone is what a wrong gradient produces; and both on a short row say nothing about the case that motivated the change (Table 64).

So the proposer stage can now train on a proposal of any length the server will generate, and the length selection is gone. The run in Sec. 3.67 that reached iteration 9 reached it because its one trainable proposal happened to be short enough; that is no longer a condition.

This is the same failure as the OpenRSI audit's, seen from the other side. That audit (Sec. 3.69) records that their RL loop runs a group-relative objective at sixteen samples per group over a uniformly sampled fixed task set with the upstream framework's `check_reward_nonzero_std` left at `None` in all four launchers, so unanimous groups are retained and contribute exactly zero gradient. It is taken up here because it is the same phenomenon we measured, with the sign of the design decision reversed.

Of our 22 refused tasks, 12 were solved by every rollout under every scaffold and 9 by none, and those tasks came from the mixed subset — chosen precisely for $0 < p < 1$ on the bare model. The two systems differ only in what they do about it. Ornith's guard G4 *refuses* such a task and pays in yield, which is why fewer than half our iterations form a task group and why the refusal rate

Table 65: Class histogram of the benchmark against the proposer's own corpus, `out/classify_olympiadbench.jsonl` and `out/classify_generated.jsonl`. The two denominators differ and are given: 675 is the benchmark, 693 is the generated corpus, and percentages are of each file's own total. `unclassifiable` is a refusal, not a class: of the benchmark's 121, 100 were sample disagreements and 21 truncations.

	benchmark (n=675)		generated (n=693)	
class	count	share	count	share
`symbolic`	203	30.1%	92	13.3%
`bounded_search`	147	21.8%	**289**	**41.7%**
`unclassifiable`	121	17.9%	64	9.2%
`geometry`	92	13.6%	79	11.4%
`combinatorial`	90	13.3%	169	24.4%
`proof_or_construction`	22	**3.3%**	**0**	**0.0%**
executable reach	532	**78.8%**	629	90.8%
CAS (`symbolic_cas`)	295	**43.7%**	171	24.7%

is the first number we report. A released, reproducible system instead *keeps* it, multiplies it by an advantage of exactly zero, and pays in tokens. **Neither removes the underlying problem, which is that a group is generated before anything can tell you it will be unanimous** — the cost is incurred at generation and only discovered at scoring. What differs is visibility: one design surfaces the cost as a refusal rate, the other reports nothing at all, and the second is the more dangerous presentation because the wasted fraction of every batch does not appear in any number the system prints. Our own polarisation result says the fraction is not small and not stable either: it depends on the scaffold, which the loop is simultaneously training.

3.71 What fraction of a competition benchmark a program can check, and what a self-generated corpus turns out to be a sample of

The setting. Every verification result in this project is bounded by a question we had not measured: how much of the benchmark is checkable by machine at all. The answer decides a build. If proof-or-construction problems are a large share, a proof assistant is the only way to reach them and Lean has to be built; if they are a small share, the effort belongs in a symbolic backend that serves the majority. The recommendation to build the symbolic backend and leave Lean unbuilt was made on this measurement, so it is recorded with it.

The configuration. Served model `qwen38-27b`, 3 **samples per problem, and a class is assigned only on unanimity** — any disagreement, and any truncation, falls to `unclassifiable` rather than to a majority vote. The taxonomy and the class-to-backend map are `reimpl/ornith_repro/classify.py`; the run is `reimpl/classify_benchmark.py`. "Executable reach" is not a judgement, it is exactly the set of classes for which `backend_for(cls)` is not `None` (`bounded_search`, `combinatorial`, `symbolic`, `geometry`); "CAS" is exactly the `symbolic_cas` backend (`symbolic`, `geometry`).

The result, and the build decision it carries. 532 of 675 benchmark problems (78.8%) are in executable reach and 295 (43.7%) are claimed by a computer algebra system, against 22 problems (3.3%) that are proof-or-construction (Table 65). **A proof assistant is the only route to** 3.3%

of this benchmark, and a symbolic backend is a route to 43.7% **of it.** That ratio, better than thirteen to one, is why Lean is unbuilt and why the symbolic backend was written instead. The figure is an upper bound on what any executable checker can decide here and a matching lower bound on how little a proof assistant would add, and both are what a build decision needs.

The finding that was not the reason for the run: a self-generated corpus is not a sample of the domain it imitates. Run against the proposer's own 693 tasks the same classifier returns **zero proof-or-construction problems** — not few, none — against the benchmark's 3.3%, and 41.7% bounded search against the benchmark's 21.8%, a factor of **1.91**×. The proposer was never asked for a distribution and never told to avoid proofs as such; it was asked for "exactly ONE correct final answer, a specific closed form". **So the answer format, and not the difficulty target, is what selected the mathematics.** A requirement written to make grading possible silently deleted an entire class of problem and nearly doubled another, and it did so without appearing anywhere in the loop's reward, its difficulty gate or its novelty term. Any self-improvement loop that grades on a final answer inherits this, and inherits it invisibly: the corpus it trains on drifts toward what its grader can read, which is a different object from the domain it believes it is sampling.

Caveats.

- The classifier is the same model that wrote the generated corpus, so a shared blind spot between writer and classifier would be invisible here. The unanimity-over-three rule bounds per-sample noise, not a systematic bias shared by both roles.
- 17.9% of the benchmark is `unclassifiable` and is excluded from both derived rows. If those 121 problems were disproportionately proofs, 3.3% is an underestimate; the run does not settle that, and the honest reading of the build decision is that proof-or-construction is between 3.3% and 21.2% of the benchmark, still far below the CAS share.
- The denominators differ (675 against 693) and no attempt is made to match them; the comparison is between two corpora, not two arms of one experiment.

3.72 How wrong are self-written answer keys: three numbers, two instrument failures, and one case that needed no external check

The setting. A self-improvement loop with no external answer key trains on keys the model wrote itself. Sec. 3.65 caught a single mis-keyed task and preserved its witness; this asks the rate, and asks it against the only comparator that makes the number mean anything — *the same verifier run against professionally curated keys.* Without that arm a generated-key error rate is uninterpretable, because it confounds how often the generator is wrong with how often the verifier is.

The arc is reported rather than the endpoint, because the corrections are the useful part. The number went 72.5%, then 12.9%, then 8.60%, and each move was an instrument failure rather than new evidence about the generator.

The configuration. Served model `qwen38-27b`, context 65536, `reasoning_effort=low`, run by `reimpl/verify_generated.py` over the cascade in `reimpl/ornith_repro/verify.py`. Verdicts are three-valued (`verified` / `refuted` / `unverifiable`) on top of the three-valued comparator `symbolic_compare`, and **difference must be proved** — by disjoint Arb enclosures or an exact structural mismatch — while equality may rest on evidence. A refutation additionally requires

Table 66: Every run of the key check, in order. "Decided" is verified + refuted; coverage is decided over the file's own row count; the error rate is refuted over decided. The two rows in bold are the like-for-like pair. Counts are given beside every rate because 16 of 244 and 6.56% read very differently.

pool	file	rows	ver.	ref.	decided	coverage	error
generated	`verify_generated.jsonl`	634	11	29	40	6.3%	72.50%
generated	`verify_generated_v2.jsonl`	693	230	34	264	38.1%	12.88%
generated	`verify_generated_sym.jsonl`	693	228	16	244	35.2%	6.56%
generated	`verify_generated_cov.jsonl`	693	287	27	314	**45.3**%	**8.60**%
published	`verify_olympiad.jsonl`	675	172	49	221	32.7%	22.17%
published	`verify_olympiad_sym.jsonl`	240	40	3	43	17.9%	6.98%
published	`verify_olympiad_sym2.jsonl`	675	229	6	235	34.8%	2.55%
published	`verify_olympiad_cov.jsonl`	675	268	8	276	**40.9**%	**2.90**%

a substantiated witness. `ERROR` rows are excluded from "decided" but retained in the coverage denominator.

The three numbers, and what each correction taught. 72.5% — *a truncation bug that deleted verifications and spared refutations.* The first run decided only 40 of 634 rows and called 29 of them wrong. The generation cap truncated the verifier's own program-writing, and truncation is not symmetric across verdicts: a run that gets far enough to *refute* has already produced its witness, while a run that would have *verified* is cut off before it can and is recorded as unverifiable. The bug therefore removed the denominator's successes and kept its failures, which is the worst possible shape for a rate. **A coverage of** 6.3% **was the visible symptom and it was sitting in the table beside the number the whole time.**

12.9% — *a real number that could not be interpreted, because the comparator was broken.* With the truncation fixed, coverage rose to 38.1% and the generated-key error rate fell to 12.88%. It was uninterpretable: the same instrument scored *published* keys at 22.17%, and a comparator that refutes better than a fifth of professionally curated answers is measuring its own formatting intolerance. 12.9% sat *below* that floor. The lesson is the one this project keeps relearning — **a treatment number is not a result until the control number is credible** — and the fix was the exact three-valued comparator, which must *prove* difference rather than fail to prove equality.

6.56% *against* 2.55%, *then* 8.60% *against* 2.90%. With the exact comparator the published-key floor drops to 2.55% (6 of 235) and the generated rate to 6.56% (16 of 244). After the coverage work of Sec. 3.73 the like-for-like pair is **8.60**% (27 of 314, coverage 45.3%) against **2.90**% (8 of 276, coverage 40.9%). **Self-written keys are wrong about three times as often as published ones**, and the residual 2.90% is the instrument's own floor rather than a property of the generator.

Reading all sixteen refutations individually established it is the generator, not the instrument. This was done on `verify_generated_sym.jsonl`**, the** 6.56% **run, which is the file that contains exactly sixteen refutations**; the 8.60% run contains twenty-seven and was not read exhaustively. We record that distinction because attaching "we read all sixteen" to the 8.60% figure would overstate what was audited.

The decisive case needs no external check at all. The proposer wrote the same problem **three times** and asserted **three different answers**:

source file	statement	asserted
`gen_tasks_main.jsonl`	"…ordered pairs (a, b) of positive integers with $a \le b \le 100$ such that $a + b$ divides $a^2 + b^2$"	182
`gen_tasks_main.jsonl`	"…with $1 \le a \le b \le 100$ such that $a + b$ divides $a^2 + b^2$"	100
`gen_novel.jsonl`	"…with $1 \le a \le b \le 100$ such that $a^2 + b^2$ is divisible by $a + b$"	86

The three statements are mathematically identical — "positive integers" already forces $a \ge 1$ — and **the verifier returned** 186 **for all three, in every run**. We confirmed independently by exhaustive enumeration that 186 is correct. **Two of these came out of the same file asserting** 182 **and** 100 **for the same question**, so no external authority is required to know the generator is wrong: *it contradicts itself*, and a consistent verifier agrees with neither. That is the cleanest available separation of "the generator writes wrong keys" from "the checker misreads right ones", and it is why the witness field is mandatory in the task store.

Caveats.

- **Both rates are conditional on being decided**, at 45.3% and 40.9% coverage. The undecided majority is not known to be correct. If keys the verifier cannot read are disproportionately wrong, both rates are underestimates, and there is no reason to assume the unreadable half resembles the readable one.
- The two pools have different coverage (45.3% against 40.9%), so the 3× ratio compares two conditional rates over differently selected subsets, not two matched samples.
- Sixteen and six refutations respectively: the Wilson interval on 6/235 reaches past 5%, so the *ordering* is solid but the multiplier is not resolved to better than about a factor of two.
- The exhaustive-enumeration confirmation of 186 was run by us, outside the loop's own verifier, and is the one place in this section where an external check was used.

3.73 Tripling the program budget: coverage rises on both pools, and two claims we had been carrying do not survive their own artifacts

The setting. The key check in Sec. 3.72 abstains on most of what it sees, and an abstention is not a neutral outcome: it removes a task from every rate computed downstream. The question is whether abstentions are a *decision* the verifier is making or a *budget* it is running out of. If the latter, more tokens buy coverage at no cost to soundness, because the model's opportunity to emit a program changes while the rule that judges the program does not. This is the fifth instance in this project of a budget sized for the visible output while the model reasons first, and it belongs with the other four.

The configuration. `qwen38-27b`, context 65536, effort `low`. The post-fix runs use `max_new_tokens`= 12288 in both `solve_structured` and `check_witness`. Failure buckets are assigned by `classify_reason()` in `reimpl/verify_generated.py` over the free-text `detail` field.

The result: coverage rises on both pools. 34.8% → 40.9% on published keys and 35.2% → 45.3% on generated ones (Table 67). On published keys the largest bucket, `no_program_produced`, falls 272 → 156, a 42.6% **cut**, and that is the sense in which the budget was the binding constraint: those are cases where the model never emitted a checkable program, not cases where it emitted one and the program lost.

Table 67: Coverage and the abstention buckets, before and after. "Before" is `_sym/_sym2`, "after" is `_cov`. Coverage denominators are the files' own row counts (693 generated, 675 published). Buckets are counted over abstentions only, so the columns sum to the abstention totals (446 → 376 and 439 → 398).

	generated keys		published keys	
	before	after	before	after
decided	244	314	235	276
coverage	**35.2**%	**45.3**%	**34.8**%	**40.9**%
error rate among decided	6.56%	8.60%	2.55%	2.90%
`no_program_produced`	140	**49**	**272**	**156**
`bound_falsified`	**181**	**205**	43	54
`bound_rerun_failed`	59	47	32	30
`other`	30	37	46	83
`witness_rejected`	22	16	15	27
`round_trip_mismatch`	5	11	12	9
`no_witness`	5	6	4	6
`programs_disagree`	4	5	12	21
`bound_unjustified`	—	—	3	12

Two claims we had been carrying, corrected against the artifacts. Both were stated in a form that the files do not support, and both fail in the same way — a statement true of one pool asserted of both.

"The largest failure bucket was cut by 43%*" is true of the published pool only.* On the *generated* pool the largest bucket before the fix was not `no_program_produced` (140) but `bound_falsified` (181), and it **rose** to 205, up 13.3%. The 65% drop on the generated pool is in `no_program_produced` (140 → 49), which was the second bucket, not the first. The defensible statement is the pool-specific one.

"The dominant bucket moved from no-program-produced to cannot-justify-its-bound" is false on both pools. On published keys `no_program_produced` is dominant before (272, 62.0% of abstentions) *and after* (156, 39.2%); even pooling all three bound-family buckets gives 96, still well below 156. On generated keys `bound_falsified` was *already* dominant before the fix, so nothing moved — it merely widened. **What the artifacts do support**, and what we should have said, is a shift in the ratio: on the generated pool bound-failures to no-program-failures goes from 1.29× to 4.18×, and the bound family rises from 53.8% to 67.0% of abstentions; on the published pool `no_program_produced` falls from 62.0% to 39.2% of abstentions while remaining the largest. The direction of the original claim is right and its statement was wrong, and the class a symbolic solver dissolves is indeed the growing one on the pool that matters — but it did not become dominant, it already was.

"Without weakening soundness" is not established, and one piece of evidence points the other way. The argument was that only the model's chance to emit a program changed, not the decision rule. We can no longer assert that from these artifacts:

- `reimpl/` is not under version control, and `verify_sound.py` — the file holding the decision rule — has an mtime of 2026-09-05 03:02, **between** the before-runs (ending 23:03 on 09-04) and the after-runs (04:07 and 05:30 on 09-05). **So "the decision rule did not change" cannot be checked**, and we record that as a hole in our own apparatus rather than as a premise.

- Supporting but not conclusive: the decision-rule message vocabulary is byte-identical across all four runs, the comparator states are the same three, and `bound_scale`= 10 in every run. `verify.py` and `symbolic.py` both predate all four runs.
- **The refutation rate among decided rose in both pools**, 6.56% → 8.60% and 2.55% → 2.90%. Coverage that arrives with a higher refutation rate is consistent with newly-decided cases being genuinely harder, and it is also consistent with a looser rule; these files do not separate the two. If the published-key rate is being used as the instrument's error floor, that floor moved from 2.55% to 2.90% and every generated-key rate must be read against the floor from its own run.

Caveats. The "tripling" is consistent with the current code ($12288 = 3 \times 4096$) but the pre-fix constant is not recorded anywhere we could find, so the factor is inferred, not measured. Wall-clock rose 20% on the generated pool and 77% on the published one, which is the cost side of the trade and is not proportional to the coverage gained. And the lesson generalises past this run: the version-control hole is why the task store's records carry their backend and comparator *in the row*, so that a later reader never has to date a claim by a file mtime.

3.74 The informative fraction, and the difference between "not unanimous" and "carries gradient"

The setting. The headline number for the generated arm has been that generated tasks reach the informative region 37.9% of the time against the fixed pool's 25.7%. It is a real measurement and it has been quoted in a stronger sense than it supports, because *informative* was operationalised permissively. This section reports the figure, the strict reading beside it, and the reconciliation, since the two differ by construction rather than by noise.

The two definitions. `scarcity.py` implements $1 - P(\text{unanimous})$: **any** group that is not unanimous counts. That is the weakest possible statement of "carries gradient". A group at one success in sixteen is not unanimous, but under a binary reward every group-relative estimator reduces to $\alpha(k)\,(r - \bar{r})$, so its advantage vector has fifteen entries of one sign and one of the other and the update is carried by a single rollout that vanishes on resampling. The strict reading, fixed in `PREREG_anchor_difficulty.md` before it was computed, requires **at least two successes and at least two failures** in the drawn group: the advantage vector then has at least two entries of each sign and the group survives the loss of any one sample. Both are computed exactly by the same hypergeometric argument over the same problems with the same truncation handling, in `strict_informative.py`, so the only thing that differs between the columns is the definition:

$$P_{\text{perm}} = 1 - \frac{\binom{c}{G} + \binom{n-c}{G}}{\binom{n}{G}}, \qquad P_{\text{strict}} = \sum_{k=2}^{G-2} \frac{\binom{c}{k}\binom{n-c}{G-k}}{\binom{n}{G}}.$$

The result: the strict reading roughly halves the number, and most of the advantage with it. The generated pool falls from 0.379 to **0.203** and the fixed pool from 0.257 to **0.165** (Table 68). **The gap survives but shrinks from** 12.3 **points to** 3.7, and the multiplier from 1.48× to 1.23×. So the honest form of the headline is: generation supplies more non-unanimous groups than the fixed pool, and about half of that surplus is groups so lopsided that a group-relative objective can do almost nothing with them. **The permissive number is the one that was quoted**, and it should not be quoted again without the strict one beside it.

Table 68: Informative fraction under both readings, $G = 16$, problems with ≥ 16 resolved rollouts, `out/strict_informative.json`. Denominators are problems, and they differ between pools for the reason given in the caveats. "bigcap" re-runs the same pools at a larger generation cap. The generated bigcap row is the *completed* run (11,088 rollouts over 693 problems); it was first computed from that run while it was still in flight, which gave 0.415 on 270 problems, and that partial value is superseded here.

pool	n prob.	permissive	strict	strict/perm.	always solved
generated (`blocks_generated`)	622	**0.379**	**0.203**	0.534	347
fixed pool (`blocks_low`)	230	**0.257**	**0.165**	0.644	164
generated, bigcap	677	0.383	0.235	0.614	369
fixed pool, bigcap	240	0.283	0.167	0.588	163

Reconciliation with the 52-task measurement. Sec. 3.65 reported three of 52 accepted tasks in the mixed band, all three from retrieval. Those were measured at $k = 5$, and the three are 3/5, 2/5 and 3/5 — so **under the strict definition all three still qualify, and strict equals permissive at $3/52 = 0.058$ there**. The two measurements are therefore not in tension: the strict-versus-permissive gap is a property of large k, where a 1/16 group can be counted as informative, and it closes at $k = 5$ where 1/5 is the only lopsided non-unanimous state available. The 52-task figure remains the harsher number by a wide margin, because it measures *accepted, model-written* tasks rather than a pool of benchmark problems.

Caveats, and two are large.

- **Unequal exclusion, and it cuts the wrong way.** Truncated rollouts are dropped rather than scored wrong, and problems below 16 resolved samples are skipped. The fixed pool loses 108 of 338 problems that way (32%; 21.0% of its rollouts truncated), the generated pool only 71 of 693 (10%; 2.1% truncated). Truncation correlates with difficulty, so the problems dropped from the fixed pool are disproportionately its hard ones, which biases the fixed pool's informative fraction *downward* and inflates the gap. The denominators are 622 against 230 and are not matched.
- **Neither number is a cap-independent property of a pool.** At the larger cap the same fixed problems give 0.283 permissive and the generated pool 0.383. The gap survives the cap change; the individual values do not, and 25.7% should always be quoted with its cap.
- The two pools were run at matched caps for the headline comparison (n=16, `max_tokens` 16384, effort `low`), which is what makes that one comparison fair even though the denominators are not matched.
- The strict threshold of two is a choice, pre-registered but not derived. It is the smallest value that makes the criterion robust to the loss of any single rollout; a larger threshold would lower both columns further and is not reported.

3.75 Grouping granularity: a pre-registered sweep with an interior optimum, and the point where significance and usefulness part company

The setting. Sec. 3.65 left weakness-targeted generation without a grouping to target. A curriculum that aims at weakness needs groups whose weakness actually differs, and this project

has twice retired a component — the learned router, and the MEDS clustering — for failing exactly that test: a learned structure that made real distinctions made them no better than a size-matched random structure of the same shape. `PREREG_grouping_gate.md` was written before any statistic below existed and fixed the statistic, the null, the arms and the decision rule. It named four outcomes in advance, and the one that occurred — fine passes, coarse does not — was assigned the action "proceed to weakness-ranked generation on fine labels".

The registered claim is narrow and worth restating, because the machinery is shared with a closed result: this asks whether a grouping identifies *differential weakness*, where MEDS was retired for failing to show that behavioural clusters have less *gradient conflict* than a random partition. Same test, different question, and this section is not a rehabilitation of that one.

The configuration.

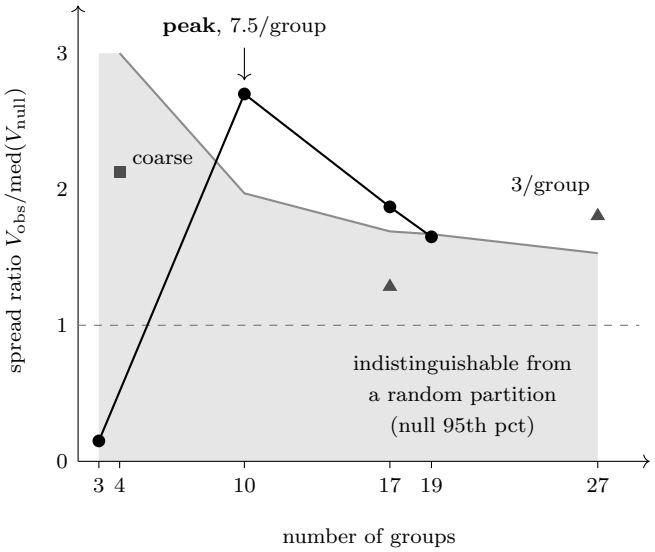
served model	`qwen38-27b` (Qwen/Qwen3.8-27B), `--tp 1 --dp 8`, context 65536
probe	105 problems, from the 118-problem probe of the search half
inclusion rule	≥ 8 *resolved* rollouts; truncated generations dropped, not scored wrong
outcome source	`out/blocks_low.jsonl` (n=16, `max_tokens` 16384)
	+ `out/blocks_low_more.jsonl` (n=16, `max_tokens` 24576)
reasoning effort	`low`, identical for every rollout in both blocks
labels	`out/fine_labels.jsonl`: 3 samples/problem, **unanimity required**
permutation draws	2000 per arm, seed 20260905, each arm against its own size-matched null
probe split	`probe_frac`= 0.35, seed 20260904, stratified by subfield

The method. The statistic is the weighted between-group variance of per-group accuracy, $V = \sum_g (n_g/N)\,(\mathrm{acc}_g - \mathrm{acc}_{\mathrm{pool}})^2$, computed in `spread_gate.weighted_between_variance`. The null permutes the *assignment* of problems to groups while holding every problem's measured $\hat{p}$ fixed, which is what makes it a fair reference: small groups inflate observed variance through sampling noise, and a size-matched permutation carries exactly the same inflation, so the comparison is against noise of the right shape rather than against zero. Effect size is V_{obs} over the null median; p is the one-sided fraction of draws at or above V_{obs}.

Granularity is swept by varying the pooling threshold — groups smaller than `min_size` are merged into a single `other` before anything is computed — and by crossing the topic label with the answer type. **Granularity is never varied using accuracy.** Groups are built from the topic label, the answer type and the pooling threshold and from nothing else; partitioning on the very quantity whose spread is being tested would manufacture the result. The sweep is `reimpl/granularity_sweep.py`, the gate itself `reimpl/spread_gate.py`, and the curve is `out/granularity_sweep.json`.

The result. The curve has an interior maximum (Fig. 2). **The four coarse subfields fail**: ratio 2.13 at $p = 0.148$, and their failure was predicted in writing before the run. **Ten fine topic labels pass and are the peak**: ratio 2.70 at $p = 0.009$, a median of 7.5 problems per group. Beyond the peak the effect size falls monotonically — 1.87 at seventeen groups, 1.65 at nineteen — and below the peak it collapses, with three groups landing at 0.15, which is *less* spread than a random partition of the same shape. Two points do not make a curve and this is why the sweep was run: the two arms the pre-registration named would have shown a rise from 2.13 to 2.70 and said nothing about where it stops.

The finding: significance and usefulness diverge, and the null band is the mechanism. At twenty-seven groups $p = 0.006$ *beats* the peak's $p = 0.009$ while the effect size has fallen from



scheme	grps	/grp	ratio	p
fine topic	3	19.0	0.15	0.900
coarse subfield	4	22.5	2.13	0.148
fine topic	**10**	**7.5**	**2.70**	**0.009**
fine topic	17	5.0	1.87	0.024
topic × type	17	5.0	1.29	0.219
fine topic	19	4.0	1.65	0.053
topic × type	27	3.0	1.81	**0.006**
excluded, degenerate:				
fine topic	1	105	∞	1.000

Figure 2: Spread ratio against grouping granularity, 105 probe problems, 2000 permutation draws per arm, seed 20260905. The shaded region is each granularity's own null 95th percentile expressed as a ratio, so a point inside it is indistinguishable from a size-matched random partition; at three groups that band runs off the top of the axis at 4.25. The band *narrows* as groups get finer — 3.00 at four groups, 1.97 at ten, 1.53 at twenty-seven — which is the whole of the effect described below. Circles are the fine-topic arm at five pooling thresholds, the square is the four coarse subfields, triangles cross the topic label with the answer type. The dashed line at 1 is "no structure".

2.70 to 1.81 and each group holds three problems. Both are valid tests — the null is size-matched by construction, so the p-value is honest at every granularity — but they are not equally useful, and **a reader who ranks these partitions by p picks the one a curriculum cannot act on**. "Generate more problems like these three problems" is not a category; it is three problems.

The reason is visible in Fig. 2 and is worth stating as the general lesson. The null's 95th percentile, in ratio units, is 3.00 at four groups, 1.97 at ten and 1.53 at twenty-seven: **the bar a partition must clear falls as the partition gets finer**, because more groups means more of them and their mean spread is estimated more tightly. So the twenty-seven-group arm is significant not because its structure is stronger but because its null is narrower. **The p-value answers "is this more than noise", and the effect size answers "is there enough of it to aim at"; only the second is what a curriculum needs, and the two are ordered differently here.** This is why the sweep reports the median problems-per-group beside every point, and why we report both columns rather than the one that sorts most favourably.

A defect in our own apparatus, recorded with its mechanism. The first pass of this sweep reported **a single group of** 105 **problems as the optimum**. The arithmetic is correct and the conclusion is nonsense: pooling at `min_size`= 20 leaves one surviving group, one group has zero between-group variance *by definition*, its permutation null is also identically zero, and 0/0 was taken through the ratio as ∞. A maximum over effect size then selected it. The failure mode is the one this project keeps finding — a number that is an artefact of the instrument being read as a measurement — and it is dangerous here specifically because ∞ sorts first and $p = 1.000$, the strongest possible disconfirmation, sits in the adjacent column being ignored. `granularity_sweep.py` now excludes partitions with fewer than two surviving groups or a non-finite ratio before any maximum is taken,

and reports a second maximum restricted to partitions of at least five problems per group. Both maxima land on the same row, ten fine topics at 2.70, which is the only reason the headline is unchanged.

Caveats.

- **The clustering arm never ran.** Arm 3 of the pre-registration — embedding clusters at k matched to the label count — was not executed. **No claim is made that labels beat clusters**, and the pre-registration's tie-break rule in favour of labels was never invoked because there is nothing to break a tie against.
- **The probe is 105 problems.** Every group at the peak holds a median of 7.5 of them. The ordering of the interior points is not resolved by this probe.
- **Ten versus seventeen is not itself tested.** The sweep establishes that the curve has an interior maximum and that it is at ten among the granularities tried; it does not establish that ten differs significantly from seventeen. Each arm is tested against its own null, not against the other arms, and no multiplicity correction is applied across the eight arms because the pre-registered decision concerned the fine-versus-coarse contrast only.
- **Two of the points come from a different labelling scheme.** The seventeen-group comparison is instructive precisely because both schemes reach seventeen groups and disagree: fine topic gives 1.87 at $p = 0.024$, topic × answer type gives 1.29 at $p = 0.219$. Granularity is therefore *not* a sufficient description of a partition, and the curve should not be read as a function of the group count alone.

What this implies for an evolving taxonomy, which is the reason the sweep was run. The curve says structure becomes usable somewhere around five to eight problems per group and is gone by three. A taxonomy that splits a mastered topic in two, at this probe size, turns a 7.5-problem group into two groups of about 3.8 — **below the point where the curve says the structure stops being actionable**, and into the regime where the test still passes. So an evolving taxonomy is not blocked by needing better splitting logic; it is blocked by probe size, and the honest next step is a larger probe rather than a cleverer split. That conclusion is a consequence of the shape of Fig. 2, which is why the figure is here and why it could not have been replaced by its table.

3.76 A second seed measures the noise floor, and it is the size of the effect

Sec. 3.66 reported one seed and declined to interpret a one-task difference. A second independent replicate at seed 43, identical in every other respect, now says how much of that was scatter. This is the measurement the replicate existed for.

Match-SOTA	seed 42	seed 43	pooled
evolved, `Frontis-MA1-35B`	60%	60%	12/20 = 60%
base, `Qwen3.6-35B-A3B`	70%	50%	12/20 = 60%
delta	−10 pt	+10 pt	**0 pt**
their published delta		+20 pt (50% → 70%)	

The finding. The base cell moved 20 points between two seeds — 70% to 50% — which is exactly the size of the effect OpenRSI reports. The evolved cell was stable at 60% across both. So the run-to-run noise floor on a single cell of this benchmark is as large as the published difference between two cells, and the sign of our measured delta flips between seeds (−10 pt, then +10 pt) purely from reseeding.

This converts the argument of Sec. 3.57 from a power calculation into a measurement. There we showed *a priori* that at $n = 10$ nothing short of 10/10 separates from a 5/10 baseline. Here we observe directly that the instrument's own scatter spans the claimed effect. Pooled over both seeds the Match-SOTA model-swap delta is *exactly zero*, 12/20 against 12/20, where the published claim is +20 points.

The one signal that replicates, reported in OpenRSI's favour. Surpass-SOTA — their stricter metric, $g > 0.1$ — moves in the claimed direction in *both* seeds and by the same margin each time: 40% against 20% at seed 42, and 20% against 0% at seed 43, pooling to 6/20 against 2/20. That is +20 points twice from independent draws, and it is the only quantity in this reproduction that behaves the way their claim predicts. We report it because it is real and because it would be easy to omit having led with a null. It is not significant ($p = 0.235$) and the counts are tiny, and because both seeds run the same ten tasks the twenty trials are clustered rather than independent, so the effective sample is nearer ten. But the direction is consistent, and a reader deciding whether their evolved model does anything should weigh a replicated directional signal on the strict metric against a pooled zero on the permissive one.

What we therefore claim, and what we do not. We do not claim to have refuted OpenRSI's model-swap result. Two seeds on ten tasks cannot, and our own numbers reproduce neither their base (50%; we see 70% and 50%) nor their evolved figure (70%; we see 60% twice). We claim something narrower and, we think, more useful: **that this benchmark at its published protocol — ten tasks, one sample each — has a per-cell noise floor of about twenty points, and therefore cannot support a twenty-point claim from single runs.** That is a statement about the measuring instrument, it is measured rather than argued, and it applies to anyone using NatureBench Lite at this size, ourselves included. The remedy is not a better model but more samples per task, and their harness already exposes `n_samples_per_task` to provide them.

3.77 Which metric you choose decides the answer, and Match-SOTA is the worst of the four

Sec. 3.76 reported a pooled zero on Match-SOTA and a replicated +20 points on Surpass-SOTA. Asking why one metric moves and the other does not turns out to be the most useful thing in this reproduction, and it is a criticism of the headline metric rather than of either model.

Where the instability lives. Comparing the two seeds task by task, **four of ten tasks flip their Match-SOTA classification between seeds, in both cells**. The flips are not spread over the range; they are concentrated at the threshold. `s41467-025-63412-3` moves from $g = -0.004$ to $g = +0.007$, and that 0.011 change in a continuous score flips a full ten points of Match-SOTA. `s42256-023-00627-3` moves from −0.015 to +0.059, likewise across zero. Match-SOTA thresholds g at exactly 0, and $g = 0$ is where the mass of this task distribution sits — six of ten tasks land within ±0.09 of it. **The metric discretises the score precisely where the data is densest, so it converts small continuous noise into maximal binary noise.**

Four metrics, one disagreeing.

metric (evolved − base)	seed 42	seed 43	consistent?
Match-SOTA ($g \geq 0$, a count)	−10 pt	+10 pt	**no, flips**
Surpass-SOTA ($g > 0.1$, a count)	+20 pt	+20 pt	yes
mean g (continuous)	+0.0328	+0.0576	yes
median g (continuous)	+0.0064	+0.0147	yes

Three of the four favour the evolved model in both seeds. The one that does not is the one OpenRSI leads with, and it is the one whose threshold sits in the densest part of the distribution.

But the advantage is in margins, not in wins, and it is not significant. It would be easy to stop there and report a vindication. A paired test, with the task as the pairing unit, does not support one. Evolved beats base on 6 of 9 scored tasks at seed 42 and 5 of 9 at seed 43; pooled, 11 of 18 task-seeds, sign-test $p = 0.48$. So the evolved model does *not* win on more tasks. It wins by *larger margins* on the tasks where it wins, which is exactly why the magnitude-sensitive metrics (mean g, and Surpass-SOTA, which requires clearing 0.1) favour it while the count at zero does not.

What we take from this. Two things, and neither is a verdict on OpenRSI's model. First, **a reader of this benchmark should not use Match-SOTA alone**: it is the least stable of the four summaries available from the same data, for a structural reason that will recur on any task set whose scores cluster near the SOTA reference. Reporting mean g beside it costs nothing, since the evaluator already returns it. Second, our own headline in Sec. 3.76 — a pooled zero — is a statement about the *count* metric, and we have been careful to say so there rather than letting it stand as a claim that the evolved model does nothing. On the magnitude metrics it consistently does something, in both seeds, and that something is not statistically resolvable at ten tasks.

Reproducibility notes

Every number in Sec. 3 names the run that produced it. Benchmarks are scored with the same grader across arms; the grader's own bias was measured and corrected once (an earlier version discarded unbalanced final boxes, which penalised degraded checkpoints specifically). Intervals are reported on differences, paired by item, never as independent per-arm intervals.